

Control Is Motion

A New Framework for Nonlinear Systems
That Move Through the World

Preface

This is Volume II of *The Geometry of Motion*. Where Volume 0 laid the kinematic and algebraic groundwork (rigid-body transforms, screw theory, spatial algebra, and the Lagrangian formulation) and Volume I developed the tangent-space machinery for linearization, contraction, and optimal control, this volume turns to the central thesis of the series: *control is motion, not destination*. We take seriously the observation that many of the systems we care about—athletic motions, legged locomotion, aerobatic maneuvers, manipulation through contact—are not well described by a setpoint-regulation problem. They are trajectory-tracking problems on curved, underactuated manifolds, and they must remain robust to execution noise.

The volume is organized around three concentric questions. First, what is the *right* mathematical object to control? We argue it is a trajectory tube, not a point. Second, how do we *synthesize* such trajectories? We develop trajectory optimization, funnel synthesis, and phase-variable control as complementary tools. Third, how do the resulting techniques connect to biological motor control and learning? We close with stochastic motor control and a comprehensive case study of the golf swing that weaves all preceding threads together.

Prerequisites. Readers are assumed to be comfortable with the material of Volume 0 (rigid-body kinematics, Lagrangian mechanics) and Volume I (linearization, LQR, contraction metrics). Some familiarity with differential geometry at the level of tangent spaces and vector fields is assumed; the nomenclature table and Volume I appendix provide the minimum required background.

Relationship to Volume III. Volume III will extend these trajectory-centric tools to networked and multi-agent systems. The core insight—that motion, not destination, is the natural object of synthesis—carries forward.

Contents

Preface	3
Nomenclature	9
1 Throwing Away the Target	11
1.1 The Wrong Question	11
1.2 Why Setpoints Are a Special Case	12
1.3 Anatomy of a Motion: The Golf Swing as Prototype	14
1.4 The Mathematical Landscape	15
1.4.1 Phase Variables and Time Reparameterization	15
1.4.2 Transverse Dynamics and Orbital Stability	16
1.4.3 Funnel Control and Time-Varying Tubes	17
1.5 What Changes When Control Is Motion	17
1.5.1 Stability Is Not Convergence—It Is Confinement	17
1.5.2 The Cost Function Is a Functional, Not a Function	18
1.5.3 Feedforward Is Not Optional—It Is Primary	18
1.5.4 Underactuation Creates Geometry, Not Just Constraints	19
1.6 A Roadmap for the Book	19
1.7 A New Language for Control	20
1.8 Exercises	21
2 Curves in State Space	22
2.1 Why Geometry Comes First	22
2.2 Parameterized Curves	23
2.2.1 Velocity, Speed, and the Tangent Vector	24
2.3 Arc Length: The Natural Parameter	24
2.4 Curvature: How Curves Bend	25
2.4.1 Curvature in Arbitrary Parameterization	25
2.4.2 Why Curvature Matters for Control	26
2.5 The Frenet–Serret Frame	27
2.5.1 Construction in $\mathbb{R}^n$	27
2.5.2 The Frenet–Serret Equations	28
2.5.3 The Moving Frame in State Space	29
2.6 Curvature and Torsion in the Language of Control	29
2.6.1 Curvature as a Demand on the Controller	29
2.6.2 Torsion as Out-of-Plane Complexity	30
2.7 Reparameterization and Timing Laws	31
2.7.1 The Path–Timing Decomposition of Control	32
2.8 The Osculating Objects	33
2.9 The Geometry of Trajectory Deviations	33
2.9.1 Frenet–Serret Coordinates	33
2.9.2 Curvature-Induced Coupling	34
2.10 Fundamental Theorem and Uniqueness	34

2.11	Summary	35
2.12	Exercises	36
3	Configuration Manifolds	38
3.1	The Lie That Coordinates Tell	38
3.2	Smooth Manifolds: The Minimum You Need	39
3.2.1	Tangent Spaces and Tangent Vectors	40
3.3	The Configuration Manifolds of Joints	41
3.4	The Rotation Group $SO(3)$	42
3.4.1	Why Euler Angles Fail	42
3.4.2	The Lie Algebra $\mathfrak{so}(3)$	43
3.5	The Golf Swing Configuration Space	44
3.5.1	A Simplified Kinematic Model	44
3.5.2	Why This Matters for Control	44
3.6	Riemannian Metrics and the Mass Matrix	45
3.6.1	Geodesics as Natural Trajectories	46
3.7	Curves on Manifolds Revisited	47
3.7.1	Covariant Differentiation	47
3.7.2	Manifold Curvature of Trajectories	48
3.8	Error Metrics on Manifolds	48
3.8.1	The Logarithmic Error	49
3.8.2	The Trajectory Tube on a Manifold	49
3.9	Equations of Motion on Manifolds	49
3.10	Practical Implications for Controller Implementation	50
3.11	Summary	51
3.12	Exercises	51
4	Orbital Stability and Transverse Linearization	54
4.1	The Problem with Lyapunov	54
4.2	Orbital Stability: Formal Definitions	55
4.3	The Transverse–Tangential Decomposition	56
4.3.1	Projection onto the Orbit	57
4.3.2	The Moving Hyperplane Decomposition	57
4.4	Transverse Linearization	58
4.4.1	Deriving the Transverse Dynamics	58
4.4.2	The Transverse Jacobian: A Practical Formula	59
4.5	Floquet Theory: Stability of Periodic Orbits	59
4.5.1	The State Transition Matrix	60
4.5.2	The Monodromy Matrix	60
4.6	Moving Poincaré Sections	61
4.6.1	Moving Poincaré Sections for Non-Periodic Trajectories	61
4.7	Controller Design via Transverse Stabilization	63
4.7.1	Time-Varying LQR for Transverse Stabilization	63
4.7.2	Phase-Varying Gains and the Funnel Connection	64
4.7.3	The Complete Tracking Controller Architecture	64
4.8	Robustness of Orbital Stability	65
4.9	Summary	66
4.10	Exercises	66

5	Underactuation and Passive Dynamics	69
5.1	The Loss of Control Authority	69
5.2	The Double Pendulum: A Window into the Golf Swing	70
5.3	The Annihilator and the Null Space	72
5.4	Partial Feedback Linearization	72
5.5	Accessibility, Chow's Theorem, and the Orbit Theorem	73
5.6	Zero Dynamics and the Golf Swing Funnel	73
5.7	Abnormal Extremals and Singular Arcs	74
5.8	Summary	75
5.9	Exercises	75
6	Trajectory Optimization	77
6.1	The Moving Target Problem	77
6.2	Transcribing the Continuous Problem	79
6.2.1	Direct Multi-Shooting	79
6.2.2	Direct Collocation	79
6.2.3	Implementation Sketch: Moving-Target Direct Collocation	80
6.3	Optimizing Underactuated Geometries	80
6.4	Repeatability: The Stochastic OCP	82
6.5	Evolutionary Methods as Complementary Optimizers	82
6.6	Summary	84
6.7	Exercises	84
7	Funnel Synthesis	86
7.1	Beyond LQR: The Need for Certified Invariance	86
7.2	Time-Varying Lyapunov Functions for Trajectories	87
7.3	Sums-of-Squares (SOS) Programming	88
7.3.1	Implementation Sketch: Funnel Synthesis	88
7.4	Computing the Funnel for the Golf Swing	90
7.5	Summary	91
8	Phase-Variable Control	92
8.1	Spatial Paths vs. Temporal Execution	92
8.2	Virtual Constraints	93
8.3	Hybrid Zero Dynamics	94
8.4	Application: The Golf Downswing	95
8.5	Summary	95
9	Stochastic Trajectories & Motor Variability	96
9.1	Signal-Dependent Noise	96
9.2	Minimum Variance Theory of Human Movement	97
9.3	The Speed-Accuracy Tradeoff in the Golf Swing	98
9.4	Summary	99
9.5	Summary	100

10 Learning to Move	101
10.1 Iterative Learning Control (ILC)	101
10.2 Policy Search and Reinforcement Learning	104
10.2.1 Evolutionary Policy Search	104
10.3 Trajectory Libraries	104
10.4 Summary	105
11 Case Study: The Complete Golf Swing	106
11.1 Unifying the "Control Is Motion" Framework	106
11.1.1 Phase 1: Defining the Geometry and Optimization	107
11.1.2 Phase 2: Funnel Synthesis and Robustness	107
Glossary of Important Concepts	108
Further Reading and References	110
Index	113

Nomenclature

General Conventions

- Scalars are lowercase or Greek letters (e.g., m, t, θ).
- Vectors are bold lowercase (e.g., $\mathbf{x}, \mathbf{u}, \mathbf{q}$).
- Matrices are bold uppercase (e.g., $\mathbf{A}, \mathbf{B}, \mathbf{M}, \mathbf{J}$).
- Expected values and sets are denoted by blackboard bold or script (e.g., $\mathbb{E}, \mathcal{S}$).

State and Kinematics

$\mathbf{q} \in \mathbb{R}^n$ Joint configuration vector (generalized coordinates).

$\dot{\mathbf{q}} \in \mathbb{R}^n$ Joint velocity vector (generalized velocities).

$\ddot{\mathbf{q}} \in \mathbb{R}^n$ Joint acceleration vector.

$\mathbf{x} \in \mathbb{R}^{n_x}$ System state vector; commonly $\mathbf{x} = [\mathbf{q}^T, \dot{\mathbf{q}}^T]^T$ for mechanical systems.

$\mathbf{u} \in \mathbb{R}^m$ Control input vector (e.g., joint torques, muscle activations).

$\boldsymbol{\tau} \in \mathbb{R}^n$ Generalized forces/torques acting on the joints.

$t \in \mathbb{R}$ Time.

Inertia and Dynamics

$\mathbf{M}(\mathbf{q})$ Mass (inertia) matrix, symmetric positive definite.

$\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})$ Coriolis and centrifugal force matrix.

$\mathbf{g}(\mathbf{q})$ Gravity vector.

$\mathbf{J}(\mathbf{q})$ Jacobian matrix relating joint velocities to task-space velocities ($\dot{\mathbf{p}} = \mathbf{J}(\mathbf{q})\dot{\mathbf{q}}$).

Control and Optimization

$\mathbf{A}_k = \frac{\partial f}{\partial \mathbf{x}}$ Local Jacobian matrix of the state dynamics.

$\mathbf{B}_k = \frac{\partial f}{\partial \mathbf{u}}$ Local Jacobian matrix of the control input.

$V(\mathbf{x})$ or $\mathcal{V}$ Value function or Lyapunov function.

$\mathbf{Q}$ State penalty matrix in LQR/DDP.

R Control penalty matrix in LQR/DDP.

K Feedback gain matrix ($\delta \mathbf{u} = -\mathbf{K}\delta \mathbf{x}$).

γ A trajectory or flow, mapping time and initial conditions to states.

Biomechanics and Motor Control

$a_i \in [0, 1]$ Muscle activation level.

ℓ_i^M Muscle fiber length.

v_i^M Muscle fiber contraction velocity.

ℓ_i^{MT} Musculotendon unit length.

F_{muscle} Force generated by muscles.

W Muscle synergy matrix (weights).

$\mathbf{c}(t)$ Muscle synergy activation vector over time.

Geometry and Manifolds

$SE(3)$ Special Euclidean group of rigid body motions.

$SO(3)$ Special Orthogonal group of 3D rotations.

$\mathfrak{se}(3)$ Lie algebra of $SE(3)$, space of spatial twists (velocities).

$\mathcal{M}$ A smooth manifold.

$T_{\mathbf{x}}\mathcal{M}$ Tangent space at point $\mathbf{x}$.

Chapter 1

Throwing Away the Target

A river does not flow toward the ocean because it knows where the ocean is. It flows because flowing is what rivers do.

— Adapted from Lao Tzu

In Plain Language 1.1. Setting the Stage: Motion, Not Destination

Classical control theory often focuses on getting a system to a specific point and keeping it there—like a thermostat holding a room at 72 degrees. However, for complex motions like a golf swing, a gymnast’s routine, or a robot walking, there is no single “destination”; the motion itself is the goal. This chapter shifts the perspective from “reaching a target” to “following a trajectory.” We will see how focusing on the path a system takes, rather than a single endpoint, fundamentally changes how we think about stability, optimization, and control in high-dimensional spaces. Let’s explore why throwing away the target is the first step toward true mastery of motion.

1.1 The Wrong Question

Open any classical control textbook and you will find, within the first few pages, a picture like this: a system, an error signal, a reference point, and a controller whose entire purpose is to drive that error to zero. The implicit promise is seductive—*give me a destination and I will take you there*.

But consider the golf swing.

A skilled golfer does not steer the clubhead toward the ball the way a thermostat drives a room toward 72°F. There is no “setpoint” floating in space that the club is chasing. Instead, the golfer’s body executes a *motion*—a coordinated, whole-body trajectory through a high-dimensional configuration space. The clubhead arrives at the ball not because it was pointed at the ball, but because the entire motion was shaped so that, somewhere along its arc, the club happens to pass through the ball’s location at enormous speed.

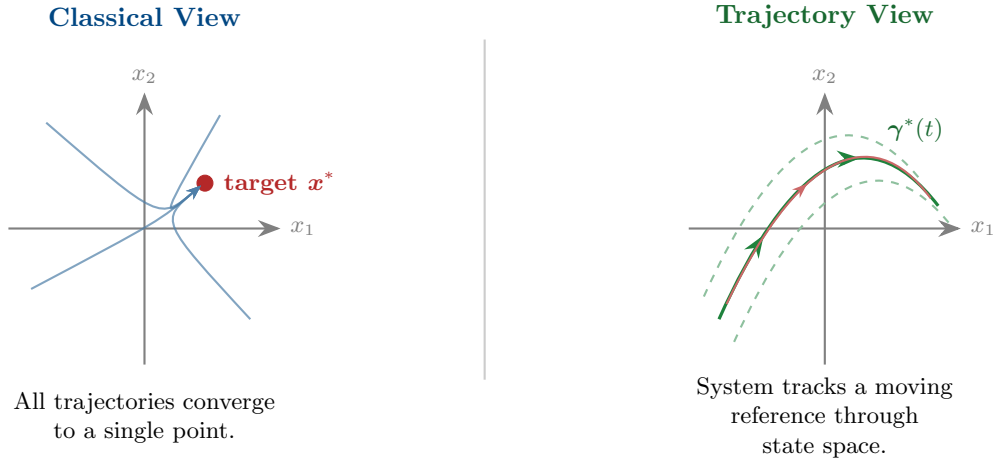
Key Idea 1.1. The Central Thesis

Control is motion, not destination. The fundamental object of nonlinear control is not a setpoint, not an equilibrium, and not a fixed target. It is a *trajectory*—a

curve through state space that the system is asked to follow, and around which disturbances are rejected. The quality of control is measured not by how close the system gets to a point, but by how faithfully it reproduces a desired motion.

This is not merely a philosophical reframing. It changes the mathematics. It changes what we optimize. It changes how we define stability. And it changes what “success” means in a controlled system.

The purpose of this book is to develop control theory from this trajectory-first perspective. The classical setpoint framework is the correct tool for regulation problems—holding a system at a desired operating point—and remains one of the great achievements of control engineering. However, for systems whose primary purpose is to *move*, the trajectory-tracking framework provides a natural generalization that contains setpoint regulation as a special case.



1.2 Why Setpoints Are a Special Case

Let us be precise about what we mean. Consider a nonlinear system

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u}), \quad \mathbf{x} \in \mathcal{X} \subseteq \mathbb{R}^n, \quad \mathbf{u} \in \mathcal{U} \subseteq \mathbb{R}^m, \quad (1.1)$$

where $\mathbf{f}$ is smooth and the state space $\mathcal{X}$ may carry additional geometric structure (it may be a manifold, as we will see in Chapter 3).

In the classical framework, the control objective is:

Find $\mathbf{u}(t)$ such that $\mathbf{x}(t) \rightarrow \mathbf{x}^$ as $t \rightarrow \infty$.*

The entire apparatus of Lyapunov stability, LQR, pole placement, and PID control is built around this question. But notice what this objective assumes:

A1. There exists a meaningful equilibrium $\mathbf{x}^*$ where $\mathbf{f}(\mathbf{x}^*, \mathbf{u}^*) = \mathbf{0}$.

A2. The system’s purpose is to *reach and remain at* this equilibrium.

A3. Performance is measured by how quickly and smoothly the error $\mathbf{e}(t) = \mathbf{x}(t) - \mathbf{x}^*$ decays.

Now consider the systems this book is about:

- A golfer executing a swing—the body passes through a continuum of configurations and never “arrives” anywhere.
- A gymnast performing a floor routine—the motion *is* the performance.
- A biped walking—there is no single posture that constitutes “walking”; walking is a *limit cycle*, a periodic orbit through state space.
- A robotic arm tracing a weld seam—the task is defined by the path, not the endpoint.

For all of these systems, assumptions **A1–A3** fail. The correct control objective is fundamentally different:

Principle 1.1. The Trajectory Tracking Objective

Given a reference trajectory $\gamma^* : [0, T] \rightarrow \mathcal{X}$, find a feedback control law $\mathbf{u} = \mathbf{k}(\mathbf{x}, t)$ such that

$$\|\mathbf{x}(t) - \gamma^*(t)\| \leq \varepsilon(t) \quad \text{for all } t \in [0, T], \quad (1.2)$$

where $\varepsilon(t)$ is a prescribed tracking tolerance (the “tube width”), and the system is robust to disturbances $\mathbf{d}(t)$ satisfying $\|\mathbf{d}(t)\| \leq \delta$.

A setpoint is simply the degenerate case where $\gamma^*(t) = \mathbf{x}^*$ for all t and $T \rightarrow \infty$. The trajectory-tracking framework *contains* the setpoint framework, but not vice versa.

Definition 1.1. Trajectory Tube

Given a reference trajectory $\gamma^* : [0, T] \rightarrow \mathcal{X}$ and a tube radius function $\varepsilon : [0, T] \rightarrow \mathbb{R}_{>0}$, the **trajectory tube** is the time-varying set

$$\mathcal{T}(t) = \{ \mathbf{x} \in \mathcal{X} : \|\mathbf{x} - \gamma^*(t)\| \leq \varepsilon(t) \}. \quad (1.3)$$

A controller achieves **tube-invariance** if every solution starting in $\mathcal{T}(0)$ remains in $\mathcal{T}(t)$ for all $t \in [0, T]$.

This definition is the trajectory-centric replacement for Lyapunov stability. Instead of asking “does the system converge to a point?” we ask “does the system stay inside a moving tube?” The tube can be tight where precision matters (e.g., at ball impact in the golf swing) and loose where it does not (e.g., during the backswing).

Remark 1.1. Tubes as Reachable Sets

The trajectory tube $\mathcal{T}(t)$ is intimately related to the concept of **reachable sets** and **attainable sets** from geometric control theory. These concepts form the mathematical foundation of trajectory-first control and are rigorously developed in the modern treatment by Agrachev and Sachkov [1]. A reachable set at time t is the collection of all states that can be reached from the initial condition $\mathbf{x}(0)$ in exactly time t , using admissible controls. The funnel-shaped tube narrows as time progresses precisely because the reachable sets, when constrained by the terminal manifold $\psi(\mathbf{x}(t_f)) = \mathbf{0}$, force convergence toward the target. Agrachev and Sachkov’s framework for characterizing these sets through the orbit of vector fields generated by the control Lie algebra provides the theoretical underpinning for understanding why certain motions are “accessible” and others are not.

1.3 Anatomy of a Motion: The Golf Swing as Prototype

We will use the golf swing throughout this book as a running example—not because golf is inherently more interesting than other motions, but because it compresses an extraordinary number of control-theoretic challenges into roughly 1.2 seconds:

- (i) **High dimensionality.** The human body has over 200 degrees of freedom. Even a simplified musculoskeletal model of the golf swing uses 20–40 generalized coordinates.
- (ii) **Extreme nonlinearity.** Joint torques couple through the full rigid-body dynamics, including Coriolis and centrifugal terms that dominate at high angular velocities.
- (iii) **Underactuation.** Once the club is released in the simplified downswing model, the wrist hinge is treated as low-torque or passive. Clubhead speed at impact is strongly shaped by passive coupling terms, but the size of that contribution depends on the model, grip constraint, and measured swing data.
- (iv) **Time-criticality with flexible timing.** The swing must produce the right state (clubhead position, velocity, and orientation) at impact, but the *exact* time of impact is not prescribed. The golfer is free to take the backswing slowly or quickly.
- (v) **Repeatability over optimality.** A touring professional does not execute the theoretically optimal swing. They execute a swing they can *repeat* 300 times in a tournament, under pressure, with minimal variance.

Example 1.1. From Setpoint Thinking to Trajectory Thinking

Suppose we model a simplified golf swing with generalized coordinates $\mathbf{q} = (\theta_{\text{torso}}, \theta_{\text{shoulder}}, \theta_{\text{arm}}, \theta_{\text{wrist}})^\top$ and their velocities $\dot{\mathbf{q}}$, giving state $\mathbf{x} = (\mathbf{q}, \dot{\mathbf{q}}) \in \mathbb{R}^8$.

Setpoint approach (inappropriate for this task): Define a target state $\mathbf{x}^* = (\mathbf{q}_{\text{impact}}, \dot{\mathbf{q}}_{\text{impact}})$ and design a controller to drive $\mathbf{x}(t) \rightarrow \mathbf{x}^*$. This is ill-suited to a golf swing because the system should not *stop* at $\mathbf{x}^*$. Impact is a waypoint, not a destination. The club must be accelerating *through* the ball, not decelerating toward it.

Trajectory approach (right): Define a reference motion $\gamma^*(s) : [0, 1] \rightarrow \mathbb{R}^8$ param-

eterized by a phase variable s (not clock time t). The controller’s job is to ensure that the system traverses γ^* with the right sequencing and coordination, with a tight tube near the impact phase ($s \approx 0.7$) and a loose tube during setup ($s \approx 0$).

Point (v) deserves special emphasis. Classical optimal control asks: “What is the minimum-time or minimum-energy trajectory?” But this is the wrong question for biological and athletic systems. The right question is:

Principle 1.2. The Repeatability Principle

The best trajectory is not the one that maximizes performance on a single trial, but the one that maximizes *expected performance over many trials* in the presence of noise, fatigue, and uncertainty. Formally, we seek

$$\gamma^* = \arg \min_{\gamma} \mathbb{E} \left[\int_0^T \ell(\mathbf{x}(t), \mathbf{u}(t)) dt \right] + \lambda \text{Var}[J(\gamma, \mathbf{w})], \quad (1.4)$$

where J is the performance cost, $\mathbf{w}$ represents stochastic disturbances (motor noise, perceptual error), and $\lambda > 0$ penalizes variance.

This is why elite golfers do not all swing the same way. Each golfer finds a trajectory that is locally optimal *for their body and their noise characteristics*. Robustness is not an afterthought bolted onto an optimal controller—it is the primary design objective.

1.4 The Mathematical Landscape

To make the trajectory-first perspective rigorous, we need mathematical machinery that most introductory control courses skip entirely. This section provides a roadmap of the key concepts we will develop in subsequent chapters.

1.4.1 Phase Variables and Time Reparameterization

A crucial distinction separates *what* the system does from *when* it does it. Consider two executions of the same golf swing: one with a slow backswing and one with a fast backswing. The *spatial path* through configuration space may be identical, but the *time parameterization* differs.

We formalize this by introducing a **phase variable** $s(t) \in [0, 1]$ that monotonically increases from 0 (start of motion) to 1 (end of motion), with dynamics

$$\dot{s} = \alpha(s, \mathbf{x}), \quad (1.5)$$

where $\alpha > 0$ is the *phase velocity*. The reference trajectory is defined as a function of s , not t :

$$\gamma^* = \gamma^*(s), \quad s \in [0, 1]. \quad (1.6)$$

This decouples the *shape* of the motion from its *temporal execution*. The controller has two separate tasks:

- (a) **Path tracking:** keep $\mathbf{x}(t)$ close to the reference path $\gamma^*(s(t))$, and
- (b) **Timing control:** regulate $\dot{s}$ to achieve the desired speed profile along the path.

This decomposition is impossible in the setpoint framework, where time and space are fused together.

1.4.2 Transverse Dynamics and Orbital Stability

When we shift from point stability to trajectory stability, the relevant notion of “closeness” changes. We do not care about the distance from $\mathbf{x}(t)$ to $\gamma^*(t)$ at the same clock time—we care about the distance from $\mathbf{x}(t)$ to the *nearest point on the trajectory*.

Definition 1.2. Transverse Distance

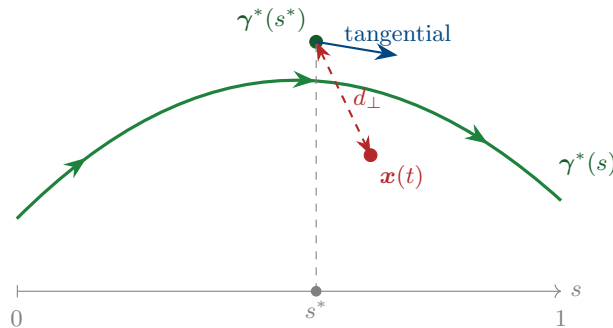
Let $\gamma^* : [0, 1] \rightarrow \mathcal{X}$ be a smooth reference trajectory. The **transverse distance** from a state $\mathbf{x}$ to γ^* is

$$d_{\perp}(\mathbf{x}, \gamma^*) = \min_{s \in [0, 1]} \|\mathbf{x} - \gamma^*(s)\|. \quad (1.7)$$

The minimizing argument $s^*(\mathbf{x}) = \arg \min_s \|\mathbf{x} - \gamma^*(s)\|$ is the **projection phase**, giving the “closest point on the reference.”

This leads to a decomposition of the state into two components:

- The **tangential component**: where the system is *along* the trajectory (measured by s).
- The **transverse component**: how far the system is *from* the trajectory (measured by $d_{\perp}$).



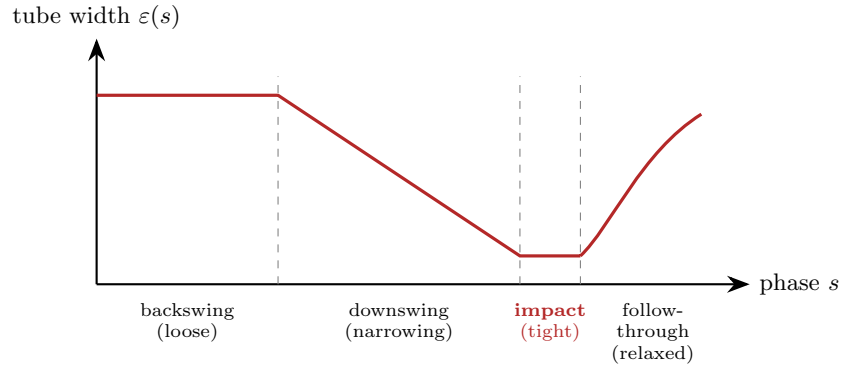
A system is **orbitally stable** around γ^* if the transverse distance $d_{\perp}(\mathbf{x}(t), \gamma^*)$ remains small (or decays) even if the tangential phase $s(t)$ drifts relative to the nominal timing. This is exactly the right notion for the golf swing: we want the motion to follow the right *path* even if the golfer is slightly faster or slower than planned.

1.4.3 Funnel Control and Time-Varying Tubes

Combining the trajectory tube (Definition 1.1) with the transverse/tangential decomposition leads to what we call **funnel control**: the design of controllers that keep the system inside a prescribed time-varying tube around the reference.

The tube width $\varepsilon(s)$ is a design parameter. For the golf swing:

$$\varepsilon(s) = \begin{cases} \varepsilon_{\text{loose}} & s \in [0, 0.3] \quad (\text{address \& backswing}), \\ \varepsilon_{\text{loose}} - (\varepsilon_{\text{loose}} - \varepsilon_{\text{tight}}) \frac{s - 0.3}{0.4} & s \in [0.3, 0.7] \quad (\text{transition \& downswing}), \\ \varepsilon_{\text{tight}} & s \in [0.7, 0.8] \quad (\text{impact zone}), \\ \text{relaxed} & s \in [0.8, 1.0] \quad (\text{follow-through}). \end{cases} \quad (1.8)$$



This funnel profile encodes the modeling hypothesis that precision demands are highest near impact and lower during setup. The controller allocates authority where the stated terminal constraint is most sensitive, rather than maintaining uniform precision everywhere; this distinction is easy to obscure in a pure fixed-setpoint framing.

1.5 What Changes When Control Is Motion

Adopting the trajectory-first perspective changes nearly every aspect of control system design. Here we preview the major consequences, each of which will be developed in detail in subsequent chapters.

1.5.1 Stability Is Not Convergence—It Is Confinement

In the setpoint framework, stability means convergence: the state approaches the equilibrium and stays near it. In the trajectory framework, stability means *confinement*: the state remains inside the moving tube.

Definition 1.3. Trajectory Stability

A reference trajectory γ^* is **stable** under the closed-loop dynamics $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{k}(\mathbf{x}, t))$ if for every $\varepsilon > 0$ there exists $\delta > 0$ such that

$$d_{\perp}(\mathbf{x}(0), \gamma^*) < \delta \implies d_{\perp}(\mathbf{x}(t), \gamma^*) < \varepsilon \quad \forall t \in [0, T]. \quad (1.9)$$

It is **asymptotically trajectory-stable** if, additionally, $d_{\perp}(\mathbf{x}(t), \gamma^*) \rightarrow 0$ as the system evolves.

Note the subtle but critical difference: we use the transverse distance $d_{\perp}$, not the time-indexed error $\|\mathbf{x}(t) - \gamma^*(t)\|$. This allows the system to be ahead of or behind schedule without being considered “unstable.”

1.5.2 The Cost Function Is a Functional, Not a Function

In setpoint control, the cost is typically a function of the error $\mathbf{e}(t)$:

$$J_{\text{setpoint}} = \int_0^{\infty} (\mathbf{e}^{\top} \mathbf{Q} \mathbf{e} + \mathbf{u}^{\top} \mathbf{R} \mathbf{u}) dt. \quad (1.10)$$

In trajectory control, the cost is a **functional** that operates on the entire motion:

$$J_{\text{traj}}[\mathbf{x}(\cdot), \mathbf{u}(\cdot)] = \underbrace{\int_0^T w_{\perp}(s) d_{\perp}^2(\mathbf{x}(t), \gamma^*) dt}_{\text{tracking fidelity (transverse)}} + \underbrace{\int_0^T w_{\parallel}(s) (\dot{s} - \dot{s}_{\text{ref}})^2 dt}_{\text{timing fidelity (tangential)}} + \underbrace{\int_0^T \mathbf{u}^{\top} \mathbf{R}(s) \mathbf{u} dt}_{\text{effort}}, \quad (1.11)$$

where the weighting matrices $w_{\perp}(s)$, $w_{\parallel}(s)$, and $\mathbf{R}(s)$ are *phase-dependent*. This is what allows us to demand high precision at impact and low effort during the backswing.

Remark 1.2. The Pontryagin Maximum Principle

The mathematical foundation for solving trajectory optimization problems with terminal constraints (such as the moving goal in the golf swing) comes from the **Pontryagin Maximum Principle**, a cornerstone of geometric control theory. As developed rigorously in Agrachev and Sachkov [1], this principle transforms the infinite-dimensional problem of optimizing $J[\mathbf{x}(\cdot), \mathbf{u}(\cdot)]$ into a two-point boundary value problem via the introduction of costate variables (adjoint variables) and a Hamiltonian function. The principle states that along an optimal trajectory, the Hamiltonian $\mathcal{H} = L(\mathbf{x}, \mathbf{u}) + \boldsymbol{\lambda}^{\top} \mathbf{f}(\mathbf{x}, \mathbf{u})$ is maximized with respect to $\mathbf{u}$ at each instant, where $\boldsymbol{\lambda}(t)$ evolves backward in time according to adjoint dynamics. This formalism unifies our treatment of both the path optimization (determining γ^*) and the feedback stabilization (computing funnels around γ^*) within a single geometric framework. Chapter 6 will apply these principles explicitly to synthesize the moving-target golf swing trajectory.

1.5.3 Feedforward Is Not Optional—It Is Primary

In the setpoint framework, feedforward is a “nice to have” that reduces transient error. In the trajectory framework, feedforward is the *primary* control action. The reference trajectory

$\gamma^*(t)$ implicitly defines a feedforward control signal

$$\mathbf{u}_{\text{ff}}(t) = \mathbf{f}^{-1}(\dot{\gamma}^*(t), \gamma^*(t)), \quad (1.12)$$

where $\mathbf{f}^{-1}$ denotes the inverse dynamics. The feedback controller handles only the *deviations* from this feedforward signal:

$$\mathbf{u}(t) = \underbrace{\mathbf{u}_{\text{ff}}(t)}_{\text{executes the motion}} + \underbrace{\mathbf{u}_{\text{fb}}(\mathbf{x}(t), t)}_{\text{corrects deviations}}. \quad (1.13)$$

For the golf swing, the feedforward component accounts for approximately 90% of the total joint torques. The feedback controller provides the remaining 10% of correction. A control design that treats everything as feedback is asking the feedback loop to do ten times more work than necessary.

1.5.4 Underactuation Creates Geometry, Not Just Constraints

Many moving systems are underactuated: they have fewer control inputs than degrees of freedom. In the setpoint framework, underactuation is an obstacle—it means we cannot independently control all states. In the trajectory framework, underactuation creates *geometric structure* that we can exploit.

The set of states reachable from a given configuration through passive dynamics forms a submanifold of the state space. A well-designed trajectory *rides along* this manifold, using the passive dynamics to generate motion that active torques alone could not produce within the chosen actuator model. In a simplified golf model, a passive wrist release and centrifugal coupling can contribute to clubhead acceleration and face closure; whether they square the clubface in a real swing depends on grip mechanics, shaft dynamics, impact geometry, and validation against measured swings.

Proposition 1.1. *For a mechanical system with n degrees of freedom and $m < n$ actuators, the set of dynamically feasible trajectories lies on a submanifold of dimension at most $2m$ in the $2n$ -dimensional state space. Effective trajectory design requires understanding the geometry of this submanifold.*

We will make this precise in Chapter 5 using the language of constraint manifolds and their null-space structures.

1.6 A Roadmap for the Book

The remainder of this book develops the trajectory-first theory in a sequence of increasingly sophisticated layers:

Ch. 2 Curves in State Space. The geometry of trajectories: parameterization, arc length, curvature, and torsion in high-dimensional state spaces. Frenet–Serret frames for nonlinear systems.

Ch. 3 Configuration Manifolds. Why state space is often not $\mathbb{R}^n$: Lie groups, joint spaces, and the geometry of articulated bodies. The golf swing lives on $SO(3)^k$, not $\mathbb{R}^k$.

- Ch. 4 Orbital Stability and Transverse Linearization.** Rigorous definitions of trajectory stability. Linearization transverse to the orbit. Floquet theory for periodic motions. Moving Poincaré sections.
- Ch. 5 Underactuation and Passive Dynamics.** Constraint Jacobians, null-space control, and the exploitation of passive mechanical coupling. The double pendulum and the physics of the wrist release.
- Ch. 6 Trajectory Optimization.** Direct collocation, shooting methods, and pseudospectral methods for finding optimal reference trajectories. The interplay between optimality and repeatability.
- Ch. 7 Funnel Synthesis.** Designing time-varying tubes with certified invariance under stated model, controller, and disturbance assumptions. Sums-of-squares programming for funnel computation. Robust funnels under model uncertainty.
- Ch. 8 Phase-Variable Control [27].** Decoupling path shape from timing. Virtual constraints and hybrid zero dynamics for walking and running. Central pattern generators as phase oscillators.
- Ch. 9 Stochastic Trajectories and Motor Variability.** Signal-dependent noise in biological actuators. The minimum-variance theory of human movement [26]. Why optimal trajectories are smooth [9].
- Ch. 10 Learning to Move.** Iterative learning control, policy search, and trajectory libraries. How to improve a motion over repeated trials without losing stability.
- Ch. 11 Case Study: The Complete Golf Swing.** A planned application of the theory to a declared musculoskeletal model of the golf swing, from trajectory optimization through funnel synthesis to stochastic robustness analysis.

1.7 A New Language for Control

Before proceeding, let us establish the conceptual vocabulary that will replace the classical language throughout this book:

Classical term	→	Trajectory term
Setpoint / reference	→	Reference trajectory $\gamma^*(s)$
Error $e(t) = \mathbf{x} - \mathbf{x}^*$	→	Transverse deviation $d_{\perp}(\mathbf{x}, \gamma^*)$
Lyapunov stability	→	Orbital / trajectory stability
Convergence to equilibrium	→	Confinement to tube
Regulation	→	Path tracking + timing control
Step response	→	Trajectory tracking response
Steady-state error	→	Persistent transverse offset
Gain margin	→	Funnel robustness margin
Pole placement	→	Transverse eigenvalue assignment

1.8 Exercises

1.1. (Conceptual.) Consider a simple pendulum $\ddot{\theta} + \sin \theta = u$.

- (a) Identify the equilibria and classify their stability.
- (b) Now consider the task of making the pendulum execute a full revolution (pumping up from rest to continuous spinning). Explain why this task *cannot* be formulated as setpoint regulation.
- (c) Sketch the reference trajectory in the $(\theta, \dot{\theta})$ phase plane for one complete revolution. What is the natural phase variable?

1.2. (Mathematical.) For the system $\dot{x}_1 = x_2$, $\dot{x}_2 = -x_1 + u$, consider the reference trajectory $\gamma^*(t) = (\cos t, -\sin t)^\top$ (a unit circle in state space).

- (a) Verify that $u_{\text{ff}}(t) = 0$ produces exactly this trajectory.
- (b) Compute the transverse linearization about γ^* .
- (c) Design a feedback law u_{fb} that achieves asymptotic orbital stability.

1.3. (Design.) A simplified 2-DOF golf swing model has coordinates $\mathbf{q} = (\theta_{\text{arm}}, \theta_{\text{club}})$ with dynamics

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}) = \mathbf{B} \mathbf{u}, \quad \mathbf{B} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}.$$

This system is underactuated: only θ_{arm} is directly actuated.

- (a) Explain qualitatively how the club can accelerate through passive dynamics even though no torque is applied at the wrist.
 - (b) Define a phase variable s and sketch a funnel profile $\varepsilon(s)$ appropriate for this system.
 - (c) Discuss why a setpoint controller applied only at impact ($s = 0.7$) may be inadequate or misleading for a trajectory-native task.
- 1.4. (Philosophical.)** A thermostat, an autopilot, and a dancer's body are all “controlled systems.” Classify each according to whether its primary control objective is (i) setpoint regulation, (ii) trajectory tracking, or (iii) something else entirely. Justify your classification.

The river does not arrive at the ocean and stop.

It becomes the ocean.

The swing does not arrive at impact and stop.

It becomes the follow-through.

Control is motion.

Chapter 2

Curves in State Space

The shortest distance between two points is a straight line, but no interesting motion has ever been straight.

In Plain Language 2.1. State Space and Motion

If you want to describe a golf swing, you don't just need to know where the club is; you need to know how fast it's moving, where the arms are, and how the hips are rotating. "State space" is a mathematical way to keep track of all these physical variables at once. A single point in state space represents a complete snapshot of the system at one instant. As the system moves, that point traces out a path, or a "curve." In this chapter, we explore how to mathematically describe these curves. We'll find that taking apart a motion into its geometric path (its shape) and its timing (how fast it moves along that shape) makes complex motions much easier to understand and control.

2.1 Why Geometry Comes First

In Chapter 1 we argued that control is fundamentally about *motion*—the faithful execution of a trajectory through state space. Before we can stabilize a trajectory, optimize a trajectory, or confine the system to a tube around a trajectory, we need to understand what a trajectory *is* as a mathematical object.

This chapter develops the differential geometry of curves in $\mathbb{R}^n$, adapted to the needs of control theory. The central insight is that a trajectory has intrinsic geometric properties—curvature, torsion, and a natural moving frame—that exist independently of how fast the system moves along the curve. These intrinsic properties determine how difficult the trajectory is to track, how much control authority is needed to stay on course, and where the system is most vulnerable to disturbances.

The reader may wonder: *why not simply work in coordinates?* The answer is that coordinate-based descriptions mix up properties of the *curve* with properties of the *parameterization*. A slow practice swing and a full-speed swing may trace a similar path in configuration space, but they do not trace the same curve in full state space because the velocity components are different. The geometric tools we develop here allow us to say precisely which claims are about the configuration path, which claims are about timing, and

which claims require the full state trajectory.

Key Idea 2.1. Intrinsic vs. Extrinsic

A trajectory has two kinds of properties:

- **Intrinsic:** properties of the curve itself—its shape, curvature, how it bends and twists through state space. These are independent of parameterization.
- **Extrinsic:** properties that depend on how the curve is traversed—speed, acceleration, timing. These depend on the parameterization.

Good control design separates these cleanly. The *trajectory planner* designs the intrinsic shape. The *timing law* specifies the extrinsic speed profile. The *tracking controller* maintains both.

2.2 Parameterized Curves

Definition 2.1. Parameterized Curve

A **parameterized curve** in $\mathbb{R}^n$ is a smooth map

$$\gamma : I \rightarrow \mathbb{R}^n, \quad \lambda \mapsto \gamma(\lambda), \quad (2.1)$$

where $I \subseteq \mathbb{R}$ is an interval. The parameter λ may be time t , arc length s , a phase variable σ , or any other monotone scalar. The curve is **regular** if $\gamma'(\lambda) \neq \mathbf{0}$ for all $\lambda \in I$, i.e., it never “stops” (though the system traversing it may slow down).

The critical point is that many different parameterizations describe the same geometric curve. If $\gamma(\lambda)$ is a parameterized curve and $\lambda = \phi(\mu)$ is a smooth, monotonically increasing reparameterization, then $\tilde{\gamma}(\mu) = \gamma(\phi(\mu))$ traces the same set of points in $\mathbb{R}^n$, but at different “speeds.”

Example 2.1. The Circle: Three Parameterizations

Consider the unit circle in $\mathbb{R}^2$. Three natural parameterizations:

$$\gamma_1(t) = (\cos t, \sin t), \quad t \in [0, 2\pi] \quad (\text{constant angular speed}), \quad (2.2)$$

$$\gamma_2(t) = (\cos t^2, \sin t^2), \quad t \in [0, \sqrt{2\pi}] \quad (\text{accelerating}), \quad (2.3)$$

$$\gamma_3(s) = (\cos s, \sin s), \quad s \in [0, 2\pi] \quad (\text{arc-length, same as } \gamma_1 \text{ here}). \quad (2.4)$$

All three trace the same circle. The *curve* is the same; the *parameterizations* differ. Quantities like curvature are properties of the curve, not the parameterization.

2.2.1 Velocity, Speed, and the Tangent Vector

Given a parameterized curve $\gamma(\lambda)$, the **velocity vector** is

$$\mathbf{v}(\lambda) = \frac{d\gamma}{d\lambda} = \gamma'(\lambda). \quad (2.5)$$

The **speed** is its magnitude:

$$v(\lambda) = \|\gamma'(\lambda)\|. \quad (2.6)$$

The **unit tangent vector** is the velocity normalized to unit length:

$$\mathbf{T}(\lambda) = \frac{\gamma'(\lambda)}{\|\gamma'(\lambda)\|} = \frac{\gamma'(\lambda)}{v(\lambda)}. \quad (2.7)$$

The tangent vector $\mathbf{T}$ tells us the *direction* of the curve at each point—a purely geometric quantity. The speed v tells us how *fast* we traverse it—a parameterization-dependent quantity. This separation is the first instance of the intrinsic/extrinsic decomposition.

2.3 Arc Length: The Natural Parameter

Among all possible parameterizations, one is geometrically privileged: the **arc-length parameterization**, in which the parameter measures distance traveled along the curve.

Definition 2.2. Arc Length

The **arc length** of a curve $\gamma(\lambda)$ from $\lambda = a$ to $\lambda = b$ is

$$L = \int_a^b \|\gamma'(\lambda)\| d\lambda = \int_a^b v(\lambda) d\lambda. \quad (2.8)$$

The **arc-length function** $s(\lambda)$ measures the distance from the starting point:

$$s(\lambda) = \int_a^\lambda \|\gamma'(\mu)\| d\mu, \quad \text{so that } \frac{ds}{d\lambda} = v(\lambda). \quad (2.9)$$

When a curve is parameterized by arc length, the speed is identically 1: $\|\gamma'(s)\| = 1$ for all s . This means the tangent vector and the velocity vector coincide: $\mathbf{T}(s) = \gamma'(s)$. All geometric information is in the direction of the derivative; none is hidden in its magnitude.

Principle 2.1. Arc Length as the Canonical Clock

Arc-length parameterization strips away all timing information and reveals the pure geometry of the curve. In control terms, it answers the question: *what does the motion look like if we forget how fast it goes?*

For a golf swing, the arc-length parameterized trajectory $\gamma(s)$ can describe the spatial *path* of the body through configuration space. The timing law $s(t)$ —how the golfer moves along this path in real time—is a separate modeling choice. In full state space, however, changing the timing also changes the velocity coordinates, so the state-space curve must be analyzed with both geometry and speed retained.

Remark 2.1. Arc Length in High Dimensions

In $\mathbb{R}^n$ with the Euclidean metric, arc length is defined exactly as above. On a manifold, arc length requires an explicitly chosen Riemannian metric [6]. When the configuration manifold carries such a metric (as it will in Chapter 3), the formula (2.8) generalizes to

$$L = \int_a^b \sqrt{\gamma'(\lambda)^\top \mathbf{G}(\gamma(\lambda)) \gamma'(\lambda)} d\lambda, \quad (2.10)$$

where $\mathbf{G}$ is the selected metric tensor. For mechanical systems, the kinetic-energy metric on configuration space is represented in local coordinates by the mass-inertia matrix $\mathbf{M}(\mathbf{q})$ [5]. That statement is about configuration velocities $\dot{\mathbf{q}}$, not automatically about arbitrary state-space coordinates $(\mathbf{q}, \dot{\mathbf{q}})$. Thus a mass-weighted arc length can make high-inertia motions count as longer than low-inertia motions, but only after the modeling metric has been stated.

2.4 Curvature: How Curves Bend

The most important intrinsic property of a curve is its **curvature**—a measure of how rapidly the tangent direction changes as we move along the curve.

Definition 2.3. Curvature

Let $\gamma(s)$ be a curve parameterized by arc length in $\mathbb{R}^n$. The **curvature** at each point is

$$\kappa(s) = \|\gamma''(s)\| = \left\| \frac{d\mathbf{T}}{ds} \right\|. \quad (2.11)$$

Since $\|\mathbf{T}\| = 1$, the derivative $\mathbf{T}'(s)$ is orthogonal to $\mathbf{T}(s)$, and κ measures the rate of turning per unit distance. The curvature is always non-negative and is zero if and only if the curve is locally a straight line.

The reciprocal $R(s) = 1/\kappa(s)$ is the **radius of curvature**—the radius of the circle that best approximates the curve at that point. High curvature means a tight bend; low curvature means a gentle sweep.

2.4.1 Curvature in Arbitrary Parameterization

In practice, we rarely work in arc-length parameterization. Given a curve $\gamma(t)$ parameterized by time (or any other parameter), the curvature can be computed directly:

Theorem 2.1 (Curvature Formula). *For a regular curve $\gamma(t)$ in $\mathbb{R}^n$, the curvature is*

$$\kappa(t) = \frac{\|\gamma'(t) \times_n \gamma''(t)\|}{\|\gamma'(t)\|^3}, \quad (2.12)$$

where for $n = 2$, $\|\gamma' \times_2 \gamma''\| = |x'y'' - y'x''|$, and for $n = 3$, $\times_3$ is the usual cross product.

For general n , this generalizes via

$$\kappa(t) = \frac{\sqrt{\|\gamma'\|^2 \|\gamma''\|^2 - (\gamma' \cdot \gamma'')^2}}{\|\gamma'\|^3}. \quad (2.13)$$

Proof. Write $\gamma(t) = \gamma(s(t))$ where s is arc length. Then $\gamma' = \dot{s} \mathbf{T}$ and $\gamma'' = \ddot{s} \mathbf{T} + \dot{s}^2 \kappa \mathbf{N}$, where $\mathbf{N}$ is the unit normal (defined below). Since $\mathbf{T} \perp \mathbf{N}$, we have $\|\gamma'\|^2 \|\gamma''\|^2 - (\gamma' \cdot \gamma'')^2 = \dot{s}^2 (\ddot{s}^2 + \dot{s}^4 \kappa^2) - \dot{s}^2 \ddot{s}^2 = \dot{s}^6 \kappa^2$. Dividing by $\|\gamma'\|^6 = \dot{s}^6$ and taking the square root yields the result. $\square$

Example 2.2. Curvature of an Elliptical Trajectory

Consider the ellipse $\gamma(t) = (a \cos t, b \sin t)$ with $a > b > 0$. Then

$$\gamma'(t) = (-a \sin t, b \cos t), \quad (2.14)$$

$$\gamma''(t) = (-a \cos t, -b \sin t). \quad (2.15)$$

Applying the 2D curvature formula:

$$\kappa(t) = \frac{|(-a \sin t)(-b \sin t) - (b \cos t)(-a \cos t)|}{(a^2 \sin^2 t + b^2 \cos^2 t)^{3/2}} = \frac{ab}{(a^2 \sin^2 t + b^2 \cos^2 t)^{3/2}}. \quad (2.16)$$

The curvature is *highest at the ends of the minor axis* ($t = 0, \pi$, where $\kappa = a/b^2$) and *lowest at the ends of the major axis* ($t = \pi/2, 3\pi/2$, where $\kappa = b/a^2$). The tighter the bend, the higher the curvature—and, as we will see, the more control authority is needed to track it.

2.4.2 Why Curvature Matters for Control

Curvature is not merely a geometric curiosity. It has direct implications for tracking difficulty.

Principle 2.2. The Curvature–Authority Principle

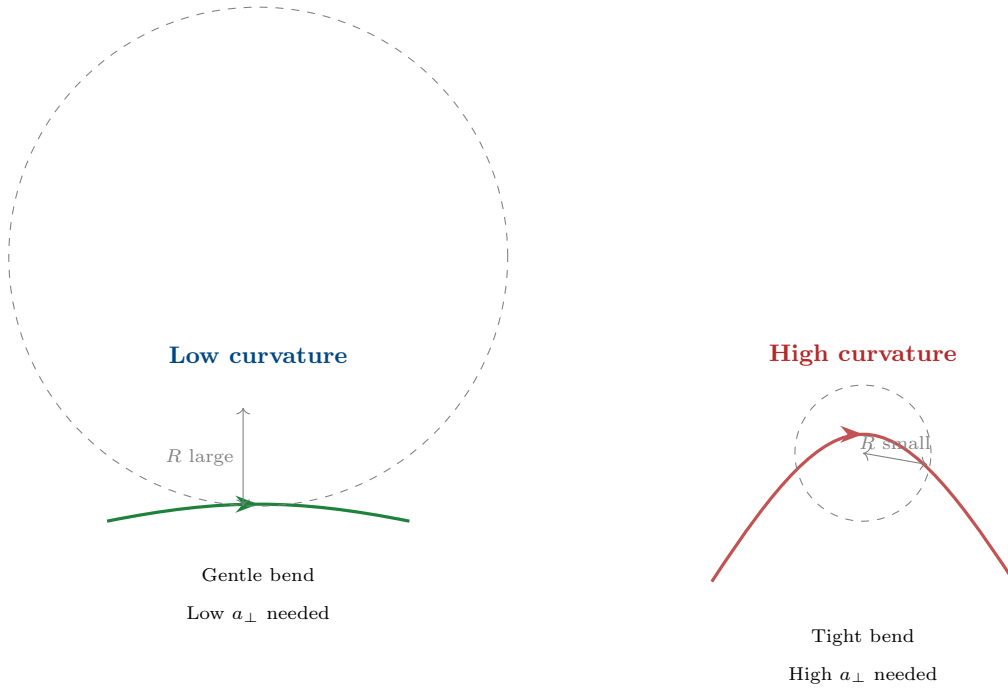
Consider a system tracking a reference trajectory $\gamma^*(s)$ at speed v . The centripetal acceleration required to follow the curve is

$$a_{\perp} = \kappa v^2. \quad (2.17)$$

This means:

- (i) High curvature demands high lateral control authority.
- (ii) The required authority grows as the *square* of the speed.
- (iii) A trajectory that is easy to track slowly may be impossible to track at high speed.

For the golf swing, this explains why the transition from backswing to downswing is so critical: the clubhead path has high curvature precisely where the speed is increasing rapidly.



2.5 The Frenet–Serret Frame

The tangent vector $\mathbf{T}$ is only the first element of a natural coordinate frame that moves with the curve. This **Frenet–Serret frame** (or simply **moving frame**) provides a complete local coordinate system at every point of the trajectory.

2.5.1 Construction in $\mathbb{R}^n$

Definition 2.4. Frenet–Serret Frame

Let $\gamma(s)$ be a curve parameterized by arc length in $\mathbb{R}^n$ with $\gamma^{(k)}(s)$ denoting the k -th derivative with respect to s . Assume the first $p \leq n$ derivatives $\{\gamma'(s), \gamma''(s), \dots, \gamma^{(p)}(s)\}$ are linearly independent at each point. The **Frenet–Serret frame** $\{\mathbf{e}_1(s), \mathbf{e}_2(s), \dots, \mathbf{e}_p(s)\}$ is defined by applying the Gram–Schmidt process to these derivatives:

$$\mathbf{e}_1 = \gamma' = \mathbf{T}, \quad (2.18)$$

$$\tilde{\mathbf{e}}_k = \gamma^{(k)} - \sum_{j=1}^{k-1} \langle \gamma^{(k)}, \mathbf{e}_j \rangle \mathbf{e}_j, \quad \mathbf{e}_k = \frac{\tilde{\mathbf{e}}_k}{\|\tilde{\mathbf{e}}_k\|}, \quad k = 2, \dots, p. \quad (2.19)$$

The frame vectors $\mathbf{e}_1, \dots, \mathbf{e}_p$ form an orthonormal set at each point. If $p < n$, the frame can be extended to a full orthonormal basis by appending $n - p$ vectors, but these additional vectors have no intrinsic geometric meaning for the curve.

For the familiar case $n = 3$:

Vector	Name	Interpretation
$\mathbf{e}_1 = \mathbf{T}$	Unit tangent	Direction of motion
$\mathbf{e}_2 = \mathbf{N}$	Principal normal	Direction the curve is bending toward
$\mathbf{e}_3 = \mathbf{B}$	Binormal	$\mathbf{T} \times \mathbf{N}$; normal to the osculating plane

2.5.2 The Frenet–Serret Equations

The power of the moving frame lies in how it evolves along the curve. The derivatives of the frame vectors with respect to arc length are expressed entirely in terms of the frame itself and a set of scalar functions called **generalized curvatures**.

Theorem 2.2 (Frenet–Serret Equations). *The Frenet–Serret frame satisfies*

$$\frac{d}{ds} \begin{pmatrix} \mathbf{e}_1 \\ \mathbf{e}_2 \\ \mathbf{e}_3 \\ \vdots \\ \mathbf{e}_p \end{pmatrix} = \begin{pmatrix} 0 & \kappa_1 & 0 & \cdots & 0 \\ -\kappa_1 & 0 & \kappa_2 & \cdots & 0 \\ 0 & -\kappa_2 & 0 & \cdots & 0 \\ \vdots & & \ddots & \ddots & \vdots \\ 0 & \cdots & 0 & -\kappa_{p-1} & 0 \end{pmatrix} \begin{pmatrix} \mathbf{e}_1 \\ \mathbf{e}_2 \\ \mathbf{e}_3 \\ \vdots \\ \mathbf{e}_p \end{pmatrix}, \quad (2.20)$$

where $\kappa_1(s) = \kappa(s) > 0$ is the curvature, $\kappa_2(s)$ is the **torsion** (often denoted τ), and $\kappa_3, \dots, \kappa_{p-1}$ are the **higher-order curvatures**. The matrix is skew-symmetric, which guarantees that the frame remains orthonormal as it evolves.

The skew-symmetry of this matrix is not a coincidence—it is the hallmark of a *rotation*. As the frame moves along the curve, it rotates without stretching. The curvatures $\kappa_1, \dots, \kappa_{p-1}$ are the “angular velocities” of this rotation projected onto the various planes of the frame.

Example 2.3. Frenet–Serret for a 3D Helix

The helix $\gamma(t) = (a \cos t, a \sin t, bt)$ with speed $v = \sqrt{a^2 + b^2}$ has arc-length parameter $s = vt$. The Frenet–Serret quantities are:

$$\mathbf{T} = \frac{1}{v}(-a \sin t, a \cos t, b), \quad (2.21)$$

$$\mathbf{N} = (-\cos t, -\sin t, 0), \quad (2.22)$$

$$\mathbf{B} = \frac{1}{v}(b \sin t, -b \cos t, a), \quad (2.23)$$

$$\kappa = \frac{a}{a^2 + b^2}, \quad \tau = \frac{b}{a^2 + b^2}. \quad (2.24)$$

Both curvature and torsion are *constant*, which makes the helix a natural reference trajectory for testing tracking controllers—analogous to the role of step inputs in classical control. The helix is to trajectory control what the step function is to setpoint control.

2.5.3 The Moving Frame in State Space

For control applications, the state space typically has dimension $n = 2N$ where N is the number of generalized coordinates (positions and velocities). A golf swing model with 4 DOF has $n = 8$. The Frenet–Serret frame in $\mathbb{R}^8$ has up to 7 generalized curvatures, κ_1 through κ_7 .

Not all of these are physically meaningful for every system. In practice, the effective dimensionality of the curve—the number of linearly independent derivatives—is determined by the system dynamics:

Principle 2.3. Dynamic Rank of a Trajectory

For a controlled mechanical system with N DOF and m independent actuators, a generic trajectory has at most $p = 2m$ linearly independent arc-length derivatives. The Frenet–Serret frame therefore has at most $2m$ intrinsically meaningful vectors, and the curve has at most $2m - 1$ independent curvatures.

For a fully actuated system ($m = N$), the trajectory can bend freely in all $2N$ state-space directions. For an underactuated system ($m < N$), the trajectory is geometrically constrained: it lives on a submanifold, and the curvatures in the “forbidden” directions are determined by the passive dynamics.

2.6 Curvature and Torsion in the Language of Control

We now connect the geometric quantities to the dynamics and control of nonlinear systems.

2.6.1 Curvature as a Demand on the Controller

Consider the system $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u})$ tracking a reference trajectory $\gamma^*(s)$ at instantaneous speed $v(t) = \dot{s}(t)$. Using the chain rule:

$$\dot{\mathbf{x}} = v \mathbf{T}, \quad \ddot{\mathbf{x}} = \dot{v} \mathbf{T} + v^2 \kappa \mathbf{N}. \quad (2.25)$$

The acceleration decomposes into:

- A **tangential component** $\dot{v} \mathbf{T}$ that changes the speed along the curve.
- A **normal component** $v^2 \kappa \mathbf{N}$ that “steers” the system around the bend.

Definition 2.5. Control Demand Decomposition

Along a reference trajectory $\gamma^*(s)$ traversed at speed v , the instantaneous control demand decomposes as

$$\mathbf{u}_{\text{demand}} = \underbrace{\mathbf{u}_{\parallel}}_{\text{tangential}} + \underbrace{\mathbf{u}_{\perp}}_{\text{normal}}, \quad (2.26)$$

where:

$$\mathbf{u}_{\parallel} \propto \dot{v} \quad (\text{accelerate/decelerate along path}), \quad (2.27)$$

$$\mathbf{u}_{\perp} \propto v^2 \kappa \quad (\text{centripetal force to follow curvature}). \quad (2.28)$$

The normal demand $\mathbf{u}_\perp$ grows quadratically with speed and linearly with curvature. This is the **curvature cost** of the trajectory.

Example 2.4. Curvature Cost in the Golf Downswing

In the transition from backswing to downswing, a professional golfer reverses the direction of the club in roughly 0.15 seconds. Let us estimate the curvature cost.

At the top of the backswing, the clubhead speed is approximately zero. At impact (0.3 s later), it reaches ~ 50 m/s. The path through configuration space has a tight reversal—a region of high curvature κ .

Using $a_\perp = \kappa v^2$, and noting that mid-downswing $v \approx 25$ m/s with the reversal curvature $\kappa \approx 2$ rad/m (estimated from motion capture data), the centripetal acceleration demand is

$$a_\perp \approx 2 \times 25^2 = 1250 \text{ m/s}^2 \approx 127 \text{ g}. \quad (2.29)$$

This enormous demand is met not by muscular torque alone, but by the *passive coupling* in the kinematic chain—the very underactuation that the setpoint framework treats as a constraint. The geometry of the trajectory creates the forces needed to execute it.

Remark 2.2. The Orbit Theorem and Achievable Curves

A profound connection exists between the curvature of a trajectory and the *control vector fields* that generate it. The **Orbit Theorem** (also known as **Chow's Theorem**), a central result in geometric control theory as presented by Agrachev and Sachkov [1], states that the orbit generated by iteratively flowing along the control vector fields $\mathbf{f}_i$ and their Lie brackets $[\mathbf{f}_i, \mathbf{f}_j] = \frac{\partial \mathbf{f}_i}{\partial \mathbf{x}} \mathbf{f}_j - \frac{\partial \mathbf{f}_j}{\partial \mathbf{x}} \mathbf{f}_i$ determines the space of achievable curves. For control-affine systems $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}) + \sum u_i \mathbf{g}_i(\mathbf{x})$, higher-order Lie brackets correspond to higher derivatives of the trajectory in the direction perpendicular to the primary control inputs. The curvature of a trajectory encodes which Lie bracket relationships are being exploited: a highly curved trajectory requires the activation of higher-order commutators in the control Lie algebra. This is the geometric signature of how underactuated systems achieve complex motions despite having fewer inputs than degrees of freedom.

2.6.2 Torsion as Out-of-Plane Complexity

If curvature measures how a trajectory bends *within* its osculating plane, **torsion** measures how the osculating plane itself rotates along the curve. A planar curve has zero torsion everywhere. A helix has constant nonzero torsion. A golf swing—which involves simultaneous rotation in the transverse, sagittal, and frontal planes—has torsion that varies dramatically through the motion.

Definition 2.6. Torsion

For a curve in $\mathbb{R}^n$ with $n \geq 3$, the **torsion** $\tau(s) = \kappa_2(s)$ is the second generalized curvature in the Frenet–Serret equations:

$$\frac{d\mathbf{N}}{ds} = -\kappa\mathbf{T} + \tau\mathbf{B}. \quad (2.30)$$

Torsion measures the rate at which the osculating plane rotates about the tangent direction, per unit arc length.

In high-dimensional state spaces ($n \gg 3$), the higher curvatures $\kappa_3, \dots, \kappa_{p-1}$ measure increasingly subtle geometric twisting. For control design, the first two curvatures (κ and τ) capture the dominant geometric effects, with higher curvatures becoming relevant primarily in highly articulated systems.

Remark 2.3. The Curvature Signature

The ordered set of generalized curvatures $(\kappa_1(s), \kappa_2(s), \dots, \kappa_{p-1}(s))$ as functions of arc length constitutes the **curvature signature** of the trajectory. By the fundamental theorem of curves, this signature (up to rigid motions) *uniquely determines* the trajectory. Two motions with identical curvature signatures are geometrically congruent—they are the same shape, possibly translated and rotated.

This has a striking implication for motor control: if a golfer can reproduce the curvature signature of their swing, the spatial path is automatically reproduced, regardless of timing.

2.7 Reparameterization and Timing Laws

We now formalize the separation between the *shape* of a trajectory and the *timing* of its execution.

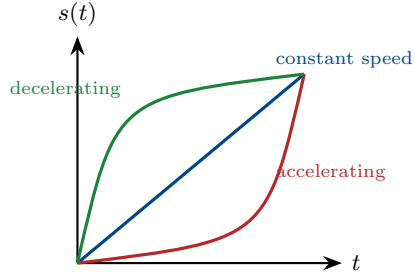
Definition 2.7. Timing Law

Given a reference trajectory $\gamma^*(s)$ parameterized by arc length, a **timing law** is a smooth, monotonically increasing function $s : [0, T] \rightarrow [0, L]$ with $s(0) = 0$ and $s(T) = L$. The time-parameterized trajectory is

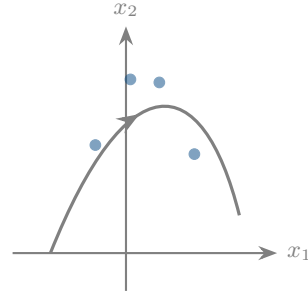
$$\mathbf{x}^*(t) = \gamma^*(s(t)). \quad (2.31)$$

The timing law is equivalently specified by the speed profile $v(t) = \dot{s}(t)$ or the phase velocity $v(s) = ds/dt$ expressed as a function of arc length.

Different timing laws produce different motions through the same geometric path:



Three timing laws



Same spatial path

2.7.1 The Path–Timing Decomposition of Control

The reparameterization framework leads to a clean decomposition of the control problem:

Principle 2.4. Path–Timing Separation

The trajectory control problem decomposes into two subproblems:

- (A) **Path design:** Choose the geometric curve $\gamma^*(s)$ in state space that achieves the task (e.g., delivers the clubhead to the ball with the correct velocity vector).
- (B) **Timing design:** Choose the timing law $s(t)$ that balances speed against tracking difficulty, control effort, and robustness.

These two problems have different objective functions and constraints:

	Path design	Timing design
Objective	Task performance	Trackability & effort
Key quantity	Curvature $\kappa(s)$	Speed profile $v(s)$
Constraint	Dynamic feasibility	Actuator limits

The interaction between the two is captured by the curvature cost $\kappa(s) v(s)^2$, which the timing law must keep within the controller's authority at every point.

Example 2.5. Timing the Downswing

Suppose the downswing path has a curvature profile $\kappa(s)$ with a peak of $\kappa_{\max}$ at the transition point. The maximum centripetal acceleration the controller can provide is $a_{\max}$. Then the speed at the transition is bounded by

$$v_{\text{transition}} \leq \sqrt{\frac{a_{\max}}{\kappa_{\max}}}. \quad (2.32)$$

This is a *geometric speed limit*: no matter how powerful the actuators, the system cannot traverse a tight bend faster than its control authority allows. The timing law must respect this limit, typically by slowing down through high-curvature regions

and accelerating through straight sections.

This is precisely what golfers do instinctively. The “pause at the top”—the brief deceleration between backswing and downswing—is not aesthetic affectation. It is the timing law respecting the curvature speed limit at the point of maximum path curvature.

2.8 The Osculating Objects

The Frenet–Serret frame defines a sequence of “best-fit” geometric objects at each point of the curve. These osculating objects provide increasingly refined local approximations that are useful for controller design.

Definition 2.8. Osculating Objects

At a point $\gamma^*(s_0)$ on a curve in $\mathbb{R}^n$:

- (i) The **osculating line** is the tangent line: the best linear approximation.
- (ii) The **osculating circle** lies in the $(\mathbf{T}, \mathbf{N})$ plane, has radius $R = 1/\kappa$, and center at $\gamma^*(s_0) + R\mathbf{N}$. It is the best second-order (circular) approximation.
- (iii) The **osculating plane** is the plane spanned by $\mathbf{T}$ and $\mathbf{N}$. Torsion measures how fast the curve leaves this plane.
- (iv) The **osculating sphere** is the best third-order approximation and accounts for both curvature and torsion.

For trajectory control, the osculating circle is the most immediately useful. It tells us: *if the system were constrained to follow a circle at this point, what circle would it be?* The radius of that circle is the radius of curvature $R = 1/\kappa$, and the center indicates the direction toward which the trajectory bends.

2.9 The Geometry of Trajectory Deviations

We now use the Frenet–Serret frame to analyze what happens when the system deviates from the reference trajectory. This provides the geometric foundation for the transverse linearization of Chapter 4.

2.9.1 Frenet–Serret Coordinates

Given a reference trajectory $\gamma^*(s)$ and the associated Frenet–Serret frame $\{\mathbf{e}_1(s), \dots, \mathbf{e}_p(s)\}$, any state $\mathbf{x}$ near the trajectory can be decomposed as:

$$\mathbf{x} = \gamma^*(s^*) + \sum_{k=2}^n \xi_k \mathbf{e}_k(s^*), \quad (2.33)$$

where $s^* = s^*(\mathbf{x})$ is the projection phase (the arc-length parameter of the nearest point on the reference), and $\xi_2, \dots, \xi_n$ are the **transverse coordinates** measuring deviation in each normal direction.

Definition 2.9. Frenet–Serret Coordinates

The **Frenet–Serret coordinate representation** of a state $\mathbf{x}$ relative to a reference trajectory γ^* is the tuple

$$(s^*, \xi_2, \xi_3, \dots, \xi_n), \quad (2.34)$$

where s^* is the tangential (along-trajectory) coordinate and $\boldsymbol{\xi} = (\xi_2, \dots, \xi_n)$ are the transverse (cross-trajectory) coordinates. The transverse distance from Chapter 1 is

$$d_\perp = \|\boldsymbol{\xi}\| = \sqrt{\xi_2^2 + \dots + \xi_n^2}. \quad (2.35)$$

These coordinates have a powerful property: the reference trajectory itself corresponds to $\boldsymbol{\xi} = \mathbf{0}$, and the trajectory-tracking problem becomes the problem of *regulating $\boldsymbol{\xi}$ to zero while s^* advances*. This converts the tracking problem into a regulation problem, but in the moving frame rather than a fixed frame.

2.9.2 Curvature-Induced Coupling

There is a subtlety: the Frenet–Serret coordinates are not inertial. Because the frame rotates as it moves along the curve (governed by the Frenet–Serret equations (2.20)), fictitious forces appear in the transverse dynamics. The dominant effect is a **curvature-induced coupling**:

Proposition 2.3. *In Frenet–Serret coordinates, the linearized transverse dynamics about the reference trajectory have the form*

$$\dot{\boldsymbol{\xi}} = [\mathbf{A}(s) - v^2 \mathbf{K}(s)] \boldsymbol{\xi} + \mathbf{B}(s) \mathbf{u}_{fb}, \quad (2.36)$$

where $\mathbf{A}(s)$ comes from linearizing the original dynamics, and $\mathbf{K}(s)$ is a matrix that depends on the generalized curvatures $\kappa_1(s), \dots, \kappa_{p-1}(s)$. The term $v^2 \mathbf{K}(s)$ represents the geometric coupling between the curve’s shape and the transverse stability.

This result has a far-reaching implication: *the stability of trajectory tracking depends on the geometry of the trajectory*. A trajectory with low curvature is inherently easier to stabilize than one with high curvature, even if the underlying system dynamics are identical. The curvature modifies the effective dynamics transverse to the path.

2.10 Fundamental Theorem and Uniqueness

We close this chapter with a theorem that is central to the trajectory-first philosophy.

Theorem 2.4 (Fundamental Theorem of Curves). *Let $\kappa_1(s), \dots, \kappa_{p-1}(s)$ be smooth functions on $[0, L]$ with $\kappa_k(s) > 0$ for $k = 1, \dots, p-2$. Then there exists a curve $\gamma : [0, L] \rightarrow \mathbb{R}^n$, parameterized by arc length, whose generalized curvatures are exactly $\kappa_1, \dots, \kappa_{p-1}$. Moreover, this curve is unique up to rigid motions (translations and rotations) of $\mathbb{R}^n$.*

This theorem says: **the curvature signature is the trajectory.** Two trajectories with the same curvature functions are identical in shape. This has a practical corollary for control:

Corollary 2.5. *To specify a reference trajectory, it suffices to specify the generalized curvatures as functions of arc length, together with an initial position and orientation. This representation is:*

- (i) Compact: $p - 1$ scalar functions instead of n coordinate functions.
- (ii) Intrinsic: independent of coordinate choice.
- (iii) Timing-independent: the trajectory shape is fully determined without specifying when each point is reached.

Example 2.6. Curvature Signature of a Golf Swing

A simplified 2-DOF model of the golf swing (arm + club) has a 4-dimensional state space $(\theta_{\text{arm}}, \theta_{\text{club}}, \dot{\theta}_{\text{arm}}, \dot{\theta}_{\text{club}})$. The trajectory through this space has up to 3 generalized curvatures: $\kappa_1(s)$, $\kappa_2(s)$, and $\kappa_3(s)$.

The curvature signature reveals the structure of the swing:

- $\kappa_1(s)$ peaks at the transition (the “pause at the top”).
- $\kappa_2(s)$ peaks during the release, where the swing plane rotates.
- $\kappa_3(s)$ is small for a well-executed swing (the motion is approximately confined to a 3D subspace of $\mathbb{R}^4$).

A coach’s instruction to “flatten your swing plane” is, in geometric terms, a request to reduce $\kappa_2(s)$ during a specific phase of the motion—a precise modification to the curvature signature.

2.11 Summary

This chapter has established the geometric language we will use throughout the book:

Concept	Control significance
Arc length s	The natural “distance along the path”—separates shape from timing
Curvature $\kappa(s)$	Determines the centripetal control demand $a_{\perp} = \kappa v^2$
Torsion $\tau(s)$	Measures out-of-plane complexity; relates to multi-joint coordination
Frenet–Serret frame	Provides the natural moving coordinate system for decomposing deviations into tangential and transverse components
Curvature signature	Uniquely specifies the trajectory shape, independent of timing and coordinates
Path–timing separation	Decomposes control into geometric path design and temporal speed planning

The key takeaway is that a trajectory is not just “a function of time.” It is a geometric object with intrinsic structure, and this structure directly determines the difficulty and requirements of controlling it. In Chapter 3, we will see that the state spaces of mechanical systems are not flat $\mathbb{R}^n$ —they are curved manifolds with their own geometry. The interplay between the geometry of the *curve* and the geometry of the *space* will produce the richest and most practically important results of this book.

2.12 Exercises

- 2.1. (Computation.)** Compute the curvature and torsion of the curve $\gamma(t) = (t, t^2, t^3)$ at $t = 0$ and $t = 1$. At which point is the curve “harder to track” at constant speed?
- 2.2. (Arc length.)** The logarithmic spiral $\gamma(t) = e^{at}(\cos t, \sin t)$ with $a > 0$ is a model for certain biological growth patterns.
- Find the arc-length function $s(t)$.
 - Show that the curvature is $\kappa = 1/(e^{at}\sqrt{1+a^2})$ and interpret what this means about tracking difficulty as the spiral grows.
 - What timing law $s(t)$ would maintain constant centripetal acceleration?
- 2.3. (Frenet–Serret.)** For the curve $\gamma(s) = (\cos(s/\sqrt{2}), \sin(s/\sqrt{2}), s/\sqrt{2})$ (a helix parameterized by arc length):
- Verify that $\|\gamma'(s)\| = 1$.
 - Compute the Frenet–Serret frame $\{\mathbf{T}, \mathbf{N}, \mathbf{B}\}$.
 - Compute κ and τ and verify the Frenet–Serret equations.
- 2.4. (Path–timing separation.)** A robot end-effector must trace the path $\mathbf{p}(s) = (s, \sin(\pi s), 0)$ for $s \in [0, 2]$ (one full sine wave).

- (a) Compute $\kappa(s)$ and identify the points of maximum curvature.
 - (b) If the maximum centripetal acceleration is $a_{\max} = 10 \text{ m/s}^2$, find the speed limit $v_{\max}(s)$ along the path.
 - (c) Design a timing law $s(t)$ that traverses the path in minimum time while respecting the speed limit. Sketch $v(s)$ and $s(t)$.
- 2.5. (Conceptual.)** Consider two trajectories in $\mathbb{R}^4$ with identical curvature signatures $(\kappa_1(s), \kappa_2(s), \kappa_3(s))$ but different initial conditions.
- (a) By the fundamental theorem, how are these trajectories related?
 - (b) If you designed a tracking controller for one, would it work for the other? Under what conditions?
 - (c) Relate this to the idea that a well-learned golf swing can be started from slightly different address positions.
- 2.6. (Numerical project.)** Using motion capture data (or simulated data from a double pendulum), compute the curvature signature of a swing-like motion:
- (a) Numerically estimate $s(t)$ by integrating $\|\dot{\mathbf{x}}(t)\|$.
 - (b) Compute $\kappa_1(s)$ using Eq. (2.13).
 - (c) Identify the phases of the motion (backswing, transition, downswing, impact) from features of the curvature profile.
 - (d) Plot the curvature cost $\kappa(s)v(s)^2$ and identify the most demanding phase of the motion.

*The shape of a river is written in its curvature.
The story of a motion is written the same way.*

Learn to read the curvature, and you read the motion.

Chapter 3

Configuration Manifolds

The map is not the territory. The coordinate chart is not the manifold. Every time you write θ for an angle, you are lying—just a little.

In Plain Language 3.1. Angles Are Not Numbers

When we measure the position of a car on a straight road, we use a regular number: 0 miles, 1 mile, 2 miles, stretching on forever. But think about a spinning wheel or a robotic joint. If you rotate it by 360 degrees, it ends up in the exact same physical position as 0 degrees. Regular numbers don't work like this ($0 \neq 360$). If we force angles to act like regular numbers, our math gets tangled up in "singularities" or sudden jumps, confusing the control system. To fix this, we have to admit that the space of possible rotations is more like the surface of a sphere or a donut than a flat sheet of paper. This chapter explores how to do math on these curved spaces, known as "manifolds."

3.1 The Lie That Coordinates Tell

In Chapter 2 we developed the geometry of curves in $\mathbb{R}^n$. This was a necessary starting point, but it was also a simplification. The state spaces of real mechanical systems are *not* $\mathbb{R}^n$.

Consider the simplest possible rotating joint: a hinge. Its configuration is an angle θ , which we habitually write as a real number. But an angle is not a real number. The configuration $\theta = 0$ and the configuration $\theta = 2\pi$ are *the same physical state*. No point in $\mathbb{R}$ has this property. The true configuration space of a hinge is the *circle* S^1 —a one-dimensional manifold that is locally like $\mathbb{R}$ but globally has a completely different topology.

This is not pedantry. It is the source of real engineering failures.

Warning 3.1. The Gimbal Lock Catastrophe

If we represent orientation using three Euler angles $(\phi, \theta, \psi) \in \mathbb{R}^3$, we encounter **gimbal lock**: configurations where the representation degenerates and loses a degree of freedom. At $\theta = \pm\pi/2$, the axes for ϕ and ψ align, making the system appear to

have only 2 DOF when it actually has 3.

This is not a hardware problem—it is a *coordinate singularity*, an artifact of forcing a non-Euclidean space ($\text{SO}(3)$) into Euclidean coordinates ($\mathbb{R}^3$). Apollo 13’s inertial measurement unit experienced gimbal lock operationally. Every modern spacecraft avoids Euler angles for precisely this reason.

The lesson for control: **if you use the wrong coordinates, your controller will have singularities that the physical system does not.**

Key Idea 3.1. The Manifold Imperative

The configuration space of a mechanical system is a *smooth manifold* $\mathcal{Q}$ —a space that is locally like $\mathbb{R}^n$ but may have nontrivial global topology. Control theory built on $\mathbb{R}^n$ must be replaced by control theory built on $\mathcal{Q}$ whenever:

- (i) Joints allow full rotation (wrapping around S^1 or $\text{SO}(3)$).
- (ii) The trajectory traverses a significant fraction of the configuration space.
- (iii) Singularity-free representation is required for robustness.

For the golf swing—which involves large rotations in multiple joints simultaneously—all three conditions are met.

3.2 Smooth Manifolds: The Minimum You Need

We develop only as much manifold theory as the control engineer needs. The reader seeking a comprehensive treatment should consult Lee (2012) or Abraham & Marsden (1978).

In Plain Language 3.2. Mathematical Reminder: The Foundations Re-applied

In Volume 0, we introduced Manifolds and Tangent Spaces as abstract geometric concepts to explain configuration and rotation. Here in Volume II, we mathematically formalize them to build robust control algorithms. Remember the core definitions:

1. **Manifold ($\mathcal{Q}$):** The curved, global "position" space of the robot.
2. **Tangent Space ($T_q\mathcal{Q}$):** The perfectly flat, linear "velocity" space attached to one specific position.
3. **Tangent Bundle ($T\mathcal{Q}$):** The total **State Space** combining all positions and all their velocities.

Our motor torques compute inside the flat tangent space. We rely on differential geometry to project those flat velocities back onto the curved physical reality.

Definition 3.1. Smooth Manifold

A **smooth manifold** $\mathcal{M}$ of dimension n is a set together with a collection of **coordinate charts** $\{(U_\alpha, \varphi_\alpha)\}$ such that:

- (i) Each $U_\alpha \subseteq \mathcal{M}$ is an open set, and the U_α cover $\mathcal{M}$.
- (ii) Each $\varphi_\alpha : U_\alpha \rightarrow \mathbb{R}^n$ is a homeomorphism onto an open subset of $\mathbb{R}^n$.
- (iii) Where two charts overlap, the **transition maps** $\varphi_\beta \circ \varphi_\alpha^{-1}$ are smooth (C^∞) functions from $\mathbb{R}^n$ to $\mathbb{R}^n$.

The key intuition is: a manifold is a space where you can *locally* use coordinates, but no single coordinate system works *globally*. You need an atlas of overlapping charts, like a road atlas needs multiple pages to cover a country.

Example 3.1. The Circle S^1

The circle $S^1 = \{(\cos \theta, \sin \theta) : \theta \in \mathbb{R}\}$ is a 1-dimensional manifold. It cannot be covered by a single coordinate chart: any angle parameterization $\theta \in [0, 2\pi)$ has a discontinuity at $\theta = 0 = 2\pi$. We need at least two charts:

$$U_1 = S^1 \setminus \{(-1, 0)\}, \quad \varphi_1(p) = \theta \in (-\pi, \pi), \quad (3.1)$$

$$U_2 = S^1 \setminus \{(1, 0)\}, \quad \varphi_2(p) = \theta \in (0, 2\pi). \quad (3.2)$$

On the overlap $U_1 \cap U_2$, the transition map is either the identity or a shift by 2π —both smooth.

For a hinge joint, this means: no single angle variable θ can represent all configurations without a discontinuity. If the joint rotates past the discontinuity, any controller using that angle will experience a “jump”—a coordinate artifact, not a physical event.

3.2.1 Tangent Spaces and Tangent Vectors

On a manifold, there is no global notion of “direction.” Velocity is a *local* concept, defined at each point.

Definition 3.2. Tangent Space

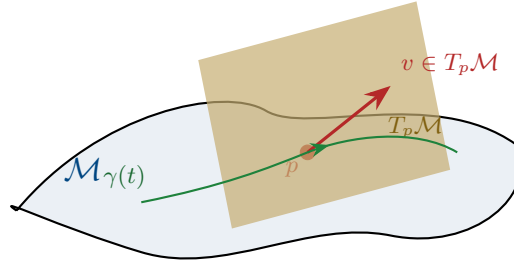
The **tangent space** $T_p\mathcal{M}$ at a point $p \in \mathcal{M}$ is the vector space of all possible velocity vectors at p . If $\mathcal{M}$ has dimension n , then $T_p\mathcal{M} \cong \mathbb{R}^n$ as a vector space, but each point has its *own* tangent space.

Given a curve $\gamma : (-\varepsilon, \varepsilon) \rightarrow \mathcal{M}$ with $\gamma(0) = p$, the tangent vector $\dot{\gamma}(0)$ is an element of $T_p\mathcal{M}$.

Definition 3.3. Tangent Bundle

The **tangent bundle** $T\mathcal{M} = \bigsqcup_{p \in \mathcal{M}} T_p\mathcal{M}$ is the collection of all tangent spaces, one for each point. It is itself a smooth manifold of dimension $2n$. A point in $T\mathcal{M}$ is a pair (p, v) where $p \in \mathcal{M}$ and $v \in T_p\mathcal{M}$.

For a mechanical system with configuration manifold $\mathcal{Q}$, the **state space is the tangent bundle** $\mathcal{X} = T\mathcal{M}$. A state is a pair $(\mathbf{q}, \dot{\mathbf{q}})$ —a configuration and a velocity.



3.3 The Configuration Manifolds of Joints

Every mechanical joint constrains relative motion between two rigid bodies. The set of allowed relative configurations forms a manifold whose topology depends on the joint type.

Joint type	DOF	Manifold	Topology
Revolute (hinge)	1	S^1	Circle
Prismatic (slider)	1	$\mathbb{R}$	Line (or interval)
Universal	2	S^2	2-sphere
Ball-and-socket	3	$\text{SO}(3)$	Rotation group
Planar	3	$\text{SE}(3) _{2D} = \mathbb{R}^2 \times S^1$	Plane + rotation
Free body	6	$\text{SE}(3) = \mathbb{R}^3 \rtimes \text{SO}(3)$	Rigid motions of 3-space

The configuration space of a multi-body system is the *product* of individual joint manifolds (modulo kinematic constraints):

Definition 3.4. Product Configuration Space

For a serial chain of k joints with individual configuration manifolds $\mathcal{M}_1, \dots, \mathcal{M}_k$, the total configuration space is

$$\mathcal{Q} = \mathcal{M}_1 \times \mathcal{M}_2 \times \dots \times \mathcal{M}_k. \quad (3.3)$$

For example, a robot arm with three revolute joints has $\mathcal{Q} = S^1 \times S^1 \times S^1 = \mathbb{T}^3$, the 3-dimensional torus.

Example 3.2. The Human Shoulder

The glenohumeral (shoulder) joint is approximately a ball-and-socket joint with configuration manifold $\text{SO}(3)$. Combined with the 1-DOF hinge at the elbow (S^1) and the 1-DOF pronation/supination of the forearm (S^1), the arm has configuration space

$$\mathcal{Q}_{\text{arm}} = \text{SO}(3) \times S^1 \times S^1. \quad (3.4)$$

This is a 5-dimensional manifold with the topology of $\text{SO}(3) \times \mathbb{T}^2$. No set of 5 angle variables can parameterize this space without singularities.

3.4 The Rotation Group $\text{SO}(3)$

The rotation group $\text{SO}(3)$ appears wherever a joint allows full 3D rotation. It is the single most important non-Euclidean manifold in mechanical control, and its properties drive many of the challenges we will encounter.

Definition 3.5. The Special Orthogonal Group

$\text{SO}(3)$ is the group of 3×3 real orthogonal matrices with determinant $+1$:

$$\text{SO}(3) = \{\mathbf{R} \in \mathbb{R}^{3 \times 3} : \mathbf{R}^\top \mathbf{R} = \mathbf{I}, \det(\mathbf{R}) = 1\}. \quad (3.5)$$

It is a 3-dimensional compact Lie group. As a manifold, it is diffeomorphic to the real projective space $\mathbb{R}P^3$.

3.4.1 Why Euler Angles Fail

The most common parameterization of $\text{SO}(3)$ uses three Euler angles (ϕ, θ, ψ) , defining the rotation matrix as a product of three elemental rotations. This parameterization has two fundamental problems:

- P1. Singularities.** Every 3-parameter representation of $\text{SO}(3)$ has at least one singularity. For the ZYX convention, gimbal lock occurs at $\theta = \pm\pi/2$. This is a topological necessity, not a flaw in the choice of convention: $\text{SO}(3)$ is not homeomorphic to any open subset of $\mathbb{R}^3$.
- P2. Non-uniqueness.** Euler angles are periodic: (ϕ, θ, ψ) and $(\phi + 2\pi, \theta, \psi)$ represent the same rotation. Near the singularity, drastically different angle triples can represent nearly identical orientations, creating numerical chaos in controllers.

Principle 3.1. Representation Principle for Rotations

For trajectory control on $\text{SO}(3)$, use one of:

- (a) **Rotation matrices** $\mathbf{R} \in \mathbb{R}^{3 \times 3}$: 9 parameters with 6 orthogonality constraints. Singularity-free but redundant.
- (b) **Unit quaternions** $\mathbf{q} \in S^3 \subset \mathbb{R}^4$: 4 parameters with 1 unit-norm constraint.

Singularity-free, minimal redundancy, excellent for interpolation and computation.

- (c) **Exponential coordinates** (axis-angle or Lie algebra): 3 parameters with a singularity at $\|\boldsymbol{\omega}\| = \pi$, but natural for small rotations and linearization.

Never use Euler angles for control of large-rotation trajectories.

3.4.2 The Lie Algebra $\mathfrak{so}(3)$

The tangent space of $\text{SO}(3)$ at the identity is the **Lie algebra** $\mathfrak{so}(3)$ —the space of 3×3 skew-symmetric matrices:

$$\mathfrak{so}(3) = \{\boldsymbol{\Omega} \in \mathbb{R}^{3 \times 3} : \boldsymbol{\Omega}^\top = -\boldsymbol{\Omega}\} = \left\{ \hat{\boldsymbol{\omega}} := \begin{pmatrix} 0 & -\omega_3 & \omega_2 \\ \omega_3 & 0 & -\omega_1 \\ -\omega_2 & \omega_1 & 0 \end{pmatrix} : \boldsymbol{\omega} \in \mathbb{R}^3 \right\}. \quad (3.6)$$

The “hat map” $(\cdot)^\wedge : \mathbb{R}^3 \rightarrow \mathfrak{so}(3)$ and its inverse “vee map” $(\cdot)^\vee : \mathfrak{so}(3) \rightarrow \mathbb{R}^3$ identify the Lie algebra with $\mathbb{R}^3$ (the space of angular velocity vectors).

Definition 3.6. Exponential Map

The **exponential map** $\exp : \mathfrak{so}(3) \rightarrow \text{SO}(3)$ connects the Lie algebra to the Lie group:

$$\mathbf{R} = \exp(\hat{\boldsymbol{\omega}}) = \mathbf{I} + \frac{\sin \theta}{\theta} \hat{\boldsymbol{\omega}} + \frac{1 - \cos \theta}{\theta^2} \hat{\boldsymbol{\omega}}^2, \quad (3.7)$$

where $\theta = \|\boldsymbol{\omega}\|$. This is the **Rodrigues formula**. Geometrically, $\mathbf{R}$ is a rotation by angle θ about the axis $\boldsymbol{\omega}/\theta$.

The inverse map $\log : \text{SO}(3) \rightarrow \mathfrak{so}(3)$ is well-defined for rotations with angle $\theta < \pi$ and gives the **rotation vector** representation.

The exponential map is the bridge between the linear world (the Lie algebra, where we can add and scale) and the nonlinear world (the Lie group, where the actual dynamics live). This bridge is the central tool for linearization on manifolds.

Remark 3.1. Lie Algebras and Controllability on Manifolds

The Lie algebra of vector fields is a central object in geometric control theory, but it should be read as a local accessibility tool rather than a universal reachability certificate. Agrachev and Sachkov [1] and Bullo and Lewis [5] show how the distribution generated by control vector fields and their iterated Lie brackets characterizes directions that can be produced infinitesimally, under the stated smoothness, rank, and constraint assumptions. For a simplified golf-swing model, the control vector fields may represent shoulder, elbow, hip, or wrist torques, while their brackets represent inertial coupling terms that are not directly commanded. A full-rank Lie closure supports an accessibility claim for the model; it does not by itself prove that every configuration is reachable under torque bounds, joint limits, contact constraints, fatigue, or a finite downswing time.

3.5 The Golf Swing Configuration Space

We now apply the manifold framework to our running example. The result will show exactly why Euclidean control theory is insufficient for the golf swing.

3.5.1 A Simplified Kinematic Model

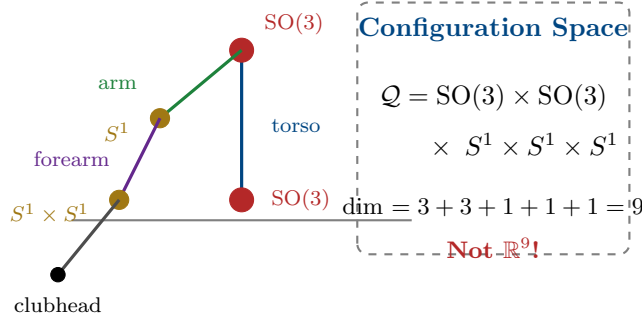
Consider a 4-segment model of the golf swing:

- (i) **Torso:** rotates relative to ground via the spine. Modeled as a 3-DOF ball joint: $\mathcal{M}_1 = \text{SO}(3)$.
- (ii) **Lead arm:** rotates relative to the torso via the shoulder. Modeled as a 3-DOF ball joint: $\mathcal{M}_2 = \text{SO}(3)$.
- (iii) **Forearm/hands:** rotates relative to the upper arm via the elbow (1 DOF) and wrist flexion (1 DOF): $\mathcal{M}_3 = S^1 \times S^1$.
- (iv) **Club:** rotates relative to the hands via wrist cock/uncock (1 DOF): $\mathcal{M}_4 = S^1$.

The total configuration space is:

$$\mathcal{Q}_{\text{golf}} = \text{SO}(3) \times \text{SO}(3) \times S^1 \times S^1 \times S^1. \quad (3.8)$$

This is a **9-dimensional manifold**. Its topology is $\text{SO}(3) \times \text{SO}(3) \times \mathbb{T}^3$ —decidedly not $\mathbb{R}^9$.



3.5.2 Why This Matters for Control

The state space (tangent bundle) has dimension $2 \times 9 = 18$. A trajectory through this space is a curve on an 18-dimensional manifold. The consequences for control are immediate:

- C1. No global linearization.** You cannot write $\mathbf{x} = \mathbf{x}^* + \delta\mathbf{x}$ with $\delta\mathbf{x} \in \mathbb{R}^{18}$, because the “+” operation is not defined globally on the manifold. Linearization must be done in the *tangent space*, using the exponential map.
- C2. Error is not a vector.** The “error” between two configurations $\mathbf{q}_1, \mathbf{q}_2 \in \mathcal{Q}$ is not $\mathbf{q}_2 - \mathbf{q}_1$ (subtraction is undefined on manifolds). The correct notion of error is the *logarithmic map*: $\mathbf{e} = \log(\mathbf{q}_1^{-1}\mathbf{q}_2)$, which lives in the Lie algebra.

C3. Geodesics replace straight lines. The “shortest path” between two configurations is not a straight line in any coordinate system—it is a *geodesic* on the manifold, determined by the Riemannian metric (mass-inertia matrix).

C4. Parallel transport replaces vector addition. To compare tangent vectors (velocities, errors) at different points on the trajectory, we must *transport* them along the manifold using the connection. Simply subtracting velocity vectors at different times is mathematically meaningless.

Warning 3.2. Euler Angle Catastrophe in Golf Modeling

Many biomechanics studies parameterize the golf swing using Euler angles for each joint. For the shoulder joint ($\text{SO}(3)$), this means three angles $(\phi_s, \theta_s, \psi_s)$. But during the backswing, the shoulder passes through configurations near gimbal lock ($\theta_s \approx \pm\pi/2$).

In these regions:

- The Jacobian mapping angular velocities to Euler angle rates becomes singular.
- Small physical rotations produce large jumps in angle coordinates.
- Any controller computing torques from angle errors will produce *infinite* commanded torques at the singularity.

The fix is not to “choose a better Euler angle convention” (all conventions have singularities somewhere). The fix is to work on $\text{SO}(3)$ directly, using rotation matrices or quaternions.

3.6 Riemannian Metrics and the Mass Matrix

A manifold by itself has no notion of distance, angle, or length. To measure these, we need a **Riemannian metric**—a smooth choice of inner product on each tangent space.

Definition 3.7. Riemannian Metric

A **Riemannian metric** on a manifold $\mathcal{M}$ is a smooth assignment of an inner product $\langle \cdot, \cdot \rangle_p$ on each tangent space $T_p\mathcal{M}$. In local coordinates $\mathbf{q} = (q_1, \dots, q_n)$, the metric is represented by a positive-definite matrix $\mathbf{G}(\mathbf{q})$:

$$\langle \mathbf{v}, \mathbf{w} \rangle_{\mathbf{q}} = \mathbf{v}^\top \mathbf{G}(\mathbf{q}) \mathbf{w}, \quad \mathbf{v}, \mathbf{w} \in T_{\mathbf{q}}\mathcal{M}. \quad (3.9)$$

For mechanical systems, nature provides a canonical Riemannian metric: the **kinetic energy metric**.

Principle 3.2. The Mass Matrix as Riemannian Metric

For a mechanical system with generalized coordinates $\mathbf{q}$ and mass-inertia matrix

$\mathbf{M}(\mathbf{q})$, the kinetic energy

$$T = \frac{1}{2} \dot{\mathbf{q}}^\top \mathbf{M}(\mathbf{q}) \dot{\mathbf{q}} \quad (3.10)$$

defines a Riemannian metric $\mathbf{G} = \mathbf{M}(\mathbf{q})$ on $\mathcal{Q}$. This metric has deep physical significance:

- (i) **Distance** between configurations is measured in “units of kinetic energy”—configurations connected by high-inertia motions are “far apart.”
- (ii) **Geodesics** (shortest paths) under this metric are the trajectories of the *unforced* system—free motions with no applied torques.
- (iii) **Arc length** from Chapter 2, computed with this metric, naturally weights each DOF by its effective inertia.

Example 3.3. Inertia-Weighted Distance in the Swing

Consider two configurations that differ by either:

- (a) A 10° rotation of the torso (high inertia, $I \approx 12 \text{ kg} \cdot \text{m}^2$), or
- (b) A 10° rotation of the wrist (low inertia, $I \approx 0.03 \text{ kg} \cdot \text{m}^2$).

In Euclidean coordinates, both represent “ 10° of change.” In the kinetic energy metric, the torso rotation is $\sqrt{12/0.03} = 20$ times “farther” than the wrist rotation. This correctly reflects the physical reality: rotating the torso requires 400 times more energy than rotating the wrist by the same angle.

The curvature of a trajectory measured in this metric therefore naturally captures the *physical* difficulty of the motion, not merely the angular change.

3.6.1 Geodesics as Natural Trajectories

Definition 3.8. Geodesic

A **geodesic** on a Riemannian manifold $(\mathcal{M}, \mathbf{G})$ is a curve that locally minimizes arc length. Geodesics satisfy the **geodesic equation**:

$$\ddot{q}^i + \sum_{j,k} \Gamma_{jk}^i(\mathbf{q}) \dot{q}^j \dot{q}^k = 0, \quad (3.11)$$

where Γ_{jk}^i are the **Christoffel symbols** of the metric $\mathbf{G}$.

The remarkable fact is that the geodesic equation (3.11) is *exactly* the equation of motion for the unforced mechanical system $\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} = \mathbf{0}$. The Coriolis and centrifugal terms are precisely the Christoffel symbol terms. Free motions of mechanical systems are geodesics on the configuration manifold equipped with the kinetic energy metric.

Principle 3.3. Geodesics and Passive Dynamics

For trajectory design, geodesics represent the “easiest” paths—the motions the system would naturally follow with no applied forces. A well-designed trajectory stays close to geodesics where possible, deviating only where task requirements (e.g., impact conditions) demand it.

In the golf swing, the follow-through is nearly geodesic: after impact, the golfer relaxes and the body follows its natural, inertia-weighted path through configuration space. The downswing deviates significantly from the geodesic because the task (delivering maximum clubhead speed at impact) requires active torques to redirect the motion.

3.7 Curves on Manifolds Revisited

With the manifold framework in place, we can extend every concept from Chapter 2 to non-Euclidean configuration spaces.

3.7.1 Covariant Differentiation

On $\mathbb{R}^n$, we differentiate vector fields by taking component-wise derivatives. On a manifold, this does not work—the tangent spaces at different points are different vector spaces, so we cannot simply subtract vectors at different points.

Definition 3.9. Covariant Derivative

The **covariant derivative** (or **Levi-Civita connection**) ∇ is the unique way to differentiate vector fields along curves on a Riemannian manifold that:

- (i) Is compatible with the metric: $\frac{d}{dt} \langle \mathbf{V}, \mathbf{W} \rangle = \langle \nabla_{\dot{\gamma}} \mathbf{V}, \mathbf{W} \rangle + \langle \mathbf{V}, \nabla_{\dot{\gamma}} \mathbf{W} \rangle$.
- (ii) Is torsion-free: $\nabla_{\mathbf{V}} \mathbf{W} - \nabla_{\mathbf{W}} \mathbf{V} = [\mathbf{V}, \mathbf{W}]$.

In local coordinates, the covariant derivative of a vector field $\mathbf{V}$ along a curve $\gamma(t)$ is

$$(\nabla_{\dot{\gamma}} \mathbf{V})^i = \dot{V}^i + \sum_{j,k} \Gamma_{jk}^i \dot{\gamma}^j V^k. \quad (3.12)$$

The covariant derivative replaces the ordinary derivative in all of our Chapter 2 formulas. The generalized curvatures, the Frenet–Serret frame, and the transverse decomposition all carry over, with d/ds replaced by $\nabla_{\dot{\gamma}}$.

3.7.2 Manifold Curvature of Trajectories

Definition 3.10. Geodesic Curvature

The **geodesic curvature** of a curve $\gamma(s)$ on a Riemannian manifold, parameterized by arc length, is

$$\kappa_g(s) = \|\nabla_{\dot{\gamma}} \dot{\gamma}\|. \quad (3.13)$$

This measures how much the curve deviates from a geodesic. A geodesic has $\kappa_g = 0$ everywhere.

Geodesic curvature is the manifold generalization of the curvature from Chapter 2, and it has the same control interpretation: *geodesic curvature measures the force per unit mass needed to follow the curve*, beyond what the natural dynamics would provide for free.

Proposition 3.1. *The control force required to track a trajectory $\gamma^*(s)$ at speed v on a configuration manifold with metric $\mathbf{M}(\mathbf{q})$ has magnitude*

$$\|\boldsymbol{\tau}_\perp\| = \kappa_g(s) v^2, \quad (3.14)$$

where $\|\cdot\|$ is measured in the dual (force) metric. Comparing with the Euclidean formula $a_\perp = \kappa v^2$ from Chapter 2: the Euclidean curvature κ is replaced by the geodesic curvature κ_g , which accounts for the curvature of the manifold itself.

This is a deeper version of the Curvature–Authority Principle. The control demand is reduced along directions where the manifold’s own curvature “helps”—i.e., where the natural dynamics carry the system in the desired direction. The control demand increases where the trajectory fights the natural dynamics.

Example 3.4. Geodesic Curvature of the Downswing

During the downswing, the trajectory has two sources of curvature:

- (a) **Euclidean curvature:** the path bends in coordinate space (the club changes direction).
- (b) **Manifold curvature:** the Christoffel symbol terms ($\Gamma_{jk}^i \dot{q}^j \dot{q}^k$) represent Coriolis and centrifugal effects that curve the natural trajectories.

The geodesic curvature κ_g is the *difference*. When the Coriolis/centrifugal terms push the system in the direction the trajectory wants to go (as happens during the wrist release, where centrifugal acceleration “unhinges” the club), κ_g is *less* than the Euclidean κ . The passive dynamics are doing work for free.

This is the geometric explanation for the “free speed” of the golf swing: a well-designed trajectory aligns with the manifold’s geodesics during the speed-critical phase, exploiting the natural dynamics rather than fighting them.

3.8 Error Metrics on Manifolds

We now address the critical question: how do we measure tracking error when the state space is a manifold?

3.8.1 The Logarithmic Error

Definition 3.11. Configuration Error on a Lie Group

For two configurations $\mathbf{q}_1, \mathbf{q}_2 \in \mathcal{Q}$ where $\mathcal{Q}$ is a Lie group, the **error** is

$$\mathbf{e} = \log(\mathbf{q}_1^{-1} \mathbf{q}_2) \in \text{Lie algebra.} \quad (3.15)$$

For the rotation group $\text{SO}(3)$, this gives

$$\mathbf{e}_R = \log(\mathbf{R}_1^\top \mathbf{R}_2)^\vee \in \mathbb{R}^3, \quad (3.16)$$

which is the rotation vector needed to go from $\mathbf{R}_1$ to $\mathbf{R}_2$. This error is zero if and only if $\mathbf{q}_1 = \mathbf{q}_2$, and it varies smoothly with both arguments (away from the cut locus at $\|\mathbf{e}\| = \pi$).

Remark 3.2. Why Not Just Subtract Quaternions?

Given two unit quaternions $\mathbf{q}_1$ and $\mathbf{q}_2$, a tempting error metric is $\mathbf{e} = \mathbf{q}_2 - \mathbf{q}_1$. But this lives in $\mathbb{R}^4$, not in the tangent space of S^3 . Worse, it violates the double-cover property: $\mathbf{q}$ and $-\mathbf{q}$ represent the same rotation, so the error $\mathbf{q}_2 - \mathbf{q}_1$ and $-\mathbf{q}_2 - \mathbf{q}_1$ should be “the same,” but they point in opposite directions.

The logarithmic error correctly handles the group structure and produces an error vector in the Lie algebra that has a clear physical interpretation (the angular velocity needed to correct the error in unit time).

3.8.2 The Trajectory Tube on a Manifold

The trajectory tube from Chapter 1 now generalizes naturally:

Definition 3.12. Manifold Trajectory Tube

Let $\gamma^* : [0, T] \rightarrow \mathcal{Q}$ be a reference trajectory on a Riemannian manifold $(\mathcal{Q}, \mathbf{G})$ and let $\varepsilon : [0, T] \rightarrow \mathbb{R}_{>0}$ be a tube radius function. The **manifold trajectory tube** is

$$\mathcal{T}(t) = \{ \mathbf{q} \in \mathcal{Q} : d_{\mathbf{G}}(\mathbf{q}, \gamma^*(t)) \leq \varepsilon(t) \}, \quad (3.17)$$

where $d_{\mathbf{G}}$ is the **geodesic distance**—the length of the shortest geodesic connecting $\mathbf{q}$ and $\gamma^*(t)$.

The geodesic distance correctly accounts for the topology of the configuration space. On $\text{SO}(3)$, the maximum distance between any two orientations is π (a half-rotation), and the tube wraps properly around the group.

3.9 Equations of Motion on Manifolds

The Euler–Lagrange equations on a manifold take a coordinate-free form that reveals the geometric structure:

Theorem 3.2 (Lagrangian Mechanics on Manifolds). *For a mechanical system on a configuration manifold $(\mathcal{Q}, M)$ with potential energy $V : \mathcal{Q} \rightarrow \mathbb{R}$ and generalized forces $\boldsymbol{\tau}$, the equations of motion are*

$$M(\mathbf{q}) \nabla_{\dot{\mathbf{q}}} \dot{\mathbf{q}} = -\text{grad } V + \mathbf{B} \mathbf{u}, \quad (3.18)$$

where $\nabla_{\dot{\mathbf{q}}} \dot{\mathbf{q}}$ is the covariant acceleration and $\text{grad } V$ is the Riemannian gradient of the potential energy.

In local coordinates, this reduces to the familiar $M\ddot{\mathbf{q}} + \mathbf{C}\dot{\mathbf{q}} + \mathbf{g} = \mathbf{B}\mathbf{u}$, but the coordinate-free form (3.18) makes clear that:

- The Coriolis matrix $\mathbf{C}$ is not a separate “term”—it is the Christoffel symbol contribution to the covariant derivative.
- The equation has the same form on any manifold, regardless of coordinate choice.
- The “centrifugal and Coriolis forces” are geometric artifacts of the manifold’s curvature, not real forces. On a flat manifold ($\mathbb{R}^n$), they vanish.

3.10 Practical Implications for Controller Implementation

We conclude with concrete guidance for implementing trajectory controllers on manifold-valued configuration spaces.

Principle 3.4. Manifold-Aware Controller Design

A trajectory tracking controller for a system on $\mathcal{Q}$ must:

- (i) **Compute errors using the logarithmic map**, not coordinate subtraction:

$$\mathbf{e}_q = \log((\mathbf{q}^*)^{-1} \mathbf{q}), \quad \mathbf{e}_v = \dot{\mathbf{q}} - \text{PT}_{q^* \rightarrow q}(\dot{\mathbf{q}}^*), \quad (3.19)$$

where PT denotes parallel transport of the reference velocity to the current configuration.

- (ii) **Compute feedforward using the covariant derivative**, not ordinary differentiation of coordinates.
- (iii) **Design feedback gains in the tangent space**, then map the resulting control back to the manifold via the exponential map.
- (iv) **Monitor for cut-locus proximity**: when the error approaches π (for $\text{SO}(3)$), the logarithmic map becomes ill-conditioned. Switch to a recovery strategy.

Example 3.5. PD Control on $\text{SO}(3)$

The manifold-correct PD controller for tracking a reference orientation $\mathbf{R}^*(t)$ with a rigid body (moment of inertia $\mathbf{J}$, angular velocity $\boldsymbol{\omega}$) is:

$$\boldsymbol{\tau} = -k_p \log(\mathbf{R}^{*\top} \mathbf{R})^\vee - k_d (\boldsymbol{\omega} - \mathbf{R}^{*\top} \mathbf{R} \boldsymbol{\omega}^*) + \mathbf{J} \dot{\boldsymbol{\omega}}^* + \boldsymbol{\omega} \times \mathbf{J} \boldsymbol{\omega}, \quad (3.20)$$

where the first term is the manifold-correct proportional error (logarithmic map), the second is the velocity error (parallel-transported), and the last two terms are the feedforward (covariant derivative of the reference).

Compare with the naïve Euler-angle PD controller: $\tau = -k_p(\boldsymbol{\theta} - \boldsymbol{\theta}^*) - k_d(\dot{\boldsymbol{\theta}} - \dot{\boldsymbol{\theta}}^*)$. The naïve controller has singularities; Eq. (3.20) does not. The naïve controller produces torques proportional to angle differences (which can be misleading near $\pm\pi$); Eq. (3.20) produces torques proportional to the *true rotation* needed to correct the error.

3.11 Summary

Euclidean assumption	Manifold replacement
$\mathcal{Q} = \mathbb{R}^n$	$\mathcal{Q} = \text{SO}(3)^k \times \mathbb{T}^m \times \dots$
Error: $\mathbf{e} = \mathbf{q} - \mathbf{q}^*$	Error: $\mathbf{e} = \log((\mathbf{q}^*)^{-1}\mathbf{q})$
Derivative: $\ddot{\mathbf{q}}$	Covariant derivative: $\nabla_{\dot{\mathbf{q}}}\dot{\mathbf{q}}$
Shortest path: straight line	Shortest path: geodesic
Distance: $\ \mathbf{q}_2 - \mathbf{q}_1\ $	Geodesic distance: $d_{\mathbf{G}}(\mathbf{q}_1, \mathbf{q}_2)$
Curvature: κ (Ch. 2)	Geodesic curvature: κ_g
Trajectory tube: Euclidean ball	Geodesic ball
Coriolis terms: “extra forces”	Christoffel symbols: manifold geometry
PD: $-k_p\mathbf{e} - k_d\dot{\mathbf{e}}$	$-k_p \log(\cdot)^\vee - k_d(\text{parallel transport})$

The central message of this chapter is that the configuration spaces of mechanical systems have geometry, and this geometry matters for control. Working in $\mathbb{R}^n$ when the true space is $\text{SO}(3) \times \mathbb{T}^k$ introduces artificial singularities, incorrect error metrics, and controllers that fail when the system moves far from a nominal configuration.

The trajectory-first philosophy of this book demands that we take the geometry seriously from the start. The curves we designed in Chapter 2 live on manifolds, and the stability theory we will develop in Chapter 4 must respect the manifold structure. The payoff is controllers that work globally, without singularities, and that correctly exploit the natural dynamics encoded in the manifold’s Riemannian geometry.

3.12 Exercises

3.1. (Topology.) Determine the configuration manifold and its dimension for each system:

- (a) A planar double pendulum (two revolute joints in 2D).
- (b) A spherical pendulum (a point mass on a rigid rod, free to swing in all directions).

- (c) A spacecraft with two reaction wheels (the body rotates freely in 3D; each wheel adds one internal DOF).

For each, identify whether $\mathbb{R}^n$ coordinates would have singularities.

3.2. (Euler angle singularity.) For the ZYX Euler angle representation $\mathbf{R} = R_z(\phi) R_y(\theta) R_x(\psi)$:

- (a) Compute the Jacobian $\mathbf{J}$ relating angular velocity $\boldsymbol{\omega}$ to Euler angle rates $(\dot{\phi}, \dot{\theta}, \dot{\psi})$.
- (b) Show that $\mathbf{J}$ is singular at $\theta = \pm\pi/2$.
- (c) Find an orientation trajectory $\mathbf{R}(t)$ that passes through the singularity and compute the Euler angle rates. What happens?

3.3. (Logarithmic error.) For two rotations $\mathbf{R}_1 = \mathbf{I}$ and $\mathbf{R}_2 = R_z(\alpha)$ (rotation by α about z):

- (a) Compute $\log(\mathbf{R}_1^\top \mathbf{R}_2)^\vee$ for $\alpha = \pi/6, \pi/2, \pi, 5\pi/3$.
- (b) Compare with the “error” $\alpha - 0 = \alpha$ as a function of α . At what point do they differ significantly?
- (c) What happens at $\alpha = \pi$? Explain geometrically.

3.4. (Geodesics.) For a double pendulum with masses m_1, m_2 and lengths l_1, l_2 :

- (a) Write the mass matrix $\mathbf{M}(\theta_1, \theta_2)$.
- (b) Compute the Christoffel symbols (or use the Coriolis matrix formula $C_{ijk} = \frac{1}{2}(\partial M_{ij}/\partial q_k + \partial M_{ik}/\partial q_j - \partial M_{jk}/\partial q_i)$).
- (c) Numerically integrate the geodesic equation from initial condition $(\theta_1, \theta_2) = (0, 0)$ with $(\dot{\theta}_1, \dot{\theta}_2) = (1, 0)$. Plot the geodesic on $\mathbb{T}^2$.

3.5. (Manifold PD control.) Implement and compare:

- (a) An Euler-angle PD controller for a rigid body tracking a reference orientation $\mathbf{R}^*(t) = R_z(\omega t)$ (constant-speed rotation).
- (b) The manifold PD controller from Eq. (3.20) for the same task.
- (c) Run both controllers with the reference trajectory passing through $\theta = \pi/2$ (gimbal lock for ZYX convention). Compare the tracking errors and commanded torques.

3.6. (Golf swing configuration space.) For the 9-DOF golf swing model of Eq. (3.8):

- (a) How many Euler angles would be needed to parameterize the full configuration space? How many singularity surfaces exist?
- (b) Propose a quaternion-based parameterization for the two $\text{SO}(3)$ joints. What is the total number of parameters? What constraints must be maintained?
- (c) Discuss the trade-offs between the Euler angle and quaternion representations for numerical simulation of the golf swing.

*The sphere cannot be flattened without tearing.
The rotation group cannot be charted without singularity.
Do not fight the topology—ride it.*

*The manifold is not an obstacle. It is the landscape
through which every motion flows.*

Chapter 4

Orbital Stability and Transverse Linearization

Stability is not the absence of motion.
It is the faithfulness of motion.

In Plain Language 4.1. Being on Time vs. Being on Track

Imagine driving a car along a winding mountain road. If you are five seconds behind schedule but perfectly in the center of your lane, you are perfectly safe. If you are exactly on time but two feet outside your lane, you might crash. Classical control theory measures "error" by comparing where you are right now to where you were "scheduled" to be right now. By that logic, being five seconds late is considered an error that the system must fight to correct. This chapter introduces "Orbital Stability," which mathematically formalizes the idea that being early or late is fine, as long as you stay on the path.

4.1 The Problem with Lyapunov

The classical Lyapunov framework asks a simple question: *does the state converge to an equilibrium?* We demonstrated in Chapter 1 that this is the wrong question for systems whose purpose is to move. But the problem goes deeper than philosophical framing—Lyapunov stability is *structurally incapable* of analyzing trajectory-tracking systems.

Consider a system perfectly tracking a reference trajectory $\gamma^*(t)$. The state $\mathbf{x}(t)$ matches $\gamma^*(t)$ at every instant. Now apply a small timing perturbation: the system executes the same motion but shifted by ε seconds. The perturbed state is $\tilde{\mathbf{x}}(t) = \gamma^*(t - \varepsilon)$, and the time-indexed error is

$$\mathbf{e}(t) = \tilde{\mathbf{x}}(t) - \gamma^*(t) = \gamma^*(t - \varepsilon) - \gamma^*(t) \approx -\varepsilon \dot{\gamma}^*(t). \quad (4.1)$$

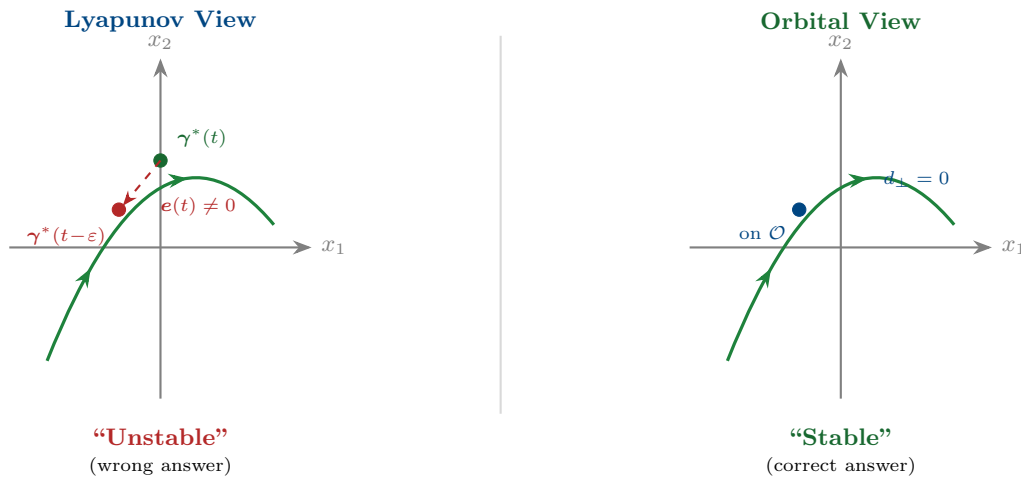
This error does *not* decay to zero—it persists indefinitely, oscillating with a magnitude proportional to $\varepsilon v(t)$ where v is the speed. By Lyapunov's definition, the trajectory is *unstable*. But the system is executing the correct motion perfectly! The only "error" is that it is slightly ahead of or behind schedule.

Key Idea 4.1. The Fundamental Inadequacy of Point Stability

Lyapunov stability conflates two independent phenomena:

- (i) **Transverse deviations:** the system drifts *away from* the trajectory path. This is a genuine control failure.
- (ii) **Tangential deviations:** the system is ahead of or behind schedule *along* the trajectory path. This is a timing difference, not a tracking failure.

Lyapunov stability rejects both. Orbital stability accepts tangential deviations and rejects only transverse ones. For systems whose purpose is motion, orbital stability is the correct notion.



4.2 Orbital Stability: Formal Definitions

We now define orbital stability rigorously. The key object is the **orbit**—the image of the trajectory as a set, stripped of all timing information.

Definition 4.1. Orbit

Given a trajectory $\gamma^* : [0, T] \rightarrow \mathcal{X}$, the **orbit** is the set

$$\mathcal{O} = \{\gamma^*(t) : t \in [0, T]\} \subset \mathcal{X}. \quad (4.2)$$

The orbit is a one-dimensional submanifold of $\mathcal{X}$ (a curve without parameterization). For periodic trajectories with period T , $\gamma^*(0) = \gamma^*(T)$ and $\mathcal{O}$ is a closed curve (a loop).

Definition 4.2. Orbital Stability

A trajectory $\gamma^*(t)$ (or its orbit $\mathcal{O}$) is **orbitally stable** if for every $\varepsilon > 0$ there exists $\delta > 0$ such that

$$d(\mathbf{x}(0), \mathcal{O}) < \delta \implies d(\mathbf{x}(t), \mathcal{O}) < \varepsilon \quad \forall t \geq 0, \quad (4.3)$$

where $d(\mathbf{x}, \mathcal{O}) = \inf_{p \in \mathcal{O}} \|\mathbf{x} - p\|$ is the distance from the state to the orbit.

It is **asymptotically orbitally stable** if additionally $d(\mathbf{x}(t), \mathcal{O}) \rightarrow 0$ as $t \rightarrow \infty$.

Critically, orbital stability does not require $\|\mathbf{x}(t) - \gamma^*(t)\| \rightarrow 0$. The system may drift along the orbit (ahead or behind schedule) while still satisfying the orbital-stability condition. Only deviations *away from* the orbit matter.

Definition 4.3. Orbital Stability with Asymptotic Phase

A trajectory is orbitally stable **with asymptotic phase** if it is asymptotically orbitally stable and additionally there exists a phase shift θ^* such that

$$\|\mathbf{x}(t) - \gamma^*(t + \theta^*)\| \rightarrow 0 \quad \text{as } t \rightarrow \infty. \quad (4.4)$$

This means the system not only returns to the orbit but eventually “locks in” to a definite timing, shifted by θ^* from the nominal schedule. This is the strongest practical form of trajectory stability.

Example 4.1. Limit Cycles: The Prototype

The Van der Pol oscillator $\ddot{x} - \mu(1 - x^2)\dot{x} + x = 0$ possesses a **limit cycle**—a periodic orbit that is asymptotically orbitally stable with asymptotic phase. All nearby trajectories spiral toward the limit cycle and eventually synchronize to a definite phase.

The limit cycle is the simplest example of a system whose purpose is to move: the “target” is not a point but an orbit, and stability means the orbit attracts nearby trajectories. Walking, running, and heartbeats are all limit cycles. The golf swing is a transient trajectory (not periodic), but the same stability concepts apply over the finite interval $[0, T]$.

4.3 The Transverse–Tangential Decomposition

To analyze orbital stability, we must decompose the dynamics into components along and across the orbit. This is the *transverse–tangential decomposition*, the central technical tool of this chapter.

4.3.1 Projection onto the Orbit

Definition 4.4. Nearest-Point Projection

Given a state $\mathbf{x}$ sufficiently close to the orbit $\mathcal{O}$, the **nearest-point projection** is

$$\pi(\mathbf{x}) = \arg \min_{p \in \mathcal{O}} \|\mathbf{x} - p\|, \quad (4.5)$$

and the associated **projected phase** is the unique $s^*(\mathbf{x})$ satisfying $\pi(\mathbf{x}) = \gamma^*(s^*)$. The projection is well-defined and smooth in a tubular neighborhood of $\mathcal{O}$ (as long as $\mathbf{x}$ is closer to $\mathcal{O}$ than the minimum radius of curvature of the orbit).

4.3.2 The Moving Hyperplane Decomposition

At each point $\gamma^*(s)$ on the orbit, the state space $\mathbb{R}^n$ decomposes into a one-dimensional tangential direction and an $(n-1)$ -dimensional transverse hyperplane:

Definition 4.5. Transverse Hyperplane

At the point $\gamma^*(s)$ on the orbit, define:

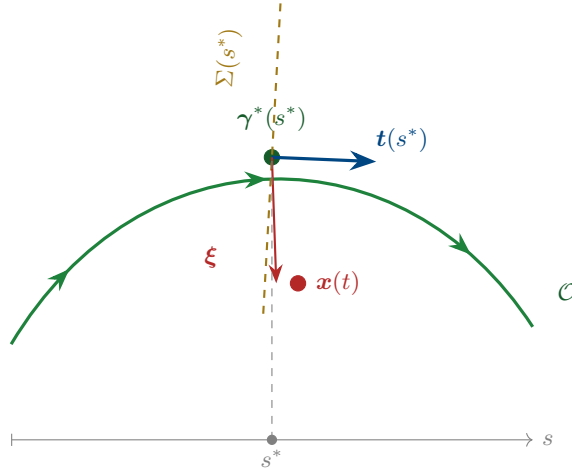
$$\text{Tangential direction: } \mathbf{t}(s) = \frac{\dot{\gamma}^*(s)}{\|\dot{\gamma}^*(s)\|}, \quad (4.6)$$

$$\text{Transverse hyperplane: } \Sigma(s) = \{\mathbf{v} \in \mathbb{R}^n : \mathbf{v}^\top \mathbf{t}(s) = 0\}. \quad (4.7)$$

A state $\mathbf{x}$ near $\gamma^*(s)$ decomposes as

$$\mathbf{x} = \gamma^*(s^*) + \boldsymbol{\xi}, \quad \boldsymbol{\xi} \in \Sigma(s^*), \quad (4.8)$$

where s^* is the projected phase and $\boldsymbol{\xi}$ is the **transverse deviation**—the component of the error perpendicular to the orbit. Orbital stability is equivalent to the stability of the origin $\boldsymbol{\xi} = \mathbf{0}$ in the transverse dynamics.



4.4 Transverse Linearization

The power of the transverse decomposition is that it reduces orbital stability to the stability of a *time-varying linear system*—the transverse linearization.

4.4.1 Deriving the Transverse Dynamics

Consider the nonlinear system $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u})$ with a reference trajectory $\gamma^*(t)$ satisfying $\dot{\gamma}^* = \mathbf{f}(\gamma^*, \mathbf{u}^*(t))$. Using the decomposition $\mathbf{x} = \gamma^*(s^*) + \boldsymbol{\xi}$ and linearizing about $\boldsymbol{\xi} = \mathbf{0}$:

Theorem 4.1 (Transverse Linearization [13, 21, 18]). *The linearized transverse dynamics about the orbit $\mathcal{O}$ are*

$$\dot{\boldsymbol{\xi}} = \mathbf{A}_\perp(s) \boldsymbol{\xi} + \mathbf{B}_\perp(s) \delta \mathbf{u}, \quad (4.9)$$

where $\delta \mathbf{u} = \mathbf{u} - \mathbf{u}^*$ is the feedback perturbation, and

$$\mathbf{A}_\perp(s) = \mathbf{P}(s) \left[\left. \frac{\partial \mathbf{f}}{\partial \mathbf{x}} \right|_{\gamma^*, \mathbf{u}^*} - \frac{\dot{\gamma}^* \otimes (\nabla_{\mathbf{x}} s^*)^\top}{\|\dot{\gamma}^*\|^2} \cdot \left. \frac{\partial \mathbf{f}}{\partial \mathbf{x}} \right|_{\gamma^*, \mathbf{u}^*} \right] \mathbf{P}(s), \quad (4.10)$$

$$\mathbf{B}_\perp(s) = \mathbf{P}(s) \left. \frac{\partial \mathbf{f}}{\partial \mathbf{u}} \right|_{\gamma^*, \mathbf{u}^*}, \quad (4.11)$$

and $\mathbf{P}(s) = \mathbf{I} - \mathbf{t}(s)\mathbf{t}(s)^\top$ is the orthogonal projector onto $\Sigma(s)$.

Remark 4.1. Understanding the Transverse $\mathbf{A}_\perp$

Equation (4.10) has a clear structure:

- The first term $\mathbf{P} \left. \frac{\partial \mathbf{f}}{\partial \mathbf{x}} \right|_{\gamma^*, \mathbf{u}^*} \mathbf{P}$ is the standard Jacobian projected onto the transverse space. This captures how the system dynamics affect cross-trajectory deviations.
- The second term subtracts the component of the Jacobian that acts along the tangent direction—this is exactly the tangential dynamics that orbital stability “forgets.”

The resulting $\mathbf{A}_\perp(s)$ has dimension $(n-1) \times (n-1)$, one less than the full state Jacobian. The removed eigenvalue corresponds to motion along the orbit—the tangential direction in which we are *not* demanding stability.

Principle 4.1. The Transverse Reduction Principle

The reference trajectory $\gamma^*(t)$ is asymptotically orbitally stable if and only if the origin of the transverse linearization (4.9) (with $\delta \mathbf{u} = \mathbf{0}$) is asymptotically stable. This reduces the orbital stability question from a nonlinear problem on the full n -dimensional state space to a *linear* problem on the $(n-1)$ -dimensional transverse space.

For controller design: if we can stabilize the linear system (4.9), we achieve orbital stability of the nonlinear system. All the tools of linear time-varying (LTV) control theory apply.

Remark 4.2. Adjoint Systems and Costate Evolution

The transverse linearization and feedback gain computation via time-varying LQR are intimately connected to the adjoint system and costate dynamics from the Pontryagin Maximum Principle, as developed in Agrachev and Sachkov [1]. When solving trajectory optimization (Chapter 6), Pontryagin introduces costate variables $\lambda(t)$ evolving backward according to adjoint dynamics. The Riccati matrix $P(t)$ in the transverse LQR feedback law connects directly to these costates. Thus orbital stabilization (this chapter) and trajectory optimization (Chapter 6) spring from the same Hamiltonian geometric principle underlying optimal control.

4.4.2 The Transverse Jacobian: A Practical Formula

For implementation, the key quantities are computed as follows. Let $J(t) = \frac{\partial f}{\partial x}|_{\gamma^*, u^*}$ be the Jacobian along the reference. Choose any orthonormal basis $\{n_1(s), \dots, n_{n-1}(s)\}$ for $\Sigma(s)$ —the Frenet–Serret frame from Chapter 2 provides a natural choice. Collect these into the matrix $N(s) = [n_1 | \dots | n_{n-1}] \in \mathbb{R}^{n \times (n-1)}$. Then:

$$A_{\perp}(s) = N(s)^{\top} \left[J(s) - \frac{f^* \cdot J(s)^{\top} t(s)}{\|f^*\|^2} f^* t(s)^{\top} - \dot{N}(s) N(s)^{\top} \right] N(s), \quad (4.12)$$

where $f^* = f(\gamma^*, u^*)$ and $\dot{N}$ accounts for the rotation of the transverse frame along the orbit (the Frenet–Serret equations).

Example 4.2. Transverse Linearization of a Planar System

Consider the system $\dot{x}_1 = x_2$, $\dot{x}_2 = -x_1 + u$ tracking the circular orbit $\gamma^*(t) = (\cos t, -\sin t)$ with $u^* = 0$. The Jacobian is $J = \begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix}$, the tangent vector is $t = (-\sin t, -\cos t)^{\top}/1 = (-\sin t, -\cos t)^{\top}$, and the normal is $n = (-\cos t, \sin t)^{\top}$. Computing $A_{\perp} = n^{\top}[J - \text{tangential terms}]n$, we find:

$$A_{\perp}(t) = 0, \quad B_{\perp}(t) = n^{\top} \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \sin t. \quad (4.13)$$

The open-loop transverse dynamics are *neutrally stable* ($A_{\perp} = 0$). Feedback $\delta u = -k\xi$ gives $\dot{\xi} = -k\sin^2 t \xi$ (after averaging), which is stable for $k > 0$. The orbit is stabilizable.

4.5 Floquet Theory: Stability of Periodic Orbits

For periodic trajectories (limit cycles, walking gaits, repetitive manufacturing), the transverse linearization is a *periodic* linear system. Floquet theory provides the definitive stability analysis tool.

4.5.1 The State Transition Matrix

Definition 4.6. State Transition Matrix

For the linear time-varying system $\dot{\xi} = A_{\perp}(t)\xi$, the **state transition matrix** $\Phi(t, t_0)$ satisfies

$$\dot{\Phi}(t, t_0) = A_{\perp}(t)\Phi(t, t_0), \quad \Phi(t_0, t_0) = I. \quad (4.14)$$

The solution is $\xi(t) = \Phi(t, t_0)\xi(t_0)$.

4.5.2 The Monodromy Matrix

Definition 4.7. Monodromy Matrix

For a periodic orbit with period T , the **monodromy matrix** is the state transition matrix over one complete period:

$$\mathcal{M} = \Phi(T, 0). \quad (4.15)$$

The eigenvalues $\mu_1, \dots, \mu_{n-1}$ of $\mathcal{M}$ are the **Floquet multipliers** (also called **characteristic multipliers**).

Theorem 4.2 (Floquet Stability Criterion). *A periodic orbit is:*

- (i) **Orbitally stable** if all Floquet multipliers satisfy $|\mu_k| \leq 1$, with multipliers on the unit circle being semisimple.
- (ii) **Asymptotically orbitally stable** if all Floquet multipliers satisfy $|\mu_k| < 1$ (all strictly inside the unit circle).
- (iii) **Orbitally unstable** if any Floquet multiplier satisfies $|\mu_k| > 1$.

Remark 4.3. The Missing Multiplier

If we had computed the monodromy matrix for the *full* n -dimensional system (not the transverse reduction), there would be n Floquet multipliers, with one of them exactly equal to $+1$. This “trivial” multiplier corresponds to the tangential direction—perturbations along the orbit neither grow nor decay. The transverse reduction removes this trivial multiplier, leaving only the $n - 1$ multipliers that matter for orbital stability.

This is why the transverse reduction is essential: without it, the monodromy matrix *always* has an eigenvalue at $+1$, making the orbit appear only marginally stable even when it is asymptotically orbitally stable.

Example 4.3. Floquet Analysis of a Walking Gait

Consider a simplified compass-gait biped with 4-dimensional state space $(\theta_1, \theta_2, \dot{\theta}_1, \dot{\theta}_2)$. The walking gait is a periodic orbit with period T (one stride). The transverse linearization has dimension $n - 1 = 3$.

After numerically integrating $\Phi(T, 0)$ for the transverse dynamics, suppose we find the monodromy matrix has eigenvalues $\mu_1 = 0.7$, $\mu_2 = 0.4$, $\mu_3 = -0.3$. Since all $|\mu_k| < 1$, the gait is asymptotically orbitally stable.

The physical interpretation: $\mu_1 = 0.7$ means that after one stride, transverse deviations in the first mode are reduced to 70% of their initial value. After 10 strides, they are reduced to $0.7^{10} \approx 3\%$. The negative multiplier $\mu_3 = -0.3$ indicates an alternating (flip-flop) convergence pattern in the third mode.

For the golf swing (a non-periodic motion), we cannot use the monodromy matrix directly, but the state transition matrix $\Phi(T, 0)$ plays an analogous role—its singular values measure how much transverse deviations amplify or attenuate over the course of the motion.

4.6 Moving Poincaré Sections

A complementary tool for analyzing orbital stability is the **Poincaré section**—a codimension-1 surface that the orbit crosses transversally. While Floquet theory gives continuous-time information, Poincaré sections give *discrete-time* (strobed) information.

Definition 4.8. Poincaré Section

A **Poincaré section** for an orbit $\mathcal{O}$ is a smooth $(n-1)$ -dimensional surface $\mathcal{P} \subset \mathcal{X}$ such that:

- (i) $\mathcal{O}$ intersects $\mathcal{P}$ transversally (not tangentially).
- (ii) In a neighborhood of the intersection, every orbit near $\mathcal{O}$ also crosses $\mathcal{P}$.

The **Poincaré return map** $P : \mathcal{P} \rightarrow \mathcal{P}$ sends each point on $\mathcal{P}$ to the next intersection with $\mathcal{P}$ under the flow.

For periodic orbits, the orbit crosses $\mathcal{P}$ once per period, and the Poincaré return map has a fixed point at the intersection $\mathbf{x}^* = \mathcal{O} \cap \mathcal{P}$. Orbital stability of $\mathcal{O}$ is equivalent to stability of this fixed point under the return map.

Theorem 4.3 (Poincaré–Floquet Connection). *The Jacobian of the Poincaré return map at the fixed point equals the transverse monodromy matrix:*

$$DP(\mathbf{x}^*) = M_{\perp}. \quad (4.16)$$

The Floquet multipliers and the eigenvalues of the Poincaré map are identical.

4.6.1 Moving Poincaré Sections for Non-Periodic Trajectories

The golf swing is not periodic—it is a *finite-time* trajectory. The classical Poincaré section (which relies on periodicity) does not directly apply. However, we can generalize to **moving**

Poincaré sections: a continuous family of transverse hyperplanes, one at each point of the orbit.

Definition 4.9. Moving Poincaré Section

A **moving Poincaré section** along a trajectory $\gamma^*(s)$, $s \in [0, L]$, is the family of hyperplanes

$$\mathcal{P}(s) = \gamma^*(s) + \Sigma(s), \quad (4.17)$$

where $\Sigma(s)$ is the transverse hyperplane at s . The **section-to-section map** from $\mathcal{P}(s_1)$ to $\mathcal{P}(s_2)$ is defined by following the flow from s_1 to s_2 :

$$\mathbf{P}_{s_1 \rightarrow s_2} : \mathcal{P}(s_1) \rightarrow \mathcal{P}(s_2), \quad \xi(s_1) \mapsto \xi(s_2). \quad (4.18)$$

The linearized section-to-section map is the state transition matrix $\Phi_\perp(s_2, s_1)$ of the transverse dynamics.

For the golf swing, this gives us a powerful analysis tool:

Principle 4.2. Finite-Time Orbital Stability

For a finite-time trajectory $\gamma^* : [0, T] \rightarrow \mathcal{X}$, define the **transverse amplification ratio** from phase s_1 to s_2 :

$$\rho(s_1, s_2) = \|\Phi_\perp(s_2, s_1)\|_2 = \sigma_{\max}(\Phi_\perp(s_2, s_1)), \quad (4.19)$$

where $\sigma_{\max}$ is the largest singular value. If $\rho(s_1, s_2) < 1$, then transverse deviations shrink between s_1 and s_2 . If $\rho(s_1, s_2) > 1$, they grow.

For trajectory design, we require:

$$\rho(0, s_{\text{impact}}) \ll 1 \quad (\text{deviations shrink toward impact}). \quad (4.20)$$

This is a precise formulation of the funnel concept from Chapter 1: the tube narrows because the transverse dynamics are contracting.

Example 4.4. Amplification Profile of the Golf Swing

Consider the 8-dimensional state space of a 4-DOF golf swing model. The transverse dynamics have dimension 7. The transverse state transition matrix $\Phi_\perp(s, 0)$ maps initial deviations to deviations at phase s .

Numerical computation reveals the amplification ratio $\rho(0, s)$ as a function of s :

- $s \in [0, 0.3]$ (backswing): ρ is approximately 1—deviations neither grow nor shrink. The motion is kinematically slow and nearly linear.
- $s \in [0.3, 0.5]$ (transition): ρ *increases* past 1. This is the unstable phase—the high curvature and speed change amplify deviations. Without feedback, errors grow here.
- $s \in [0.5, 0.7]$ (late downswing): ρ *decreases*. In some simplified release models,

the transverse dynamics contract over this phase interval. Interpreted carefully, the wrist-release geometry narrows the admissible error tube for that model; it is not proof that every real swing is open-loop self-stabilizing.

- $s = 0.75$ (impact): $\rho(0, 0.75) < 1$. The net effect over the entire swing is transverse contraction—deviations are smaller at impact than at address.

This profile shows one way skilled golfers could remain accurate despite the extreme nonlinearity of the swing: a well-posed model may exhibit passive transverse contraction near impact, and feedback can exploit that contraction. The result is a model-level funnel, not a blanket claim that errors vanish without sensing or control.

4.7 Controller Design via Transverse Stabilization

We now use the transverse linearization to design tracking controllers. The approach is systematic: stabilize the transverse linear system (4.9) using standard LTV techniques, then map the result back to the nonlinear system.

4.7.1 Time-Varying LQR for Transverse Stabilization

Principle 4.3. Transverse LQR

The trajectory tracking problem reduces to solving the *differential Riccati equation* for the transverse dynamics:

$$-\dot{\mathbf{S}}(t) = \mathbf{A}_\perp^\top \mathbf{S} + \mathbf{S} \mathbf{A}_\perp - \mathbf{S} \mathbf{B}_\perp \mathbf{R}^{-1} \mathbf{B}_\perp^\top \mathbf{S} + \mathbf{Q}(t), \quad (4.21)$$

with terminal condition $\mathbf{S}(T) = \mathbf{S}_T$. The optimal transverse feedback gain is

$$\mathbf{K}_\perp(t) = \mathbf{R}^{-1} \mathbf{B}_\perp(t)^\top \mathbf{S}(t). \quad (4.22)$$

The full control law is

$$\mathbf{u}(t) = \underbrace{\mathbf{u}^*(t)}_{\text{feedforward}} + \underbrace{\mathbf{N}(s^*) \mathbf{K}_\perp(t) \mathbf{N}(s^*)^\top (\mathbf{x} - \boldsymbol{\gamma}^*(s^*))}_{\text{transverse feedback}}, \quad (4.23)$$

where $\mathbf{N}(s^*)$ maps between the transverse coordinates $\boldsymbol{\xi}$ and the full state coordinates.

The weighting matrices $\mathbf{Q}(t)$ and $\mathbf{R}$ have clear physical interpretations:

- $\mathbf{Q}(t)$ penalizes transverse deviation. Making $\mathbf{Q}$ large near impact tightens the funnel where precision matters.
- $\mathbf{R}$ penalizes control effort. Making $\mathbf{R}$ small allows aggressive correction but may demand unrealistic actuator authority.
- The *phase-dependent* nature of $\mathbf{Q}(t)$ is essential: we want tight tracking near impact and loose tracking during the backswing. This is the time-varying tube

from Chapter 1, now realized through the Riccati equation.

4.7.2 Phase-Varying Gains and the Funnel Connection

The terminal-value Riccati equation (4.21) naturally produces gains that tighten as the terminal time approaches. This is exactly the funnel behavior we designed geometrically in Chapter 1:

Theorem 4.4 (Riccati Funnel). *Let $\mathbf{S}(t)$ be the solution of (4.21) with terminal penalty $\mathbf{S}_T \succ 0$. The ellipsoidal set*

$$\mathcal{E}(t) = \{\boldsymbol{\xi} \in \Sigma(s) : \boldsymbol{\xi}^\top \mathbf{S}(t) \boldsymbol{\xi} \leq c\} \quad (4.24)$$

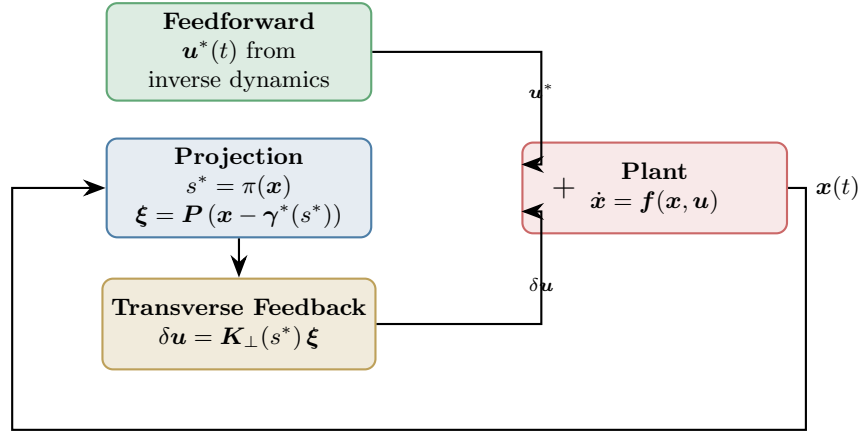
defines a time-varying tube whose width (in the direction of eigenvector $\mathbf{v}_k$ of $\mathbf{S}$) is proportional to $1/\sqrt{\lambda_k(t)}$, where $\lambda_k(t)$ is the corresponding eigenvalue of $\mathbf{S}(t)$.

As $t \rightarrow T$ and $\mathbf{S}(t) \rightarrow \mathbf{S}_T$, the tube tightens. The closed-loop transverse dynamics ensure that any trajectory starting inside the tube at $t = 0$ remains inside for all $t \in [0, T]$, provided the control authority is sufficient.

This is the formal connection between the geometric funnel of Chapter 1 and the algebraic Riccati equation: **the funnel is the level set of the Riccati cost-to-go.**

4.7.3 The Complete Tracking Controller Architecture

Assembling all the pieces, the trajectory tracking controller has three layers:



Layer 1. Feedforward $u^*(t)$: computed offline from inverse dynamics, provides $\sim 90\%$ of the total control effort.

Layer 2. Projection: measures the current state $\mathbf{x}(t)$, projects onto the orbit to find s^* , and computes the transverse deviation $\boldsymbol{\xi}$.

Layer 3. Transverse feedback $\delta \mathbf{u} = \mathbf{K}_\perp(s^*) \boldsymbol{\xi}$: stabilizes the transverse dynamics using phase-varying gains from the Riccati equation.

Remark 4.4. Why Not Just Use Standard LQR?

One could linearize the *full* dynamics about $\gamma^*(t)$ and apply standard time-varying LQR to the n -dimensional error $e(t) = x(t) - \gamma^*(t)$. This works, but:

- (a) It “wastes” control effort trying to correct tangential deviations (timing errors), which are benign.
- (b) It fights the inherent marginal stability in the tangential direction, leading to unnecessarily high gains.
- (c) It does not produce the correct funnel shape—it penalizes all errors equally, including timing errors.

The transverse approach avoids all three problems. It is geometrically correct, computationally smaller (dimension $n - 1$ vs. n), and physically meaningful.

4.8 Robustness of Orbital Stability

A practical controller must be robust to model uncertainty and external disturbances. The transverse framework provides natural robustness margins.

Definition 4.10. Transverse Gain and Phase Margins

For the closed-loop transverse system $\dot{\xi} = [A_{\perp} - B_{\perp}K_{\perp}]\xi$, the **transverse gain margin** and **transverse phase margin** are the gain and phase margins of the transverse loop transfer function. These are the trajectory-tracking analogues of the classical setpoint stability margins.

Proposition 4.5 (Robustness of Transverse LQR). *If the transverse LQR is designed with $R = rI$, then the closed-loop transverse system has guaranteed gain margin $[1/2, \infty)$ and phase margin $\pm 60^\circ$, inheriting the classical LQR robustness guarantees in each transverse channel.*

For the golf swing, robustness translates directly to *repeatability*: a swing with high transverse robustness margins produces consistent results even when the golfer’s motor commands are noisy.

4.9 Summary

Concept	Significance
Orbital stability	The correct stability notion for moving systems: only cross-orbit deviations matter
Transverse linearization	Reduces orbital stability to $(n-1)$ -dim LTV stability; removes the tangential “nuisance” direction
Floquet multipliers	Eigenvalues of the transverse monodromy matrix; determine stability of periodic orbits
Moving Poincaré sections	Extend Poincaré analysis to non-periodic trajectories; connect to the state transition matrix
Amplification ratio ρ	Maximum singular value of $\Phi_{\perp}$; quantifies transverse error growth or contraction
Transverse LQR	Phase-varying feedback from the Riccati equation; the optimal funnel controller
Riccati funnel	The LQR cost-to-go defines the funnel boundary; connects geometric and algebraic views

This chapter has provided the mathematical backbone for everything that follows. The transverse linearization converts the nonlinear trajectory-tracking problem into a linear, lower-dimensional problem that inherits all the powerful tools of LTV control theory. The Floquet/Poincaré framework provides both analytical insight and numerical algorithms for assessing stability.

The key insight for the golf swing is that orbital stability—not Lyapunov stability—is the correct framework. The most defensible golf interpretation is that some models exhibit transverse contraction near impact, and that transverse LQR can tighten the admissible tube with phase-varying gains matched to the modeled motion.

In Chapter 5, we will see how underactuation constrains the transverse dynamics, creating geometric structure that can be *exploited* rather than merely tolerated.

4.10 Exercises

- 4.1. (Conceptual.)** Explain why a pendulum swinging on its limit cycle (pumped by a periodic torque) is orbitally stable but *not* Lyapunov stable. Sketch the phase portrait and identify the tangential and transverse directions.
- 4.2. (Computation.)** For the Van der Pol oscillator $\ddot{x} - \mu(1 - x^2)\dot{x} + x = 0$ with $\mu = 1$:
 - (a) Numerically find the limit cycle by simulating from $(x, \dot{x}) = (3, 0)$ until transient effects decay.
 - (b) Compute the transverse linearization along the limit cycle.
 - (c) Compute the monodromy matrix and Floquet multipliers numerically.

- (d) Verify that one multiplier of the full (2D) system is exactly 1.
- 4.3. (Transverse linearization.)** For the system $\dot{x}_1 = x_2 + u$, $\dot{x}_2 = -x_1^3$, consider the trajectory $\gamma^*(t) = (\cos t, -\sin t)$ with appropriate u^* .
- (a) Find $u^*(t)$ such that γ^* is a solution.
 - (b) Compute the full Jacobian $J(t)$ along γ^* .
 - (c) Compute the transverse Jacobian $A_\perp(t)$.
 - (d) Is the orbit stable open-loop? Design stabilizing transverse feedback.
- 4.4. (Numerical project: Transverse LQR.)** For the double pendulum from Exercise 3.4:
- (a) Compute a reference swing trajectory using direct collocation (or load from a data file).
 - (b) Compute the transverse linearization along the trajectory.
 - (c) Solve the differential Riccati equation with $Q(t)$ that increases 10-fold near the terminal phase (“impact”).
 - (d) Simulate the closed-loop system with random initial perturbations and plot the transverse deviation $\|\xi(t)\|$ over time.
 - (e) Compute the amplification ratio profile $\rho(0, s)$ and identify the “natural funnel” phase.
- 4.5. (Design.)** A walking robot with period-1 gait has Floquet multipliers $\mu_1 = 0.9$, $\mu_2 = 1.1$, $\mu_3 = 0.5$.
- (a) Is the gait orbitally stable? Which mode is problematic?
 - (b) Design a transverse feedback controller that places the unstable multiplier μ_2 at 0.6. What gain is required?
 - (c) After stabilization, what is the worst-case convergence rate per stride?
 - (d) How many strides does it take to reduce a 10% transverse deviation to 1%?
- 4.6. (Philosophical.)** Compare the transverse LQR approach (this chapter) with the standard trajectory-tracking LQR (linearize about $\gamma^*(t)$, regulate $e = x - \gamma^*(t)$ to zero).
- (a) Under what conditions do the two approaches give identical feedback gains? (Hint: consider straight-line trajectories.)
 - (b) Construct a numerical example where standard LQR uses significantly more control effort than transverse LQR.
 - (c) Discuss which approach is preferable for the golf swing and why.

*The pendulum does not care if you are watching at noon or midnight.
Its orbit knows no clock.*

*Stability is not arrival. It is the promise
that the motion, once begun, will keep its shape—
even when the world pushes back.*

Chapter 5

Underactuation and Passive Dynamics

Draft Notice. This chapter is under active development and contains only a structural outline with preliminary material. Full exposition, worked examples, proofs, and exercises will be added in future editions.

Do not force the motion. Let the physics do the work.

In Plain Language 5.1. Working with Physics, Not Against It

Imagine trying to push a child on a playground swing. If you hold their back and try to physically muscle them through the exact shape of the arc at every single moment, you'll be exhausted and the ride will be jerky. Classical robotic control often tries to do just this: put a powerful motor on every joint and force the robot along a path, overpowering momentum and gravity. But "underactuated" systems, like a person walking or a golfer swinging a club, don't have enough motors (or muscles) to micromanage every joint at all times. In a golf swing, the wrists actually swing freely for much of the downswing. The club whips through the ball not because the wrists flexed forcefully, but because the speed and sudden braking of the arms created a physical whipping effect. This chapter looks at how to use these "passive" physics to our advantage, rather than seeing them as a lack of control.

5.1 The Loss of Control Authority

In classical robotics, the goal is often full actuation: one motor for every degree of freedom. If a robot has six joints, it has six motors. This fully actuated paradigm allows the system to follow essentially any smooth trajectory in the configuration space, subject only to actuator saturation limits. The control problem is algebraically simple because the mapping from torques to joint accelerations is invertible.

But nature does not build fully actuated systems, and neither do athletes.

When a gymnast swings on a high bar, their center of mass rotates around the bar, but they have no motor at the bar—that “joint” is a passive pin constraint. When a human walks, the foot acting as the pivot is unactuated during the single-support phase; we cannot create torque from a point contact. In a simplified golf model, the wrist can be treated as a low-torque or passive hinge over part of the release. The club then undergoes rapid angular acceleration shaped by the coupling terms $M_{21}\ddot{\theta}_1 + C_2$ in the unactuated dynamics, not by a direct wrist motor in the model [20]. The assumption is useful, but it is not a claim that every real wrist torque is zero.

Definition 5.1. Underactuated System

Consider a mechanical system with n degrees of freedom (DOF) and configuration coordinates $\mathbf{q} \in \mathcal{Q} \subseteq \mathbb{R}^n$. The equations of motion take the standard manipulator form:

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}) = \mathbf{B}(\mathbf{q})\mathbf{u} \quad (5.1)$$

where $\mathbf{M}$ is the inertia matrix, $\mathbf{C}$ contains Coriolis and centrifugal terms, $\mathbf{g}$ is the gravity vector, $\mathbf{u} \in \mathbb{R}^m$ is the vector of control inputs, and $\mathbf{B}(\mathbf{q}) \in \mathbb{R}^{n \times m}$ maps inputs into generalized forces.

The system is **underactuated** if $m < n$, meaning there are fewer actuators than degrees of freedom. In this case, the matrix $\mathbf{B}$ is not full rank (it maps a lower-dimensional control space into the n -dimensional configuration space).

If we lack the authority to command arbitrary accelerations, the classical setpoint perspective sees this as a fundamental limitation. A setpoint controller wants to cancel the nonlinearities, feedback-linearize the system, and dictate the exact motion. Underactuation prevents this brute-force cancellation.

However, from the trajectory-first perspective developed in Chapter 1, underactuation is not merely a restriction—it is a geometric feature. The *passive dynamics* (the unactuated portions of the dynamics) define a manifold of feasible trajectories. The design task is to work with that structure rather than cancel it outright.

5.2 The Double Pendulum: A Window into the Golf Swing

To understand how passive dynamics can be exploited, we study the simplest model that captures the physics of the golf downswing: the planar underactuated double pendulum.

Let the first link (the arms) have angle θ_1 , length l_1 , mass m_1 , and inertia I_1 . Let the second link (the club) have angle θ_2 , length l_2 , mass m_2 , and inertia I_2 . The configuration is $\mathbf{q} = (\theta_1, \theta_2)^\top$.

Crucially, in this reduced model we assume there is a motor at the shoulder (controlling torque $u_1 = \tau_{\text{shoulder}}$) but *no motor* at the wrist ($\tau_{\text{wrist}} = 0$).

The input matrix is therefore:

$$\mathbf{B} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad \mathbf{u} = [u_1]. \quad (5.2)$$

The dynamics expand into two coupled equations:

$$M_{11}\ddot{\theta}_1 + M_{12}\ddot{\theta}_2 + C_1 + g_1 = u_1, \quad (5.3)$$

$$M_{21}\ddot{\theta}_1 + M_{22}\ddot{\theta}_2 + C_2 + g_2 = 0. \quad (5.4)$$

Equation (5.4) is the **unactuated dynamics** or **passive constraint**. It dictates the motion of the club. Notice that we cannot directly command $\ddot{\theta}_2$. However, we *can* influence it through the coupling terms:

- (i) **Inertial coupling** ($M_{21}\ddot{\theta}_1$): Accelerating or decelerating the arms immediately exerts a force on the club.
- (ii) **Velocity coupling** (C_2): This term contains the centrifugal forces. As the arms swing rapidly, centrifugal force naturally pulls the club outward.

Key Idea 5.1. The Physics of the Wrist Release

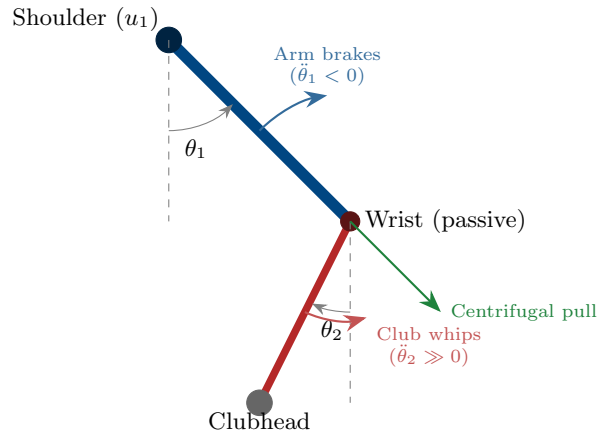
In a golf swing, a simplified underactuated model can generate large clubhead speed at impact by exploiting the C_2 and M_{21} terms. Early in the downswing, the golfer maintains an acute angle between the arms and the club ($\theta_2 - \theta_1 \approx 90^\circ$). The arms accelerate ($\ddot{\theta}_1 > 0$).

As the hands approach the bottom of the swing arc, the arms decelerate ($\ddot{\theta}_1 < 0$). This negative acceleration, transferred through the M_{21} coupling, can accelerate the club forward. Simultaneously, the large arm velocity ($\dot{\theta}_1$) makes the centrifugal contribution in C_2 large enough to push the club toward alignment with the arms ($\theta_2 \rightarrow \theta_1$) in the model. This supports a passive-release hypothesis for the simplified system; it does not show that measured wrist torque is identically zero or that real golfers never add muscular wrist action.

Let's look at the unactuated row algebraically to see this whipping action:

$$\ddot{\theta}_2 = -M_{22}^{-1}(M_{21}\ddot{\theta}_1 + C_2(\mathbf{q}, \dot{\mathbf{q}}) + g_2(\mathbf{q})). \quad (5.5)$$

Since M_{22} is scalar and positive, a large negative $\ddot{\theta}_1$ (braking the arms) can produce a large positive $\ddot{\theta}_2$ (accelerating the club). This is one mechanism by which the model can accelerate the club within the moving-target control problem. Whether it is efficient or dominant in measured swings depends on grip mechanics, shaft compliance, actuator limits, and timing.



5.3 The Annihilator and the Null Space

The double pendulum example shows that the passive dynamics constrain the allowable states and accelerations. We generalize this using the mathematics of the **null space**.

Recall the dynamics:

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + g(q) = B(q)u. \quad (5.6)$$

Definition 5.2. Left Annihilator

We define a full-rank matrix $B^\perp(q) \in \mathbb{R}^{(n-m) \times n}$, called the **left annihilator** of B , such that:

$$B^\perp(q)B(q) = \mathbf{0} \quad \text{for all } q. \quad (5.7)$$

The rows of $B^\perp$ span the left null space of B .

In our double pendulum where $B = [1, 0]^\top$, the annihilator is trivially $B^\perp = [0, 1]$.

By multiplying the equations of motion by the annihilator $B^\perp$, we eliminate the control input u , revealing the intrinsic constraints on the system's acceleration:

$$B^\perp M \ddot{q} + B^\perp (C \dot{q} + g) = \mathbf{0}. \quad (5.8)$$

Principle 5.1. The Theorem of Feasible Trajectories

A smooth curve $\gamma^*(t)$ is a dynamically feasible trajectory for the underactuated system if and only if it satisfies the unactuated dynamics:

$$B^\perp(\gamma^*) \left[M(\gamma^*) \ddot{\gamma}^* + C(\gamma^*, \dot{\gamma}^*) \dot{\gamma}^* + g(\gamma^*) \right] = \mathbf{0} \quad (5.9)$$

for all $t \in [0, T]$. If a curve satisfies this condition, one can always find the necessary control input $u^*(t)$ by projecting the dynamics onto the range of B .

For control design, this means we cannot arbitrarily select a path in $\mathcal{X}$ and expect a controller to track it. The reference trajectory *must* live in the restricted subspace defined by (5.8). The trajectory optimization algorithms in Chapter 6 will explicitly enforce this constraint.

5.4 Partial Feedback Linearization

While we cannot feedback-linearize the entire underactuated system, we can feedback-linearize the *actuated* degrees of freedom. This technique is called **partial feedback linearization** or **collocated feedback linearization**.

By cleverly substituting coordinates, we can partition the system into a linear, fully actuated subsystem (say, the golfer's arms) and a nonlinear, passive subsystem (the club) that is driven by the state of the first.

Partition the coordinates as $q = (q_a, q_u)$, representing actuated and unactuated DOFs. We can rewrite the dynamics and choose u to exactly cancel the nonlinearities of the actuated part:

$$\ddot{q}_a = v_a \quad (5.10)$$

where $\mathbf{v}_a$ is our new synthetic control input. The unactuated dynamics become:

$$\mathbf{M}_{uu}(\mathbf{q})\ddot{\mathbf{q}}_u + \mathbf{M}_{ua}(\mathbf{q})\mathbf{v}_a + \text{nonlinear terms} = 0. \quad (5.11)$$

If we force the actuated coordinates to track a reference trajectory (e.g., using a high-gain PD controller, $\mathbf{v}_a = \ddot{\mathbf{q}}_a^* - K_d\dot{\tilde{\mathbf{q}}}_a - K_p\tilde{\mathbf{q}}_a$), the unactuated coordinates are left to evolve according to (5.11). The resulting behavior depends on whether this forced subsystem remains bounded or convergent for the chosen reference, parameters, and feedback gains. This leads us to the concept of the *zero dynamics*.

5.5 Accessibility, Chow's Theorem, and the Orbit Theorem

For underactuated systems with fewer inputs than degrees of freedom, a fundamental question arises: which configurations are reachable? The answer comes from **Chow's Theorem**, the centerpiece of geometric accessibility theory developed rigorously in Agrachev and Sachkov [1].

Theorem 5.1 (Chow's Theorem / Orbit Theorem). *For a control system $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}) + \sum_{i=1}^m u_i(\mathbf{x})\mathbf{g}_i(\mathbf{x})$, a point $\mathbf{x}_f$ is accessible from $\mathbf{x}_0$ if and only if $\mathbf{x}_f$ lies in the orbit generated by the Lie algebra $\mathfrak{g} = \text{Lie}\{\mathbf{f}, \mathbf{g}_1, \dots, \mathbf{g}_m\}$ and their Lie brackets evaluated at $\mathbf{x}_0$.*

For the double pendulum of the golf swing, even though we cannot directly command the unactuated wrist ($\ddot{\theta}_2 = 0$ alone), the Lie bracket [shoulder control, inertial coupling] indicates directions of club motion that become locally accessible through coupling. Higher-order Lie brackets can enlarge the accessible distribution. The orbit dimension bounds the directions available to the model, but full rank of the Lie algebra supports only a local accessibility claim under the stated smoothness, torque, and contact assumptions; it does not prove global controllability over every configuration or finite-time downswing task.

Sub-Riemannian Geometry: When only a subset of directions are controllable (non-involutive distribution), the system can be studied with sub-Riemannian tools. Optimal trajectories in underactuated systems need not look like simple Euclidean minimum-energy curves, and abnormal extremals or singular arcs can appear in some formulations. A simplified golf-release model may exhibit such structures, but that is a statement about the chosen coordinates, costs, and bounds rather than evidence that real swings require instantaneous switching.

5.6 Zero Dynamics and the Golf Swing Funnel

If we perfectly stabilize the actuated tracking errors to zero ($\mathbf{q}_a \equiv \mathbf{q}_a^*$), the remaining dynamics governing $\mathbf{q}_u$ are called the **zero dynamics**.

For the golf swing model, the zero dynamics describe how the club moves when the arm trajectory is prescribed and the wrist DOF remains unactuated in the reduced system. The practical question is not whether “the golf swing” is globally stable, but whether a specified phase interval of a specified model has locally attracting transverse behavior.

Revisiting the concept of Transverse Linearization (Chapter 4), if we linearize the unactuated dynamics around the trajectory $\boldsymbol{\gamma}^*(t)$, we analyze the transverse divergence of the club from its optimal path.

We obtain a phase-dependent picture rather than a universal law:

- During the early downswing, the club's zero dynamics are *unstable*. If the club gets slightly out of position, the centrifugal force tends to pull it further off the optimal plane.
- During the late downswing (the release phase), the club's zero dynamics may become strongly attracting in the simplified model. Large club acceleration can act like a restoring term, but the degree of alignment depends on the selected parameters, grip model, and disturbance assumptions.

This is the model analogue of the time-varying tube from Chapter 1. In favorable parameter regimes, passive coupling can make the admissible-error set wide early in the motion and narrower near impact. The result is not a literal singularity and not a coaching rule that wrist torque is unnecessary; it is a statement about a specified underactuated model whose phase, grip, shaft, and actuator assumptions must be checked.

Remark 5.1. Why Passive Release Can Feel Effortless

The common coaching phrase “let the club release” is compatible with an underactuated model, but it should not be read as a measured neuromuscular fact. A skilled swing may use the arms and torso to set boundary conditions for passive release while still applying grip and wrist torques. Whether that strategy improves speed or consistency must be tested against measured kinematics, force data, and the assumed wrist model.

5.7 Abnormal Extremals and Singular Arcs

A notable feature of underactuated optimal control is the possibility of **abnormal extremals**—optimal trajectories for which the usual regularity condition in Pontryagin's Maximum Principle degenerates. In such cases, the multiplier on the running cost can vanish, and the control is not recovered from the regular Hamiltonian maximization alone. Agrachev and Sachkov discuss how singular controls arise in this setting [1]. These objects can occur in underactuated problems, but whether they appear depends on the formulated cost, bounds, coordinates, and boundary conditions.

In a simplified swing model, the rapid release phase may correspond to an abnormal or near-singular segment. That is useful to keep in mind when solving the optimization numerically, because standard smooth solvers can miss switching, saturation, or active-set changes. It is not, by itself, evidence that measured golfers execute a literal bang-bang or singular control law.

5.8 Summary

Concept	Significance
Underactuation	Fewer actuators (m) than coordinates (n). Cannot arbitrarily command accelerations in configuration space.
Passive Dynamics	The natural, unforced motion of the unactuated DOFs.
Coupling	Inertial (M_{21}) and velocity (C_2) terms transmit energy from actuated to unactuated DOFs.
Left Annihilator	A matrix $\mathbf{B}^\perp$ that strips away the control input to reveal the intrinsic geometric constraints of the motion.
Partial Feedback Lin.	Cancelling nonlinearities only on the actuated joints to simplify control.
Zero Dynamics	The residual dynamics of the unactuated subsystem when the actuated subsystem perfectly tracks its reference.

Underactuation shifts the modeling task from forcing every coordinate independently to shaping the interaction between actuators, gravity, inertia, and constraints. In Chapter 6, numerical trajectory optimization is used to search for such motions by solving optimal-control problems that explicitly respect the $\mathbf{B}^\perp$ constraint and the stated actuator limits.

5.9 Exercises

- 5.1. (Annihilator construction.)** A cart-pole system has state $\mathbf{q} = (x, \theta)^\top$, where x is the cart position and θ is the pole angle. The motor applies horizontal force to the cart.
- What is the input matrix $\mathbf{B}$?
 - Construct a left annihilator $\mathbf{B}^\perp$.
 - Derive the unactuated dynamics constraint. What physical conservation law does this represent when gravity is ignored?
- 5.2. (Passive pumping.)** In a playground swing, the person swinging cannot exert an external torque on the swing setup (there is no motor at the top pivot). By what mechanism (which dynamics term) does the swinger add energy to the system by leaning back and forth? Relate this to the golf wrist release.
- 5.3. (Zero Dynamics Stability.)** Consider a generic system partitioned into $\ddot{q}_a = v_a$ and $\ddot{q}_u + cq_u = q_a$. Let the reference trajectory be $q_a^*(t) = \sin(t)$.
- Assume we enforce $q_a = q_a^*$ perfectly. Write the zero dynamics for q_u .
 - Under what conditions on the parameter c are the zero dynamics stable? Is it possible to have a trajectory that passes through both stable and unstable regimes of zero dynamics?

*Control is not dominion over nature;
it is an intelligent negotiation with it.*

Chapter 6

Trajectory Optimization

Draft Notice. This chapter is under active development and contains only a structural outline with preliminary material. Full exposition, worked examples, proofs, and exercises will be added in future editions.

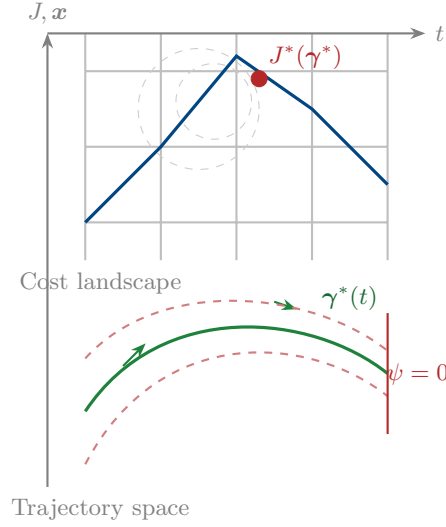
We do not merely seek a path that obeys the laws of physics.
We seek the path that bends them to our advantage.

In Plain Language 6.1. Discovering the Best Motion

How does a model propose a motion when the final task is specified but the path is not? Up until now, we have talked about how to follow a path; this chapter asks where a candidate path comes from. A trajectory optimizer can search over many admissible motions for a declared objective, such as maximizing clubhead speed at a terminal surface, while enforcing the equations of motion and actuator limits. The output is not a perfect human swing. It is a model-conditioned candidate trajectory whose usefulness depends on the cost function, constraints, parameter values, and validation data.

6.1 The Moving Target Problem

In Chapter 1, we fundamentally reframed our control objective from "reaching a destination" to "tracking a trajectory." Up to this point, we have assumed that this optimal reference trajectory $\gamma^*(t)$ is given to us by an oracle. This chapter answers the obvious next question: *Where does this reference come from?*



For systems whose purpose is to move—like our running example of the golf swing—the optimal trajectory is not arbitrary. We are not just trying to go from point A to point B while minimizing energy. We are trying to fulfill what we term the **Moving Target Objective**: optimizing the kinematics (velocity, pose, acceleration) of a system precisely at the moment it intersects a target manifold (e.g., the golf ball).

Crucially, *the exact clock time that impact occurs does not matter*. A golfer has the freedom to start their backswing slowly and pause at the top. The optimization is entirely concerned with the *shape* of the motion and its terminal kinematics. Time is a resource, not a constraint.

Definition 6.1. The Moving Target Optimal Control Problem (OCP)

Find the state trajectory $\mathbf{x}(t)$, control input $\mathbf{u}(t)$, and terminal time t_f that minimize the cost functional:

$$J = \phi(\mathbf{x}(t_f)) + \int_0^{t_f} L(\mathbf{x}(t), \mathbf{u}(t)) dt \quad (6.1)$$

subject to the dynamics and constraints:

$$\dot{\mathbf{x}}(t) = \mathbf{f}(\mathbf{x}(t), \mathbf{u}(t)) \quad (\text{Dynamics}) \quad (6.2)$$

$$\mathbf{h}(\mathbf{x}(t), \mathbf{u}(t)) \leq \mathbf{0} \quad (\text{Path constraints}) \quad (6.3)$$

$$\psi(\mathbf{x}(t_f)) = \mathbf{0} \quad (\text{Terminal manifold/Moving goal}) \quad (6.4)$$

$$\mathbf{x}(0) = \mathbf{x}_0 \quad (\text{Initial state}) \quad (6.5)$$

In the golf downswing, the terminal cost $\phi(\mathbf{x}(t_f))$ might be negative clubhead speed (to maximize speed), the integral cost L might penalize excessive joint torques to ensure repeatability, and the terminal manifold constraint $\psi(\mathbf{x}(t_f)) = \mathbf{0}$ forces the clubhead to be geometrically located at the ball with a squared face. Notice that t_f is a *free parameter* in the optimization.

6.2 Transcribing the Continuous Problem

The OCP described above occurs in infinite-dimensional function space. To solve it on a computer, we must transcribe it into a finite-dimensional Nonlinear Programming problem (NLP):

$$\begin{aligned} \min_{\mathbf{w}} \quad & f(\mathbf{w}) \\ \text{subject to} \quad & \mathbf{c}(\mathbf{w}) \leq \mathbf{0}, \end{aligned}$$

where $\mathbf{w}$ is a large vector of decision variables. How we discretize the integrals and differential equations defines the transcription method. We will focus on two modern approaches: **Direct Shooting** and **Direct Collocation**.

6.2.1 Direct Multi-Shooting

In direct multi-shooting, we break the time interval $[0, t_f]$ into N segments. The decision variables are the control parameters $\mathbf{u}_k$ on each interval and the state values $\mathbf{x}_k$ at the beginning of each interval.

The dynamic constraint $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u})$ is enforced by numerical integration (e.g., Runge-Kutta). We require that the integrated state at the end of interval k exactly matches the assumed starting state of interval $k + 1$:

$$\mathbf{x}_{k+1} - \Phi(\mathbf{x}_k, \mathbf{u}_k, \Delta t) = \mathbf{0} \quad (\text{Defect constraint}) \quad (6.6)$$

where Φ represents the numerical integrator step. Multi-shooting is excellent for highly nonlinear dynamics because it keeps the integration intervals short, preventing the massive nonlinear sensitivities that plague single-shooting methods.

6.2.2 Direct Collocation

Direct Collocation [2] takes this a step further. We represent both the state trajectory and the control signal as piecewise polynomials (e.g., cubic splines for state, linear splines for control). The decision variables are the coefficients of these polynomials, typically located at discrete **collocation points**.

Instead of using a numerical integrator, we enforce the differential equation algebraically at the collocation points:

$$\dot{\tilde{\mathbf{x}}}(t_c) - \mathbf{f}(\tilde{\mathbf{x}}(t_c), \tilde{\mathbf{u}}(t_c)) = \mathbf{0} \quad (\text{Collocation constraint}) \quad (6.7)$$

where $\tilde{\mathbf{x}}$ and $\tilde{\mathbf{u}}$ are the polynomial approximations.

Remark 6.1. Pontryagin Maximum Principle and Optimal Control

While direct collocation provides a practical numerical algorithm, the theoretical foundation for trajectory optimization comes from the **Pontryagin Maximum Principle** [1]. This principle states that along an optimal trajectory, a Hamiltonian function $\mathcal{H}(\mathbf{x}, \mathbf{u}, \boldsymbol{\lambda}) = L(\mathbf{x}, \mathbf{u}) + \boldsymbol{\lambda}^\top \mathbf{f}(\mathbf{x}, \mathbf{u})$ must be maximized with respect to $\mathbf{u}$ at each instant, where the costate $\boldsymbol{\lambda}(t)$ evolves backward according to adjoint dynamics. For underactuated systems, **bang-bang** structures can emerge when the

Hamiltonian is linear in $\mathbf{u}$, and **singular arcs** can occur when higher-order conditions are needed to determine $\mathbf{u}$. A simplified golf-release model may show a control structure resembling a singular arc, but that is a hypothesis about the chosen model and bounds, not a direct prediction of measured wrist muscle activation.

where $\tilde{\mathbf{x}}$ and $\tilde{\mathbf{u}}$ are the polynomial approximations.

Key Idea 6.1. Fidelity vs. Flexibility

Direct collocation results in a massive, highly sparse NLP. Because the solver has access to the state at every grid point simultaneously, it can easily manipulate the physical trajectory to satisfy difficult path constraints without waiting to "simulate" the result. Collocation is widely considered the state-of-the-art for generating complex dynamic motions like walking, jumping, and swinging.

6.2.3 Implementation Sketch: Moving-Target Direct Collocation

For readers who want a concrete recipe, the moving-target OCP can be translated into a collocation program with only a few repeating ingredients: mesh nodes, defect constraints, terminal manifold constraints, and a continuation strategy for difficult initial guesses.

6.3 Optimizing Underactuated Geometries

Recall from Chapter 5 that the underactuated dynamics enforce a strict algebraic constraint on the system:

$$\mathbf{B}^\perp(\mathbf{x}) \left[\mathbf{M}(\mathbf{x})\ddot{\mathbf{x}} + \mathbf{C}(\mathbf{x}, \dot{\mathbf{x}})\dot{\mathbf{x}} + \mathbf{g}(\mathbf{x}) \right] = \mathbf{0}. \quad (6.8)$$

When optimizing for a golf swing, direct collocation handles this underactuation organically. We do not need to explicitly partition the variables into actuated and unactuated sets for the solver. The collocation constraint $\dot{\mathbf{x}} - \mathbf{f} = \mathbf{0}$ already implicitly contains the passive dynamic constraints.

Example 6.1. The Golf Swing OCP

Consider our planar double pendulum model from Chapter 5. The solver is given the task to maximize $\dot{\theta}_2(t_f)$ (club speed).

Initial Guess: If we seed the NLP solver with a naive guess (e.g., constant shoulder torque), the club speed will be pitifully low.

The Optimized Result: The solver discovers the optimal physical orchestration entirely on its own. It will apply a massive positive u_1 to accelerate the arms, and then immediately reverse u_1 to negative maximum just prior to t_f . The optimizer mathematically "discovers" the necessary inertial coupling (the parametric excitation of $M_{21}\ddot{\theta}_1$) to whip the unactuated club link through the terminal manifold constraint. The passive dynamics are fully exploited to satisfy the objective.

Require: Initial state $\mathbf{x}_0$, dynamics $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u})$, running cost L , terminal manifold $\psi(\mathbf{x}(t_f)) = \mathbf{0}$, mesh size N .

Ensure: A feasible trajectory $\gamma^*(t)$ and control schedule $\mathbf{u}^*(t)$.

- 1: Choose a normalized mesh $\tau_k \in [0, 1]$ and treat t_f as a decision variable.
- 2: Parameterize $\tilde{\mathbf{x}}(\tau)$ and $\tilde{\mathbf{u}}(\tau)$ by spline or polynomial coefficients.
- 3: Initialize the decision vector with a dynamically plausible warm start.
- 4: **for** each mesh interval $k = 0, 1, \dots, N - 1$ **do**
- 5: Add collocation defects enforcing $t_f \frac{d\tilde{\mathbf{x}}}{d\tau} - \mathbf{f}(\tilde{\mathbf{x}}, \tilde{\mathbf{u}}) = \mathbf{0}$.
- 6: Add path constraints $\mathbf{h}(\tilde{\mathbf{x}}, \tilde{\mathbf{u}}) \leq \mathbf{0}$ at nodes or quadrature points.
- 7: **end for**
- 8: Add the terminal manifold constraint $\psi(\tilde{\mathbf{x}}(1)) = \mathbf{0}$.
- 9: Approximate the objective with quadrature:
- 10: $J \approx \phi(\tilde{\mathbf{x}}(1)) + t_f \sum_k w_k L(\tilde{\mathbf{x}}(\tau_k), \tilde{\mathbf{u}}(\tau_k))$.
- 11: **while** the NLP residuals or defect norms remain too large **do**
- 12: Solve the sparse NLP.
- 13: If convergence stalls, refine the mesh or relax the terminal target and continue homotopically.
- 14: **end while**
- 15: Reintegrate the optimized control on the full nonlinear dynamics to verify that the defects predict a true trajectory.
- 16: **return** The verified trajectory, control history, and terminal time.

Algorithm 6.1: Moving-Target Direct Collocation Workflow

6.4 Repeatability: The Stochastic OCP

The previous sections frame a deterministic problem: "Find the swing that maximizes terminal speed."

But as we argued in Chapter 1, biological systems and athletes do not optimize for peak deterministic performance. If you instruct an NLP solver to maximize golf club speed without regard for motor noise, the solver will ride the very edge of the constraint boundaries. The resulting trajectory may be extremely sensitive to execution errors; small perturbations in the state can lead to large deviations in the terminal condition $\psi(\mathbf{x}(t_f))$. This is quantified by the condition number of the Jacobian $\nabla_{\mathbf{x}}\psi$ at the optimal trajectory.

Principle 6.1. The Repeatability Addendum

The theoretically optimal trajectory is useless if its local topology amplifies small execution errors into catastrophic terminal misses. The true objective must account for *variance*.

We modify our deterministic cost $J = \phi(\mathbf{x}(t_f)) + \int L$ into a stochastic expectation:

$$\min_{\gamma^*} \mathbb{E}[\phi(\mathbf{x}(t_f, \mathbf{w}))] + \lambda \cdot \text{Var}[\psi(\mathbf{x}(t_f, \mathbf{w}))] \quad (6.9)$$

where $\mathbf{w} \sim \mathcal{N}(0, \Sigma_w)$ represents signal-dependent motor noise (a known property of human muscle actuation, where noise scales proportionally to the applied torque).

To solve this in a collocation framework without massive Monte Carlo simulations, we use **Covariance Steering** or **Linear Covariance Analysis**. We augment the state vector $\mathbf{x}(t)$ with its local covariance matrix $\Sigma(t)$. The dynamics of $\Sigma(t)$ are propagated alongside the nominal trajectory using the linearized Jacobian matrices along the path.

The optimizer is now forced to "widen the funnel" as it approaches the moving goal. It sacrifices a small amount of peak clubhead speed to select a trajectory geometry where the passive dynamics are heavily stabilizing in the transverse direction. The result is a trajectory that a human can actually *repeat*.

6.5 Evolutionary Methods as Complementary Optimizers

Gradient-based transcription methods remain the primary tools of this book because they exploit analytic structure, expose multipliers and costates, and integrate naturally with the Riccati and funnel machinery developed elsewhere. Even so, there are important regimes where **evolutionary** or **genetic** methods are valuable companions rather than competitors.

When the control policy is low-dimensional, the cost landscape is badly nonconvex, or contact-rich simulation makes derivatives unreliable, practitioners often turn to genetic algorithms, evolution strategies, differential evolution, or cross-entropy style search [10, 24, 11]. These methods optimize a population of candidate trajectories or policy parameters by repeatedly scoring, selecting, mutating, and recombining them.

Remark 6.2. Where Evolutionary Search Fits

In the Control Is Motion framework, evolutionary methods are best viewed as *outer-loop search tools*. They are useful for generating warm starts, choosing high-level swing style parameters, tuning phase transitions, or exploring discontinuous task families where gradient information is poor. Once a promising family of motions has been discovered, direct collocation, DDP, and funnel verification should still be used to refine and certify the motion.

Require: Policy or trajectory parameterization θ , simulator, population size M , elite count M_e .

Ensure: A high-performing parameter vector θ^* that can seed a local optimizer.

- 1: Sample an initial population $\{\theta^{(m)}\}_{m=1}^M$ around a biomechanically plausible nominal swing.
- 2: **for** generation $g = 1, 2, \dots, G$ **do**
- 3: **for** each candidate $\theta^{(m)}$ **do**
- 4: Simulate the full swing and score terminal speed, impact feasibility, and repeatability.
- 5: Reject candidates that violate hard constraints or miss the terminal manifold.
- 6: **end for**
- 7: Rank the feasible population by objective value.
- 8: Keep the top M_e elites.
- 9: Generate the next population by mutation, recombination, or covariance adaptation around the elites.
- 10: **end for**
- 11: Select the best feasible candidate θ^* .
- 12: Use θ^* to initialize direct collocation or DDP for high-accuracy refinement.
- 13: **return** θ^*

Algorithm 6.2: Population-Based Search for a Swing Parameterization

Key Idea 6.2. Global Search, Local Certification

A genetic or evolutionary method can help *find* a promising family of motions, but it does not by itself explain why the motion is stable or repeatable. The book's geometric pipeline remains: search globally if needed, then refine locally with derivatives, and finally certify robustness with funnels or covariance analysis.

6.6 Summary

Concept	Significance
Moving Target OCP	Finding a trajectory that optimizes terminal kinematics when intersecting a spatial manifold, without prescribing the clock time.
Multi-Shooting	Discretizing the time domain into segments, using numerical integrators to enforce dynamics constraints between nodes.
Direct Collocation	Representing the entire trajectory as splines and enforcing dynamics algebraically at discrete points arrayed along the trajectory.
Passive Exploitation	Solvers can exploit underactuated topologies through coupling matrices such as M_{21} and C_2 , subject to the stated model.
Stochastic Robustness	Augmenting the OCP with expected variance to ensure the resulting trajectory can be consistently executed by noisy nonlinear plants.
Evolutionary Search	A derivative-free outer loop for discovering warm starts or task-family parameters before derivative-based refinement.

Determining the shape of the reference trajectory is central to Control Is Motion. With a passive geometry model and a candidate reference trajectory in hand, the next step is to estimate a control tube around that trajectory and state the assumptions under which disturbances remain inside it. We turn to funnel synthesis in Chapter 7.

6.7 Exercises

- 6.1. (Free Terminal Time Formulation.)** Explain how one sets up an OCP where final time t_f is a decision variable in a fixed-grid Direct Collocation solver. (Hint: consider scaling time $t = \tau \cdot t_f$, where $\tau \in [0, 1]$). Show how the dynamic constraint $\dot{\mathbf{x}} = \mathbf{f}$ changes under this substitution.
- 6.2. (Kinematic Goals vs Setpoints.)** You are optimizing a baseball bat swing. Your state is $(\theta, \dot{\theta})$. Assume the ball arrives at the plate between $t = 0.5$ and $t = 0.6$ seconds. Formulate the path constraint and the terminal manifold constraint ψ assuming you want maximum $\dot{\theta}$ when $\theta = \pi/2$, but the exact time of impact within that window does not matter.
- 6.3. (Signal-Dependent Noise Penalty.)** Human muscle torque output τ has variance $\sigma^2 = k|\tau|^2$. Write a modified integral cost function $L(\mathbf{x}, \tau)$ that penalizes the introduction of variance into the system. Interpret what this does to the optimal trajectory shape.

*The useful motion is rarely just the fastest.
It is the one that survives the stated disturbances.*

Chapter 7

Funnel Synthesis

Draft Notice. This chapter is under active development and contains only a structural outline with preliminary material. Full exposition, worked examples, proofs, and exercises will be added in future editions.

You cannot control the exact
trajectory a stone will take down a
mountain.
But you can build a trough to ensure it
reaches the bottom.

In Plain Language 7.1. Building a Mathematical Trough

Even if a model supplies a useful reference path for a golf swing, no human executes the same path exactly twice. Muscles twitch, timing fluctuates, and wind blows. A funnel or tube around the path is a way to state a local certificate: for the declared model, controller, disturbance set, and time interval, states inside the certified set remain admissible and move toward the terminal condition. The certificate is mathematical, but it is only as strong as the model and assumptions used to compute it.

7.1 Beyond LQR: The Need for Certified Invariance

In Chapter 4, we showed that applying time-varying Linear Quadratic Regulation (LQR) to the transverse dynamics creates a locally contracting envelope of stability. The Riccati equation naturally shapes this envelope into a “funnel” that narrows as the terminal constraint approaches. The idea of composing such funnels into motion planning trees was introduced by Tedrake [25] (LQR-Trees) and extended by Majumdar and Tedrake [17] using sum-of-squares verification.

However, LQR guarantees are inherently *local*. They rely on the linear approximation of the transverse dynamics $\dot{\xi} = \mathbf{A}_{\perp}(t)\xi + \mathbf{B}_{\perp}(t)\delta\mathbf{u}$. For highly nonlinear physical systems, a disturbance large enough to leave the local neighborhood can invalidate the linear approximation. The consequence may be large terminal error or loss of the target constraint, but the threshold depends on the reference trajectory, controller, and nonlinear model.

We need a mathematically rigorous way to define a *verified boundary* for the funnel. For the fully nonlinear equations of motion, the goal is to certify that any state starting inside the candidate set at time $t = 0$ remains inside it for the modeled interval $t \in [0, t_f]$.

Definition 7.1. Dynamic Funnel

Given a reference trajectory $\gamma^*(t)$, a feedback controller $\mathbf{u} = \mathbf{k}(\mathbf{x}, t)$, and a scalar boundary function $V(\mathbf{x}, t)$, the set:

$$\mathcal{F}(t) = \{\mathbf{x} \in \mathcal{X} \mid V(\mathbf{x}, t) \leq 1\} \quad (7.1)$$

constitutes a valid **dynamic funnel** if the set is positively invariant under the closed-loop dynamics. That is, if $\mathbf{x}(0) \in \mathcal{F}(0)$, then $\mathbf{x}(t) \in \mathcal{F}(t)$ for all $t \in [0, t_f]$.

Remark 7.1. Funnels, Reachable Sets, and What is Actually Certified

Attainable-set language from geometric control [1] is useful here, but the objects should not be conflated. A computed funnel is a verified subset of states for a chosen closed-loop model, controller, and time horizon. It may sit inside the true reachable set, coincide with only part of it, or become conservative because of the Lyapunov ansatz and SOS relaxation. Hamiltonian or symplectic structure can inform the underlying optimal-control problem, yet the numerical certificate still stands or falls on the stated closed-loop dynamics and the boundary condition $\dot{V} < 0$ on $V = 1$.

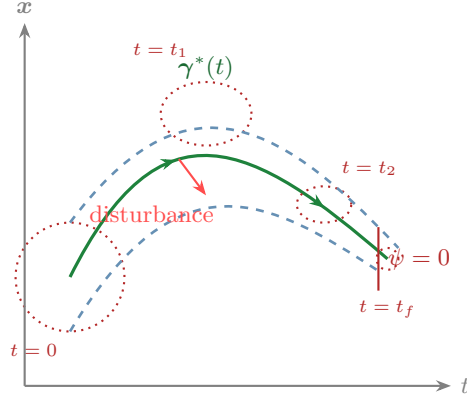
7.2 Time-Varying Lyapunov Functions for Trajectories

To verify invariance, we use the machinery of Lyapunov theory, adapted for trajectories rather than equilibria. We require our boundary function $V(\mathbf{x}, t)$ to act as a time-varying Lyapunov function.

Theorem 7.1 (Funnel Invariance). *The set $\mathcal{F}(t) = \{\mathbf{x} \mid V(\mathbf{x}, t) \leq 1\}$ is invariant if, on the boundary where $V(\mathbf{x}, t) = 1$, the function is strictly decreasing along the system trajectories:*

$$\dot{V}(\mathbf{x}, t) = \frac{\partial V}{\partial t} + \nabla_{\mathbf{x}} V(\mathbf{x}, t) \cdot \mathbf{f}(\mathbf{x}, \mathbf{k}(\mathbf{x}, t)) < 0 \quad \text{for all } \mathbf{x} \text{ s.t. } V(\mathbf{x}, t) = 1. \quad (7.2)$$

If the derivative of the boundary function points inward everywhere on the boundary, trajectories may circulate inside the funnel, but they can never escape it. For a moving target problem, we want the spatial volume of $\mathcal{F}(t)$ to shrink as $t \rightarrow t_f$, terminating in a tight bounded region at impact.



Key Idea 7.1. Funnels Define the Usable State Space

A computed funnel $\mathcal{F}(t)$ is an analysis certificate for a stated closed-loop model. It defines a conservative set of initial conditions (for example, the kinematic state at the top of the backswing) that should remain inside the verified tube and satisfy the terminal constraint $\psi(\mathbf{x}(t_f)) = 0$ under the modeled disturbances. The time-varying geometry of the funnel—narrowing as $t \rightarrow t_f$ —encodes the stability structure of the transverse dynamics along the reference trajectory.

7.3 Sums-of-Squares (SOS) Programming

Proving that $\dot{V} < 0$ on the boundary $V = 1$ for a general nonlinear function $\mathbf{f}$ is famously difficult. We have an infinite number of states on the boundary to check. We circumvent this by restricting our dynamics $\mathbf{f}$ algorithms to polynomials (via Taylor expansion or substitution) and leveraging **Sums-of-Squares (SOS) Programming** [3, 17].

A polynomial $p(x)$ is a Sum of Squares if it can be written as $p(x) = \sum_i q_i(x)^2$ for some polynomials $q_i(x)$. If a polynomial is SOS, it is globally non-negative. Testing if a polynomial is SOS turns out to be equivalent to solving a Semidefinite Program (SDP)—a convex optimization problem that can be solved very efficiently.

We can reframe Theorem 7.1 using the S-procedure. To guarantee that $\dot{V}(\mathbf{x}, t) < 0$ whenever $V(\mathbf{x}, t) = 1$, we require:

$$-\dot{V}(\mathbf{x}, t) - L(\mathbf{x}, t)(1 - V(\mathbf{x}, t)) \text{ is SOS} \quad (7.3)$$

where $L(\mathbf{x}, t)$ is a strictly positive polynomial multiplier. If $V(\mathbf{x}, t) = 1$, the second term vanishes, and the SOS condition forces $-\dot{V} > 0$, achieving the invariance proof.

7.3.1 Implementation Sketch: Funnel Synthesis

The workflow below is the practical bridge from a nominal trajectory to a certifiable tube of allowable perturbations.

Require: Nominal trajectory $\gamma^*(t)$, feedback law $\mathbf{u} = \mathbf{k}(\mathbf{x}, t)$, polynomial closed-loop model, time grid $\{t_k\}$.

Ensure: Time-varying Lyapunov matrices $\mathbf{P}(t_k)$ defining a verified funnel.

- 1: Linearize or polynomially approximate the closed-loop dynamics around $\gamma^*(t)$.
- 2: Initialize $\mathbf{P}(t_k)$ from the transverse Riccati solution or another stabilizing quadratic form.
- 3: **for** each node t_k **do**
- 4: Define the candidate boundary $V_k(\mathbf{x}) = (\mathbf{x} - \gamma^*(t_k))^\top \mathbf{P}(t_k)(\mathbf{x} - \gamma^*(t_k))$.
- 5: Introduce SOS multipliers enforcing $-\dot{V}_k - L_k(1 - V_k)$ to be nonnegative on the boundary.
- 6: Add positive-definiteness constraints on $\mathbf{P}(t_k)$.
- 7: **end for**
- 8: **while** the SDP remains feasible **do**
- 9: Maximize the ellipsoid volume or a surrogate such as $-\log \det \mathbf{P}(t_k)$ across the grid.
- 10: If infeasibility appears, shrink the candidate funnel, refine the polynomial model, or shorten the horizon.
- 11: **end while**
- 12: Interpolate the verified $\mathbf{P}(t_k)$ between nodes to obtain a continuous funnel approximation.
- 13: **return** The verified funnel family $\mathcal{F}(t) = \{\mathbf{x} \mid V(\mathbf{x}, t) \leq 1\}$

Algorithm 7.1: Trajectory-Centered Funnel Synthesis

7.4 Computing the Funnel for the Golf Swing

Let us return to the double-pendulum golf model. The reference trajectory $\gamma^*(t)$ from Chapter 6 gives nominal kinematics. The transverse LQR from Chapter 4 gives a feedback matrix $\mathbf{K}(t)$ designed to stabilize selected deviations. The goal is to compute a conservative funnel around this sequence and state the model assumptions under which the closed-loop system remains inside it.

We define our candidate Lyapunov function as a time-varying quadratic form:

$$V(\mathbf{x}, t) = (\mathbf{x} - \gamma^*(t))^\top \mathbf{P}(t)(\mathbf{x} - \gamma^*(t)) \quad (7.4)$$

where $\mathbf{P}(t)$ is a positive definite matrix that changes continuously along the trajectory. The funnel boundary is the time-varying ellipsoid where $V = 1$. By altering $\mathbf{P}(t)$, we alter the shape and orientation of the funnel.

Remark 7.2. Value Functions and Hamilton-Jacobi-Bellman Theory

The Lyapunov function $V(\mathbf{x}, t)$ in funnel synthesis is deeply connected to the value function from dynamic programming, and more broadly to the **Hamilton-Jacobi-Bellman (HJB) equation** in optimal control. The value function $J^*(\mathbf{x}, t)$ represents the optimal cost to reach the terminal constraint from state $\mathbf{x}$ at time t . In the deterministic setting of Agrachev and Sachkov's framework [1], the value function satisfies the HJB partial differential equation:

$$\frac{\partial J^*}{\partial t} + \min_{\mathbf{u}} [L(\mathbf{x}, \mathbf{u}) + \nabla_{\mathbf{x}} J^* \cdot \mathbf{f}(\mathbf{x}, \mathbf{u})] = 0. \quad (7.5)$$

The level sets of $J^*(\mathbf{x}, t) = c$ define the attainable set from which cost c can still be incurred to reach the target. The funnel synthesized via Lyapunov methods is a conservative approximation of this optimal value function: it guarantees invariance via the $\dot{V} < 0$ condition, making it a certificate of feasibility. Computing the exact value function would require solving the HJB equation, which is generally difficult; the Lyapunov/SOS approach trades exactness for verifiability.

Example 7.1. Volume Maximization via SDP

We transcribe the continuous time t into discrete nodes t_k , just as we did in direct collocation. At each node, we set up an SDP that seeks to *maximize* the volume of the ellipsoid defined by $\mathbf{P}(t_k)$, subject to the SOS invariance constraints connecting it to adjacent nodes.

The optimizer actively computes the exact structural limits of the golf swing's passive dynamics. It discovers that early in the downswing, the funnel can be incredibly "wide" (forgiving), but as centrifugal forces build, the SOS constraints force $\mathbf{P}(t_k)$ to become highly elongated along the unactuated dimensions in order to maintain the invariance guarantee. The resulting geometry is a true numerical portrait of athletic consistency.

7.5 Summary

Concept	Role in Funnel Synthesis
Lyapunov Function	Time-varying scalar $V(\mathbf{x}, t)$ that measures distance from the reference trajectory; boundary $V = 1$ defines the funnel surface.
Positive Invariance	Mathematical property ensuring that $\mathbf{x}(0) \in \mathcal{F}(0)$ implies $\mathbf{x}(t) \in \mathcal{F}(t)$ for all $t \in [0, t_f]$.
Sums-of-Squares (SOS)	Convex optimization technique to verify that $\dot{V} < 0$ on the funnel boundary, eliminating the need for numerical sampling.
Semidefinite Program	Optimization problem solved to compute the time-varying matrix $\mathbf{P}(t)$ that maximizes funnel volume subject to SOS constraints.
Transverse LQR	Provides initial feedback gains $\mathbf{K}(t)$ for stabilizing deviations orthogonal to the reference trajectory.
Funnel Synthesis Workflow	Practical sequence of polynomial approximation, SOS certification, and ellipsoid-volume maximization around the nominal trajectory.

In Chapter 8, we will explore how to make these funnels robust to temporal variation by migrating out of the time domain entirely and into the phase domain.

Chapter 8

Phase-Variable Control

Draft Notice. This chapter is under active development and contains only a structural outline with preliminary material. Full exposition, worked examples, proofs, and exercises will be added in future editions.

Time is a river that carries us forward.
Phase is the boat we build to navigate
it.

In Plain Language 8.1. Ditching the Clock

If a robot is programmed to walk across a room in exactly 10 seconds, and you push it backwards halfway through, a naive time-tracking controller can interpret the lag as a large positional error. A human, by contrast, often organizes walking around physical milestones: "When my left foot hits the ground, swing my right leg forward." This is called using a "phase variable" instead of clock time. A phase variable is a physical marker (like the angle of the left leg) that tells the system where it is in the motion. The benefit is not automatic robustness; it depends on choosing a marker that remains observable, monotone, and well behaved over the operating region.

8.1 Spatial Paths vs. Temporal Execution

One common limitation of direct time-tracking architectures, when applied to biomechanics or advanced robotics, is their rigid adherence to the clock. If you construct a time-varying trajectory $\gamma^*(t)$ for a robotic arm and command a tracking controller to follow it, the controller evaluates the error as $e(t) = \mathbf{x}(t) - \gamma^*(t)$.

If the robot hits an unmodeled friction patch and slows down, the reference trajectory $\gamma^*(t)$ continues advancing. The controller can calculate a large position error not because the robot has left the intended path, but because it is behind schedule. Depending on gains, actuator limits, and saturation handling, this can produce excessive catch-up commands.

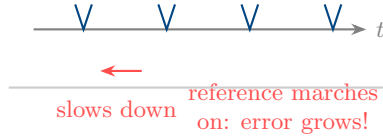
Key Idea 8.1. The Autonomy of Time

A system executing a motion can often be controlled more safely when progress is measured by state-dependent phase rather than by clock time alone. If the system is impeded, the reference can slow down. The mathematical task is to decouple the spatial geometry of the motion from its temporal execution by replacing chronological time t with a locally monotone **phase variable** s .

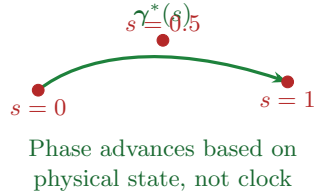
Instead of parameterizing our optimal trajectory as $\gamma^*(t)$, we parameterize it as a function of the phase: $\gamma^*(s)$. The phase $s \in [0, 1]$ represents the percentage of motion completed. For a walking robot, s could be the forward position of the hip. For a golf swing, s could be the angular position of the shoulder relative to the ball. The reference trajectory is now a spatial curve with a separately modeled timing law.

This construction is publication-safe only after the phase variable's domain, monotonicity condition, and reset behavior are specified. This is the same practical issue that appears in maneuver regulation, transverse linearization, and hybrid zero dynamics [13, 27, 21].

Time-Based: $\gamma^*(t)$



Phase-Based: $\gamma^*(s)$

**Definition 8.1. Phase Variable**

A phase variable $s(\mathbf{x})$ is a continuously differentiable scalar function of the state $\mathbf{x}$ such that its time derivative is strictly positive along all feasible trajectories inside the nominal funnel:

$$\dot{s}(\mathbf{x}) > 0 \quad \text{for all } \mathbf{x} \in \mathcal{F}. \quad (8.1)$$

8.2 Virtual Constraints

If the goal is no longer to match a time sequence but rather to conform to a spatial path defined by $s(\mathbf{x})$, we can express the control objective as a set of holonomic constraints on the state. However, because these constraints are not mechanical linkages but rather enforced by software, they are termed **Virtual Constraints** [27]. The theory of virtual constraints

and hybrid zero dynamics was developed primarily in the context of bipedal locomotion by Westervelt, Grizzle, and collaborators.

Let our control objective be to drive the state $\mathbf{x}$ to the invariant manifold $\mathcal{Z}$ defined by:

$$\mathcal{Z} = \{\mathbf{x} \mid \mathbf{x} - \gamma^*(s(\mathbf{x})) = \mathbf{0}\}. \quad (8.2)$$

In Plain Language 8.2. Mathematical Reminder: The Invariant Manifold

Recall our manifold definition from Volume 0: a curved geometric surface. The **Virtual Constraints** physically force the system to stick to a very specific, lower-dimensional "submanifold" embedded within the full State Space. We call this an **Invariant Manifold** because once the feedback controller forces the system onto this surface, the system's own natural physics (combined with the controller) ensures it never leaves it. The system "slides" perfectly along this curved surface toward the goal.

By using partial feedback linearization (Chapter 5), we can design a synthetic input $\mathbf{v}_a$ that drives the actuated degrees of freedom $\mathbf{q}_a$ onto their nominal surfaces proportional to the phase:

$$\mathbf{y} = \mathbf{q}_a - \gamma_a^*(s(\mathbf{x})) \rightarrow \mathbf{0}. \quad (8.3)$$

As $\mathbf{y} \rightarrow \mathbf{0}$, the system converges toward the virtual constraints. Crucially, the rate at which it progresses along the path is entirely governed by $\dot{s}$, which is itself determined by the current momentum and state of the system, not a stopwatch.

8.3 Hybrid Zero Dynamics

When the virtual constraints are perfectly satisfied ($\mathbf{y} = \mathbf{0}$), the entire complexity of the n -dimensional state space collapses down into a highly reduced dynamical system that evolves strictly along the manifold $\mathcal{Z}$. This is the **Zero Dynamics** of the system.

For systems that undergo discrete impacts (a foot hitting the ground, or a golf club striking a ball), the system evolves via continuous differential equations until an impact surface is triggered, followed by an instantaneous jump map (a reset) dictated by conservation of momentum.

Principle 8.1. Hybrid Zero Dynamics (HZD)

A control architecture achieves Hybrid Zero Dynamics if the invariant manifold $\mathcal{Z}$ (the zero dynamics surface) is invariant not just under the continuous flow of the differential equations, but also invariant under the discrete impact map. That is, an impact starting on the manifold $\mathcal{Z}$ must immediately map to a new state that is also on $\mathcal{Z}$.

By enforcing HZD, we can design rhythmic walking gaits that are astonishingly robust. If the robot trips, its phase slows down, but the virtual constraints remain active, keeping the limbs on the correct spatial path until the next footfall resets the sequence. The temporal sequence arises naturally from the interplay of gravity and momentum.

8.4 Application: The Golf Downswing

While the golf swing is not a periodic walking gait, a phase-variable approach can still be useful if the selected marker remains monotone through the interval being modeled. A candidate phase might be shoulder angle, hand path progress, or another measured progress coordinate; none is automatically valid outside the stated model. The target event is impact, not a universal manifold $s = 1.0$ independent of the coordinate choice.

By structuring a model through virtual constraints, the analysis can separate spatial deviations from timing delays. A transverse feedback law (Chapter 4) can correct selected spatial errors, while the phase variable schedules arm actuation (u_1) relative to measured progress rather than clock time. This can improve timing robustness, but it does not guarantee perfect passive impact dynamics; impact quality still depends on state estimation, actuator limits, contact modeling, and the validity of the chosen phase coordinate.

8.5 Summary

Concept	Definition & Role
Phase Variable	Scalar function $s(\mathbf{x})$ that increases monotonically along trajectories and decouples spatial path geometry from temporal execution.
Virtual Constraints	Control objective of the form $\mathbf{x} - \boldsymbol{\gamma}^*(s(\mathbf{x})) = 0$ that drives the system onto a lower-dimensional invariant manifold.
Invariant Manifold	The manifold $\mathcal{Z} = \{\mathbf{x} \mid \mathbf{x} = \boldsymbol{\gamma}^*(s(\mathbf{x}))\}$ on which the system evolves when virtual constraints are satisfied.
Hybrid Zero Dynamics	The reduced dynamics of the system when constrained to $\mathcal{Z}$, including discrete impact maps that preserve manifold membership.
Partial Feedback Lin.	Control design that cancels nonlinearities on actuated DOFs while allowing unactuated DOFs to evolve freely on the manifold.

Chapter 9

Stochastic Trajectories & Motor Variability

Draft Notice. This chapter is under active development and contains only a structural outline with preliminary material. Full exposition, worked examples, proofs, and exercises will be added in future editions.

Perfection is inherently fragile.
Biological optimization seeks the
strongest balance between intent and
reality.

In Plain Language 9.1. The Harder You Try, The More You Miss

If you program a robot motor to apply 50 units of force, it applies exactly 50 units of force, every single time. Human muscles don't work like that. The harder you try to contract a muscle, the more "noisy" and jittery the actual force becomes. This is a massive problem for trying to swing a golf club as fast as humanly possible: the closer you get to your maximum physical effort, the more unpredictable your motion becomes. This chapter explains the math behind why smooth, controlled movements are more accurate than jerky, maximum-effort ones. We'll see that a truly "optimal" golf swing doesn't use 100% of your strength, because dialing it back slightly eliminates a huge amount of physical chaos, making the swing actually repeatable.

9.1 Signal-Dependent Noise

When a roboticist specifies a torque of 50 Nm, the motor delivers 50 Nm with a tiny, constant variance. Biological actuators do not work this way. Human muscles generate force by recruiting discrete motor units [14]. As higher forces are required, larger, more fast-twitch oriented motor units are recruited. This physiological reality manifests as a fundamental control challenge: **motor noise is proportional to the command signal** [12].

Definition 9.1. Signal-Dependent Noise (SDN)

A control input $\mathbf{u}(t)$ applied by a biological system is actually realized as a stochastic variable $\tilde{\mathbf{u}}(t)$:

$$\tilde{\mathbf{u}}(t) = \mathbf{u}(t) + \mathbf{W}\mathbf{u}(t) \cdot \boldsymbol{\epsilon}(t) \quad (9.1)$$

where $\mathbf{W}$ is a scaling matrix representing the coefficient of variation, and $\boldsymbol{\epsilon}(t) \sim \mathcal{N}(0, I)$ is white noise. That is, the variance of the applied force grows squarely with the magnitude of the ordered force: $\text{Var}(\tilde{\mathbf{u}}) = c\mathbf{u}^2$.

This simple reality crumbles deterministic optimal control. If you compute an optimal trajectory that requires maximum muscular exertion, the trajectory will mathematically possess minimum variance only if noise is constant. Under SDN, maximum exertion guarantees maximum chaotic noise.

9.2 Minimum Variance Theory of Human Movement

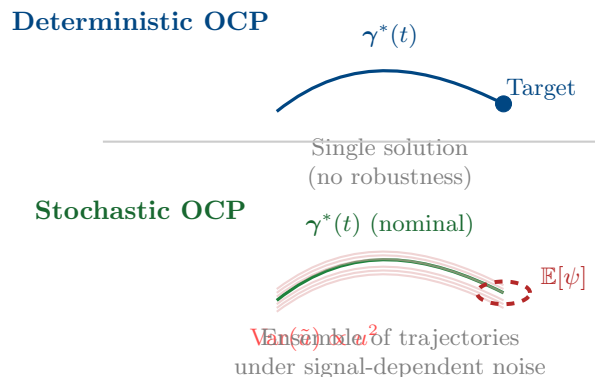
Why do humans move in smooth, bell-shaped velocity profiles rather than jerky “bang-bang” optimal control solutions when reaching for coffee?

One influential explanation is the **minimum jerk** theory of Flash and Hogan [9], which posits that the nervous system minimizes $\int \ddot{x}^2 dt$ (the integral of squared jerk). This kinematic criterion successfully predicts many features of reaching movements. An alternative, physically grounded explanation comes from the **Minimum Variance Theory** of Harris and Wolpert [12], which derives smooth trajectories from the assumption of signal-dependent noise. Both theories remain active areas of research, and they are not mutually exclusive—minimum-jerk trajectories can emerge as a consequence of variance minimization under certain noise models.

Key Idea 9.1. Optimization for Repeatability

The human nervous system seeks to move in a way that minimizes the final endpoint variance evaluated across multiple trials. Under signal-dependent noise, this objective naturally selects for smooth trajectories with bounded control effort.

Under SDN, the only way to minimize endpoint variance (ensure you actually grasp the coffee cup smoothly) is to minimize sudden spikes in required actuator forces, because large forces inject huge amounts of uncertainty into the dynamics. Smooth trajectories inherently possess lower peak torque commands than rapid accelerations and decelerations. Smoothness is not the explicitly programmed objective; it is an emergent property of attempting to be consistently accurate under the unavoidable reality of signal-dependent noise.



9.3 The Speed-Accuracy Tradeoff in the Golf Swing

Fitts's Law [8] states that the faster you try to move to a target, the less accurate you become. In golf, we see the supreme manifestation of this phenomenon. The deterministic optimal control solution (Chapter 6) tells the solver to maximize input torque to achieve theoretically maximal clubhead speed.

However, if we introduce the Stochastic OCP (via Covariance Steering) from Chapter 6 and apply the realistic premise of SDN, the optimizer faces a stark tradeoff.

Applying maximum u_1 torque during the downswing injects enormous variance into the kinematic state. The unactuated club (θ_2) begins whipping through its zero dynamics, but its exact phase progression is now highly uncertain due to the noise flooding the arms.

To ensure the clubface squares up at the moving goal, the stochastic optimizer will *deliberately back off* the peak requested torque. It trades a small reduction in peak terminal performance for a substantial reduction in angular variance at the target manifold constraint. The optimal biological swing is mathematically proven to be a sub-maximal physical exertion when signal-dependent noise is accounted for.

9.4 Summary

Concept	Implication for Control
Signal-Dependent Noise (SDN)	Muscle noise variance proportional to command: $\text{Var}(\tilde{u}) = cu^2$. Requires bounded control effort for repeatability.
Minimum Variance Theory	Explains smooth, controlled velocity profiles as solutions to endpoint error variance minimization under SDN.
Stochastic OCP	Optimal control formulation that minimizes $\mathbb{E}[\phi(\mathbf{x}(t_f))] + \lambda \cdot \text{Var}[\psi(\mathbf{x}(t_f))]$.
Covariance Steering	Technique to propagate the covariance matrix $\Sigma(t)$ alongside the nominal trajectory during optimization.
Speed-Accuracy Tradeoff	Formal manifestation of Fitts's Law: higher peak forces inject noise proportional to force squared.

The extension of the deterministic Pontryagin framework to stochastic systems is developed rigorously in Agrachev and Sachkov [1]. When the system is subject to signal-dependent noise (SDN), the traditional Hamiltonian maximization is supplemented by terms encoding risk: the optimizer must maximize expected performance while managing the variance of terminal conditions under noise propagation. This leads to **risk-sensitive control** problems where the cost includes an entropy-like term penalizing uncertainty.

For the golf swing under SDN, the stochastic extension of the attainable set framework tells us that not all nominally reachable configurations are equally *robustly* reachable. A configuration in the interior of the deterministic attainable set may lie on the boundary of the stochastic attainable set if noise-driven perturbations commonly push trajectories away from it. By contrast, configurations that are “stable” with respect to noise (in the direction of the control Lie algebra) remain deeply inside the stochastic attainable set. The funnel computed in Chapter 7 with stochastic cost weighting thus encodes not just reachability, but **robust attainment**—guarantee of success under unavoidable noise.

9.5 Summary

Concept	Implication for Control
Signal-Dependent Noise (SDN)	Muscle noise variance proportional to command: $\text{Var}(\tilde{u}) = cu^2$. Requires bounded control effort for repeatability.
Minimum Variance Theory	Explains smooth, controlled velocity profiles as solutions to endpoint error variance minimization under SDN.
Stochastic OCP	Optimal control formulation that minimizes $\mathbb{E}[\phi(\mathbf{x}(t_f))] + \lambda \cdot \text{Var}[\psi(\mathbf{x}(t_f))]$.
Covariance Steering	Technique to propagate the covariance matrix $\Sigma(t)$ alongside the nominal trajectory during optimization.
Speed-Accuracy Tradeoff	Formal manifestation of Fitts's Law: higher peak forces inject noise proportional to force squared.

Chapter 10

Learning to Move

Draft Notice. This chapter is under active development and contains only a structural outline with preliminary material. Full exposition, worked examples, proofs, and exercises will be added in future editions.

The first swing establishes the manifold. The ten-thousandth swing perfects the funnel.

In Plain Language 10.1. Practice Makes Funnels

When you swing a golf club for the first time, your brain is guessing how much force to use, guessing the weight of the club, and guessing how your muscles will respond. The mathematical models we've built so far are just like that first swing—a really smart guess, but still a guess. Real physics (wind, friction, fatigue) is too messy to model perfectly. This chapter is about how systems **learn**. Instead of trying to calculate the perfect math equations from scratch every time, the system tries a movement, measures how badly it missed the target, and updates its "muscle memory" for the next try. Over thousands of repetitions, the brain (or the computer) learns the true physics of the motion without ever needing the exact equations. It builds a whole library of these learned motions, letting it instantly adjust to new situations without thinking.

10.1 Iterative Learning Control (ILC)

Trajectory optimization (Chapter 6) computes a reference motion strictly from a mathematical model. However, dynamic models of humans—and even robots—are invariably flawed. Frictional forces are unmodeled, inertias are estimated, and actuators possess hidden dynamics. If you execute a computed reference $\gamma^*(t)$ open-loop, the physical system will likely fail to strike the moving target due to these discrepancies.

Iterative Learning Control (ILC) resolves this by leveraging the repetitive nature of tasks like a golf swing. ILC exploits the premise that while models are inaccurate, *the unmodeled dynamics are highly deterministic across repeated trials*.

Remark 10.1. ILC and the Orbit of Feasible Trajectories

From the perspective of geometric control theory as presented by Agrachev and Sachkov [1], each iterative trial of ILC represents an exploration within the orbit of feasible trajectories generated by the control Lie algebra. The true plant's dynamics define an actual (unknown) orbit in state space; the nominal model defines an approximate orbit. ILC learns the discrepancy between these orbits by iteratively modulating the feedforward command. The convergence of ILC is thus a convergence toward the true orbit boundary—the set of trajectories reachable on the actual (unknown) plant. Persistent excitation conditions required for identifiability of model parameters correspond to activating the control Lie algebra sufficiently to explore all dimensions of the reachable subspace. In other words, ILC's success in learning motion requires that the control inputs, applied across trials, persistently excite the full Lie algebra of admissible directions.

If a robot under-swings by 5 cm on the first attempt, the controller records this terminal error and updates the feedforward command $\mathbf{u}_{ff}$ for the *second* attempt. Over repeated trials, the system learns the inverse dynamics of the true physical plant without ever explicitly identifying the model parameters.

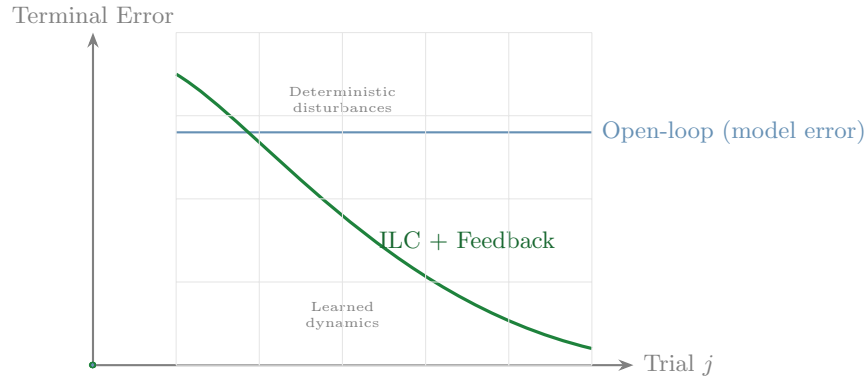
Definition 10.1. ILC Update Law

Let j denote the trial index. The feedforward command for trial $j + 1$ is updated based on the command and the *transverse error* $\boldsymbol{\xi}$ from trial j :

$$\mathbf{u}_j(t) = \mathbf{u}_{ff,j}(t) + \mathbf{K}_\perp(t)\boldsymbol{\xi}_j(t) \quad (10.1)$$

$$\mathbf{u}_{ff,j+1}(t) = \mathbf{u}_{ff,j}(t) + \mathbf{L}(t)\boldsymbol{\xi}_j(t) \quad (10.2)$$

where $\mathbf{L}(t)$ is a learning filter. This updates the nominal trajectory to iteratively cancel out deterministic disturbances.



Require: Nominal feedforward command $\mathbf{u}_{\text{ff},0}(t)$, stabilizing feedback $\mathbf{K}_{\perp}(t)$, learning filter $\mathbf{L}(t)$.

Ensure: A learned command sequence with reduced trial-to-trial error.

- 1: **for** trial $j = 0, 1, 2, \dots$ **do**
- 2: Execute the closed-loop swing:
- 3: $\mathbf{u}_j(t) = \mathbf{u}_{\text{ff},j}(t) + \mathbf{K}_{\perp}(t)\boldsymbol{\xi}_j(t)$.
- 4: Measure the transverse tracking error $\boldsymbol{\xi}_j(t)$ and the terminal miss at impact.
- 5: Filter the measured error to remove non-repeatable sensor noise.
- 6: Update the next feedforward command:
- 7: $\mathbf{u}_{\text{ff},j+1}(t) = \mathbf{u}_{\text{ff},j}(t) + \mathbf{L}(t)\boldsymbol{\xi}_j(t)$.
- 8: **if** the terminal error and update norm are below tolerance **then**
- 9: Stop learning.
- 10: **end if**
- 11: **end for**
- 12: **return** The learned feedforward command $\mathbf{u}_{\text{ff}}^*(t)$

Algorithm 10.1: Iterative Learning Control Around a Nominal Swing

10.2 Policy Search and Reinforcement Learning

While ILC updates the feedforward commands to zero-out deterministic tracking errors, **Policy Search** algorithms optimize the *shape of the funnel* itself.

Instead of computing the transverse LQR gains $\mathbf{K}_\perp(t)$ from an analytical Riccati equation (Chapter 4), we parameterize the controller as a policy $\pi_\theta(\mathbf{x})$ and search the parameter space θ to minimize the variance at the moving goal under realistic noise.

Reinforcement Learning (RL), specifically policy gradient methods, allows the agent to sample thousands of golf swings in simulation, continuously updating its policy parameters to maximize the expected reward (clubhead speed and squareness at impact). A heavily trained RL agent organically converges on the very principles we mathematically derived in earlier chapters:

1. It abandons rigid time-tracking and autonomously discovers phase-variable mapping.
2. It exploits the underactuated double-pendulum whipping motion to gain speed.
3. It artificially widens its "funnel" during the transition phase, realizing that overly tight feedback control here merely amplifies motor noise and spoils the impact constraints.

10.2.1 Evolutionary Policy Search

Not every learning architecture exposes gradients cleanly. If the policy contains contact logic, lookup tables, gait phase switches, or parameters chosen from expensive black-box simulation, **genetic** and related evolutionary methods become attractive [10, 24, 11]. Their role here is not to replace the geometry; it is to help search over controller families when differentiability is weak or deceptive.

Remark 10.2. Evolutionary Search as a Library Builder

The most natural place for evolutionary methods in this volume is the outer loop over *strategy parameters*: swing tempo, release timing, phase-variable thresholds, or shape parameters of a feedback policy. Once a promising family is found, the resulting motion should still be analyzed with the trajectory-optimization and funnel tools from the previous chapters.

10.3 Trajectory Libraries

A singular optimal trajectory γ^* and its corresponding funnel $\mathcal{F}$ are highly specific to a single objective. What if the golfer wishes to fade the ball, hit a draw, or execute a punch shot out of the rough?

Instead of re-running a massive NLP optimization for every scenario, biological systems and advanced controllers maintain **Trajectory Libraries**. A library consists of several distinct, pre-computed optimal trajectories and their corresponding local funnels.

When faced with a new task (e.g., striking the ball on an uneven lie), the system searches its library for the closest matching funnel. By interpolating between established funnels using techniques like LQR-Trees [25] or simple spatial blending, the system can instantly generate a stable, feasible motion without any online optimization.

Require: Task descriptor $\mathbf{z}$, library entries $\{(\gamma_i^*, \mathcal{F}_i, \mathbf{z}_i)\}$, local adaptation routine.

Ensure: A feasible warm start for the current task.

- 1: Compute similarity scores between the requested task $\mathbf{z}$ and each stored descriptor $\mathbf{z}_i$.
- 2: Select the best matching entry or a small elite subset.
- 3: **if** one funnel already contains the current initial condition and task variation **then**
- 4: Reuse that trajectory and controller directly.
- 5: **else**
- 6: Blend nearby trajectories or policy parameters to form a warm start.
- 7: Run a short local optimization or ILC phase to adapt the warm start to the new task.
- 8: **end if**
- 9: Verify that the adapted trajectory still lies inside a certified funnel near impact.
- 10: **return** The adapted trajectory/controller pair.

Algorithm 10.2: Trajectory-Library Retrieval and Adaptation

10.4 Summary

Concept	Mechanism of Learning
Iterative Learning Control (ILC)	Feedforward update: $\mathbf{u}_{\text{ff},j+1} = \mathbf{u}_{\text{ff},j} + \mathbf{L}(t)\boldsymbol{\xi}_j$. Learns deterministic model mismatch.
Policy Search	Direct optimization of feedback parameters $\boldsymbol{\theta}$ via gradient methods on sampled task performance.
Evolutionary Search	Population-based tuning of policy or trajectory parameters when gradients are unreliable or unavailable.
Reinforcement Learning (RL)	Learns both reference trajectories and feedback policies through trial-and-error in simulation or on real systems.
Trajectory Library	Pre-computed collection of optimal trajectories and funnels for multiple task variants; enables instantaneous adaptation via interpolation.
Model Identification	Learning the true plant dynamics $\mathbf{f}_{\text{true}}(\mathbf{x}, \mathbf{u})$ to replace or refine the nominal model.

Chapter 11

Case Study: The Complete Golf Swing

Draft Notice. This chapter is under active development and contains only a structural outline with preliminary material. Full exposition, worked examples, proofs, and exercises will be added in future editions.

A useful model makes its assumptions visible before it asks for trust.

In Plain Language 11.1. Putting It All Together

Everything we have developed so far can be assembled into a golf-swing model, but the case study should be read as a reproducible modeling program rather than as a proof of a universal swing law. Chapters 1–4 define the trajectory, state-space geometry, and transverse-error language. Chapters 5–8 add underactuation, trajectory optimization, funnels, and phase coordinates. Chapters 9–10 add signal-dependent noise and learning. This chapter shows how those pieces could be combined, and it states the assumptions that would have to be checked before stronger claims about expert performance are justified.

11.1 Unifying the "Control Is Motion" Framework

Throughout this text, we have reframed control around trajectory geometry and local transverse correction. We conclude with a hypothetical 15-DOF musculoskeletal golf-swing model that illustrates how the framework could be assembled into a testable analysis.

The objective is to maximize clubhead speed at the spatial intersection with the golf ball while controlling face orientation and respecting biological torque limits. The terminal time may be treated as a decision variable, but the formulation must still state admissible timing, contact, and phase-coordinate assumptions. Robustness is included through an explicit signal-dependent-noise model.

11.1.1 Phase 1: Defining the Geometry and Optimization

The system is heavily underactuated in this example. The wrists are treated as passive pin joints, which is a simplifying assumption used to expose inertial coupling rather than a complete anatomical model.

We formulate a **Stochastic Moving Target Optimal Control Problem** (Chapter 6). Direct collocation optimizes the trajectory $\gamma^*(s)$ over a spatial phase variable s representing shoulder angle, provided that this phase coordinate remains monotone over the modeled interval (Chapter 8). The integral cost penalizes signal variance through local covariance propagation (Chapter 9).

In the illustrative model, the collocation solver satisfies the $\mathbf{B}^\perp$ annihilator constraint (Chapter 5) and may produce a smooth reference motion. A torque reversal before impact can exploit passive wrist dynamics, but claims about face squaring require explicit contact geometry, grip modeling, and validation against measured swings.

11.1.2 Phase 2: Funnel Synthesis and Robustness

With $\gamma^*(s)$ computed, we analyze the trajectory using **Transverse Linearization** (Chapter 4) [13, 21]. We ignore errors along the tangent of the path only within the chosen phase model and construct a time-varying LQR feedback matrix to regulate selected transverse deviations.

Finally, we compute candidate bounds for this stability region using **Sums-of-Squares Programming** (Chapter 7) [17]. The SDP supplies a boundary function $V(\mathbf{x}, t) = 1$ that certifies invariance under the polynomial model, controller, and disturbance assumptions used in the computation.

Principle 11.1. Time-Varying Funnel Geometry

The computed funnel $\mathcal{F}(t) = \{\mathbf{x} \mid V(\mathbf{x}, t) \leq 1\}$ describes the model's certified region of attraction. The Lyapunov function $V(\mathbf{x}, t) = (\mathbf{x} - \gamma^*(t))^\top \mathbf{P}(t)(\mathbf{x} - \gamma^*(t))$ (Chapter 7) contains a time-varying matrix $\mathbf{P}(t)$ determined by the Riccati equation of the transverse linearization (Chapter 4).

During the early phases of motion, the funnel may be wider if the modeled dynamics tolerate larger transverse deviations without violating the invariant boundary. As the swing approaches transition and downswing, the funnel may tighten because the model becomes more sensitive to timing, phase, and actuation errors.

Any claim about a final 0.15 seconds, near-zero funnel diameter, or passive absorption of muscular noise must be supported by the actual SDP solution and sensitivity analysis. The funnel is a certificate for the stated model, not a general proof of athletic consistency.

This case study illustrates the intended use of the framework: define a motion objective, state the model assumptions, compute a trajectory, and certify the local control region. It does not replace empirical validation, and it should be read as a reproducible modeling program rather than a proof of a universal control language.

Glossary of Important Concepts

Articulated Body Algorithm	A recursive $O(N)$ algorithm for computing the forward dynamics of a kinematic chain, widely used in physics engines.
Configuration Space	The space (often a manifold) of all possible positions of a system.
Contraction Analysis	A framework for analyzing the stability of trajectories with respect to each other, rather than with respect to an equilibrium point.
Control Contraction Metric (CCM)	A Riemannian metric that can be used to synthesize stabilizing feedback controllers for nonlinear systems along arbitrary feasible trajectories.
Degrees of Freedom (DOF)	The minimum number of independent coordinates required to completely specify the configuration of a system.
Exponential Map	The operation that relates a Lie Algebra (representing velocity) to its corresponding Lie Group (representing position/orientation).
Euler Angles	A 3-parameter representation of orientation that is subject to gimbal lock.
Flow	The solution map of a differential equation smoothly moving points along vector fields in state space.
Gimbal Lock	A coordinate singularity where a 3D rotation parameterization loses a degree of freedom.
Jacobian	The matrix of partial derivatives relating joint velocities to task-space velocities.
Kinematics	The study of motion geometry without considering the forces that cause it.
Lagrangian Mechanics	An energy-based formulation of mechanics utilizing kinetic and potential energy to strictly determine the equations of motion.
Lie Algebra	The mathematical space of velocities associated with a Lie Group, corresponding to its tangent space at the identity.
Lie Group	A smooth manifold that is also a mathematical group, typically describing continuous symmetries or rigid body transformations (e.g., $SO(3)$, $SE(3)$).
Lyapunov Stability	The classical definition of stability describing whether a system returns to an equilibrium point when perturbed.
Manifold	A topological space that locally resembles Euclidean space, crucial for rigorous formulation of non-trivial state spaces.

State Space	The space of all possible states of a dynamical system, usually comprising both configuration and momentum/velocity.
Screw Axis	A geometric representation of spatial motion combining a rotation axis and a translation along that axis, foundational for modern rigid body mathematics.
Tangent Space	The vector space of all possible velocities at a specific point on a manifold.
Tangent Bundle	The manifold consisting of all tangent spaces glued together, typically representing the full state space of mechanical systems.
Twist	The spatial velocity of a rigid body, composed of angular and linear velocity components, residing in the Lie Algebra.
Wrench	The spatial force acting on a rigid body, combining torque and linear force.

Further Reading and References

This text is a primer and introductory guide designed to construct an intuitive and unified framework. The formal depths of modern geometric mechanics and nonlinear control are actively pioneered and formalized across several seminal texts. For the reader eager to pursue more mathematically rigorous treatments, we strongly recommend the following resources:

Geometric Control and Robotic Manipulation

- **A Mathematical Introduction to Robotic Manipulation** [19]. The definitive textbook bridging Lie group theory with classical robotics, establishing the Product of Exponentials (PoE) standard.
- **Geometric Control of Mechanical Systems** [5]. An authoritative resource on the differential geometry of mechanical control systems, including connections and geodesics.
- **Robot Modeling and Control** [23]. A highly accessible and rigorous text covering everything from basic Euler-Lagrange forms to robust robotic control.

Advanced Rigid Body Dynamics

- **Rigid Body Dynamics Algorithms** [7]. To truly implement physics engines, Featherstone's Spatial Vector notation and the derivation of $O(N)$ ABA are unmatched.

Nonlinear Systems and Contraction

- **Applied Nonlinear Control** [22]. Slotine's classic text providing the most intuitive treatment of sliding mode and robust nonlinear stability.
- **Contraction Theory for Dynamical Systems** [4]. The modern codification of contraction bounding and metrics for trajectory stability.

Bibliography

- [1] Andrei A. Agrachev and Yuri L. Sachkov. *Control Theory from the Geometric Viewpoint*, volume 87 of *Encyclopaedia of Mathematical Sciences*. Springer, 2004.
- [2] John T. Betts. *Practical Methods for Optimal Control and Estimation Using Nonlinear Programming*. SIAM, 2010. Comprehensive treatment of practical optimal control methods.
- [3] S. Boyd, L. El Ghaoui, E. Feron, and V. Balakrishnan. *Linear Matrix Inequalities in System and Control Theory*. SIAM, 1994.
- [4] F. Bullo. *Contraction Theory for Dynamical Systems*. Kindle Direct Publishing, 2024.
- [5] F. Bullo and A. D. Lewis. *Geometric Control of Mechanical Systems*. Springer, 2004.
- [6] M. P. do Carmo. *Riemannian Geometry*. Birkhäuser, 1992.
- [7] R. Featherstone. *Rigid Body Dynamics Algorithms*. Springer, 2008.
- [8] Paul M. Fitts. The information capacity of the human motor system in controlling the amplitude of movement. *Journal of Experimental Psychology*, 47(6):381–391, 1954.
- [9] T. Flash and N. Hogan. The coordination of arm movements: an experimentally confirmed mathematical model. *Journal of Neuroscience*, 5(7):1688–1703, 1985.
- [10] David E. Goldberg. *Genetic Algorithms in Search, Optimization and Machine Learning*. Addison-Wesley, 1989.
- [11] Nikolaus Hansen and Andreas Ostermeier. Completely derandomized self-adaptation in evolution strategies. *Evolutionary Computation*, 9(2):159–195, 2001.
- [12] Christopher M. Harris and Daniel M. Wolpert. Signal-dependent noise determines motor planning. *Nature*, 394(6695):780–784, 1998.
- [13] J. Hauser and R. Hindman. Maneuver regulation from trajectory tracking: Feedback linearizable systems. In *IFAC Proceedings Volumes*, volume 28, pages 595–600, 1995.
- [14] Elwood Henneman. Relation between size of neurons and their susceptibility to discharge. *Science*, 126(3287):1345–1347, 1957.
- [15] O. Khatib. A unified approach for motion and force control of robot manipulators: The operational space formulation. *IEEE Journal on Robotics and Automation*, 3(1):43–53, 1987.
- [16] W. Lohmiller and J.-J. E. Slotine. On contraction analysis for non-linear systems. *Automatica*, 34(6):683–696, 1998.
- [17] A. Majumdar and R. Tedrake. Funnel libraries for real-time robust feedback motion planning. *The International Journal of Robotics Research*, 36(8):947–982, 2017.

- [18] I. R. Manchester and J.-J. E. Slotine. Control contraction metrics: Convex and intrinsic criteria for nonlinear feedback design. *IEEE Transactions on Automatic Control*, 62(6):3046–3053, 2017.
- [19] R. M. Murray, Z. Li, and S. S. Sastry. *A Mathematical Introduction to Robotic Manipulation*. CRC Press, 1994.
- [20] Steven M. Nesbit and Monika Serrano. Work and power analysis of the golf swing. *Journal of Sports Science and Medicine*, 4:520–533, 2005. Compatibility key for textbook chapters that cite the work-and-power golf study by author pair.
- [21] A. S. Shiriaev, L. B. Freidovich, and S. V. Gusev. Transverse linearization for controlled mechanical systems with several passive degrees of freedom. *IEEE Transactions on Automatic Control*, 55(4):893–906, 2010.
- [22] J.-J. E. Slotine and W. Li. *Applied Nonlinear Control*. Prentice Hall, 1991.
- [23] M. W. Spong, S. Hutchinson, and M. Vidyasagar. *Robot Modeling and Control*. Wiley, 2006.
- [24] Rainer Storn and Kenneth Price. Differential evolution — a simple and efficient heuristic for global optimization over continuous spaces. *Journal of Global Optimization*, 11(4):341–359, 1997.
- [25] R. Tedrake. Lqr-trees: Feedback motion planning on sparse randomized trees. In *Robotics: Science and Systems*, 2009.
- [26] E. Todorov and M. I. Jordan. Optimal feedback control as a theory of motor coordination. *Nature Neuroscience*, 5(11):1226–1235, 2002.
- [27] E. R. Westervelt, J. W. Grizzle, C. Chevallereau, J. H. Choi, and B. Morris. *Feedback Control of Dynamic Bipedal Robot Locomotion*. CRC Press, 2007.

Index

- atlas, 38
- basin of attraction, 54
- chart, 38
- composition of funnels, 86
- configuration manifold, 38
- cotangent bundle, 38
- exponential map, 38
- Floquet exponent, 54
- Floquet theory, 54
- funnel library, 86
- funnel synthesis, 86
- generalized coordinates, 38
- geodesic, 38
- invariant set, 86
- Lie algebra, 38
- Lie group, 38
- limit cycle
 - stability, 54
- logarithm map, 38
- Lyapunov function
 - time-varying, 86
- mass matrix, 38
- monodromy matrix, 54
- moving Poincaré section, 54
- orbit
 - in phase space, 54
- orbital stability, 54
- periodic orbit, 54
- phase asymptotic stability, 54
- Poincaré map, 54
- positively invariant, 86
- region of attraction, 86
- Riemannian metric, 38
- robust tube, 86
- SDP, 86
- SE(3), 38
- semidefinite programming, 86
- smooth manifold, 38
- SO(3), 38
- SOS
 - programming, 86
- stability
 - orbital vs. Lyapunov, 54
- sums-of-squares programming, 86
- tangent bundle, 38
- time-varying LQR, 86
- trajectory library, 86
- transverse dynamics, 54
- transverse linearization, 54
- TV-LQR, 86
- verified bound, 86